



Cortex-A53 ARM

体系结构与接口技术

FS6818 部分

(V1.1)

华清远见教育集团●研发中心



170-9108-5953



2306275952/2668462267



<http://dev.hqyj.com>



support@farsight.com.cn



前言

本书从嵌入式 ARM 体系结构的基础知识、系统环境搭建到综合应用，共分三个层次深入浅出地为读者拨开萦绕于嵌入式 ARM 体系结构这个概念的重重迷雾，引领读者渐渐步入嵌入式的世界，帮助探索者实实在在地把握第三次 IT 科技浪潮的方向。本书的特色如下：

- （1） 重基础，适合教学。
- （2） 重素质，全面讲解。本书在一般性教材的基础上，对嵌入式系统的软硬件开发环境进行了大量的讲解，可以让读者更进一步、更全面地了解嵌入式的开发过程。
- （3） 重实践，与实际项目相结合。本书每个原理讲解都对应着丰富的实验内容，并且原理讲解和实验内容一一对应，使得每个知识都可以被深刻的理解运用，本书附带的光盘中给出了参考设计代码和文档。
- （4） 重应用。书中的实例对时下经常使用的功能、设备、器材进行讲解和说明，力求教材所涉及的内容能紧跟行业实际应用的需要。

华清远见
fsdev.com.cn



目录

前言	I
第 1 章 ARM 开发环境搭建	- 1 -
1.1 安装 Yagarto 工具包	- 1 -
1.2 安装 GCC 编译工具	- 3 -
1.3 安装 JRE 工具	- 3 -
1.4 USB 转串口驱动安装	- 3 -
1.5 Putty 串口终端配置	- 4 -
1.5.1 错误处理	- 6 -
1.6 Eclipse for ARM 工具使用	- 7 -
1.7 导入已有工程	- 9 -
1.8 编译工程	- 10 -
第 2 章 嵌入式 ARM 技术概论	- 11 -
2.1 ARM 体系结构的技术特征及发展	- 11 -
2.1.1 ARM 公司简介	- 11 -
2.1.2 ARM 技术特征	- 12 -
2.1.3 ARM 体系架构的发展	- 12 -
2.2 ARM 微处理器简介	- 14 -
2.2.1 ARM9 处理器系列	- 15 -
2.2.2 ARM9E 处理器系列	- 16 -
2.2.3 ARM11 处理器系列	- 16 -
2.2.4 SecurCore 处理器系列	- 16 -
2.2.5 StrongARM 和 Xscale 处理器系列	- 17 -
2.2.6 MPCore 处理器系列	- 17 -
2.2.7 最新 ARM 应用处理器发展现状	- 17 -
2.3 ARM 微处理器结构	- 18 -
2.4 ARM 微处理器的应用选型	- 18 -
2.4.1 ARM 芯片选择的一般原则	- 19 -
2.4.2 选择一款适合 ARM 教学的 CPU	- 19 -
2.5 数据类型	- 22 -
2.5.1 ARM 的基本数据类型	- 22 -
2.5.2 浮点数据类型	- 23 -
2.5.3 存储器大/小端	- 23 -
2.6 Cortex-A53 内核工作模式	- 24 -
2.7 存储系统	- 25 -
2.7.1 协处理器 (CP15)	- 26 -
2.7.2 存储管理单元 (MMU)	- 27 -
2.7.3 高速缓冲存储器 (Cache)	- 27 -
2.8 流水线	- 27 -
2.8.1 流水线的概念与原理	- 27 -
2.8.2 流水线的分类	- 28 -



2.8.3 影响流水线性能的因素.....	- 29 -
2.9 寄存器组织	- 30 -
2.10 程序状态寄存器.....	- 32 -
第 3 章 ARM 微处理器指令系统	- 36 -
3.1 ARM 处理器的寻址方式.....	- 36 -
3.1.1 数据处理指令寻址方式.....	- 36 -
3.1.2 内存访问指令寻址方式.....	- 38 -
3.2 ARM 处理器的指令集.....	- 41 -
3.2.1 数据操作指令.....	- 41 -
3.2.2 乘法指令	- 48 -
3.2.3 Load/Store 指令	- 50 -
3.2.4 跳转指令	- 56 -
3.2.5 状态操作指令.....	- 59 -
3.2.6 协处理器指令.....	- 60 -
3.2.7 异常产生指令.....	- 64 -
3.2.8 其他指令介绍.....	- 65 -
第 4 章 ARM 汇编语言程序设计	- 68 -
4.1 GNU ARM 汇编器支持的伪操作	- 68 -
4.1.1 伪操作概述.....	- 68 -
4.1.2 数据定义 (Data Definition) 伪操作.....	- 68 -
4.1.3 汇编控制伪操作.....	- 69 -
4.1.4 杂项伪操作.....	- 71 -
4.2 ARM 汇编器支持的伪指令.....	- 72 -
4.2.1 ADR 伪指令.....	- 72 -
4.2.2 ADRL 伪指令.....	- 73 -
4.2.3 LDR 伪指令.....	- 73 -
4.3 GNU ARM 汇编语言的语句格式.....	- 74 -
4.4 ARM 汇编语言的程序结构.....	- 76 -
4.4.1 汇编语言的程序格式.....	- 76 -
4.4.2 汇编语言子程序调用.....	- 77 -
4.4.3 过程调用标准 AAPCS.....	- 78 -
4.4.4 汇编语言程序设计举例.....	- 79 -
4.5 汇编语言与 C 语言的混合编程.....	- 80 -
4.5.1 GNU ARM 内联汇编.....	- 80 -
4.5.2 混合编程调用举例.....	- 83 -
第 5 章 裸机实验	- 85 -
5.1 汇编点灯实验	- 85 -
5.1.1 【实验目的】	- 85 -
5.1.2 【实验环境】	- 85 -
5.1.3 【实验内容】	- 85 -
5.1.4 【实验原理】	- 85 -
5.1.5 【实验步骤】	- 86 -



5.2 蜂鸣器实验	- 88 -
5.2.1 【实验目的】	- 88 -
5.2.2 【实验环境】	- 88 -
5.2.3 【实验原理】	- 89 -
5.2.4 【实验步骤】	- 89 -
5.3 RGB 彩灯实验	- 90 -
5.3.1 【实验目的】	- 90 -
5.3.2 【实验环境】	- 90 -
5.3.3 【实验原理】	- 90 -
5.3.4 【实验步骤】	- 90 -
5.4 串口输出实验	- 91 -
5.4.1 【实验目的】	- 91 -
5.4.2 【实验环境】	- 91 -
5.4.3 【实验原理】	- 91 -
5.4.4 【实验步骤】	- 94 -
5.5 串口输入实验	- 95 -
5.5.1 【实验目的】	- 95 -
5.5.2 【实验环境】	- 95 -
5.5.3 【实验原理】	- 95 -
5.5.4 【实验步骤】	- 95 -
5.6 复位实验	- 96 -
5.6.1 【实验环境】	- 96 -
5.6.2 【实验目的】	- 96 -
5.6.3 【实验原理】	- 96 -
5.6.4 【实验步骤】	- 97 -
5.6.5 【实验目的】	- 98 -
5.7 按键中断实验	- 98 -
5.7.1 【实验环境】	- 98 -
5.7.2 【实验原理】	- 98 -
5.7.3 【实验步骤】	- 99 -
5.8 按键轮询实验	- 100 -
5.8.1 【实验目的】	- 100 -
5.8.2 【实验环境】	- 100 -
5.8.3 【实验原理】	- 100 -
5.8.4 【实验步骤】	- 101 -
5.9 I2C 重力感应/陀螺仪实验	- 102 -
5.9.1 【实验目的】	- 102 -
5.9.2 【实验原理】	- 102 -
5.9.3 【实验内容】	- 103 -
5.9.4 【实验步骤】	- 103 -
5.9.5 【实验现象】	- 104 -
5.10 A/D 实验	- 104 -



5.10.1 【实验目的】	- 104 -
5.10.2 【实验原理】	- 105 -
5.10.3 【实验内容】	- 105 -
5.10.4 【验步骤】	- 105 -
5.10.5 【实验现象】	- 106 -

华清远见
fsdev.com.cn



第 1 章 ARM 开发环境搭建

Eclipse for ARM 是借用开源软件 Eclipse 的工程管理工具，嵌入 GNU 工具集，使之能够开发 ARM 公司 Cortex-A 系列的 CPU，这里使用 Eclipse for ARM 作为开发软件。在实验箱中的配套光盘中，有相应的软件提供。光盘路径【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：工具软件\ARM 裸机开发工具】

用户需要安装编译工具，Eclipse，串口驱动等相关软件。

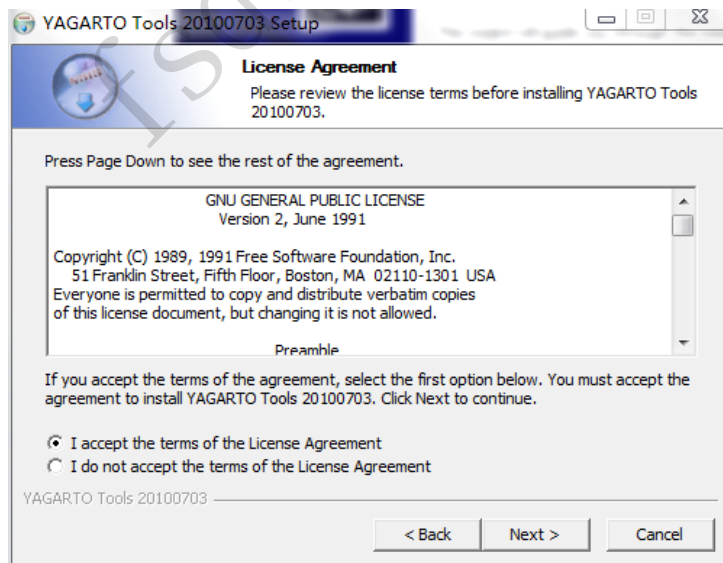
1.1 安装 Yagarto 工具包

该工具包安装后可以使用 rm, mkdir, make, touch 等命令。软件位置在【华清远见-嵌入式 ARM 实验箱资料- I -FS6818：工具软件\ARM 裸机开发工具\Yagarto 工具包】下。

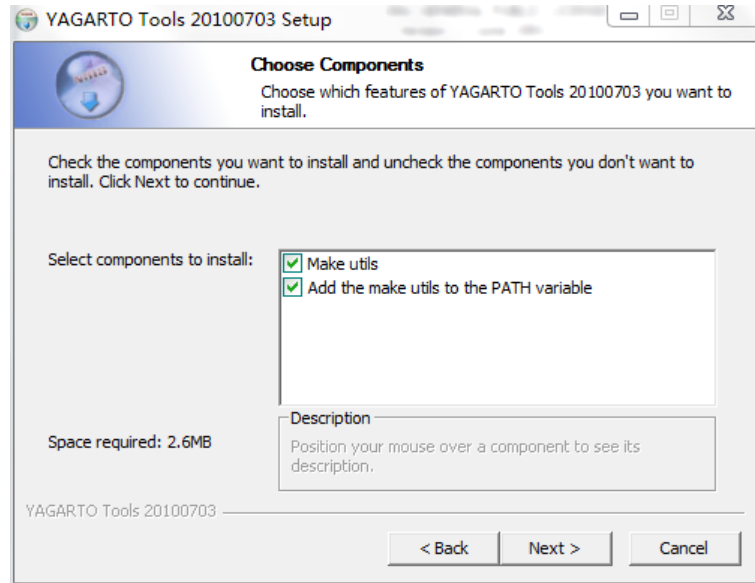
双击【yagarto-tools-20100703-setup.exe】



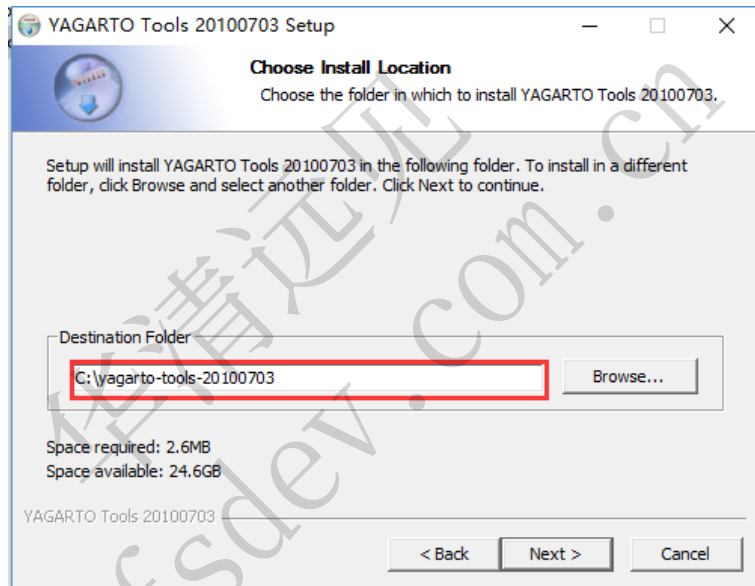
点击 Next



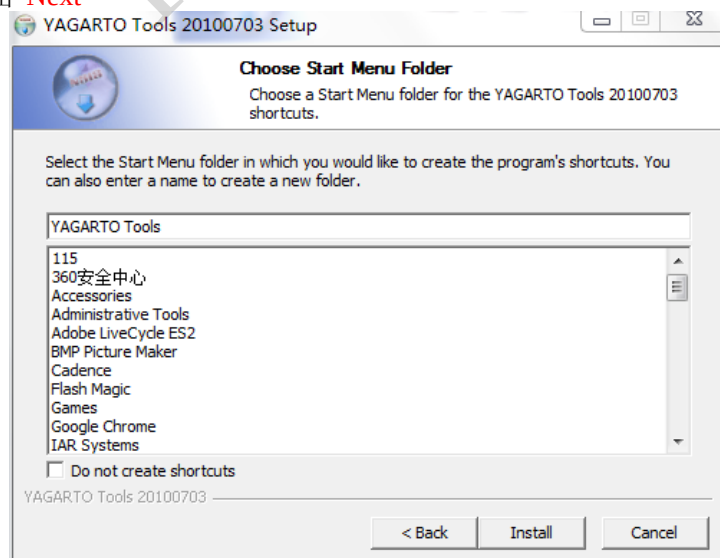
选择“I accept the terms of the License Agreement”然后点击 Next



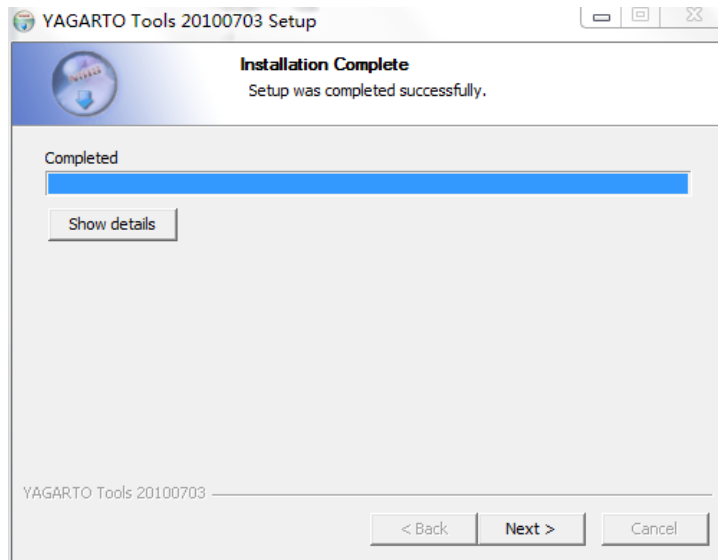
点击 **Next**，路径需要修改为没有空格以及括号的目录下，所以这里修改为C:\下



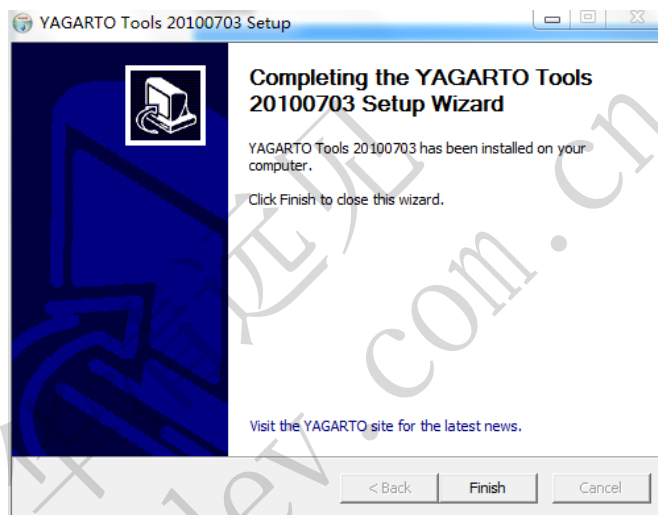
选择安装路径点击 **Next**



点击 **Install** 安装



点击 **Next**



点击 **Finish** 完成安装。

1.2 安装 GCC 编译工具

安装该工具后用户可以使用 gcc、gdb 等编译工具。软件位置在【华清远见-嵌入式 ARM 实验箱资料- I - FS6818: 工具软件\ARM 裸机开发工具\Yagarto 工具包】下

双击【yagarto-bu-2.21_gcc-4.6.2-c-c++_nl-1.19.0_gdb-7.3.1_eabi_20111119.exe】安装过程同上一节，需要注意修改安装路径。

1.3 安装 JRE 工具

JRE 为 Java Runtime Environment (Java 运行时环境) 用户需安装该工具以确保 Eclipse 可以正常运行。

运行【华清远见-嵌入式ARM实验箱资料- I -FS6818: 工具软件\ARM裸机开发工具\JRE】下的 jre-6u7-windows-i586-p-s.exe，安装时间较长，请用户耐心等待。

1.4 USB 转串口驱动安装

如果用的是华清远见标配的 CH340，运行【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:工具软件\USB 转串口驱动\CH340-USB 转串口驱动】下的 CH341SER.EXE。



等待 20 秒左右，系统会提示安装完成。可以在设备管理器中查看到串口的信息，从而确定串口号。

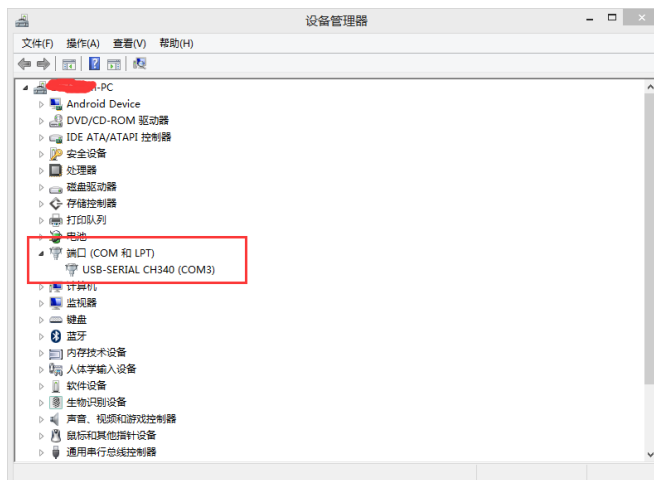


1.5 Putty 串口终端配置

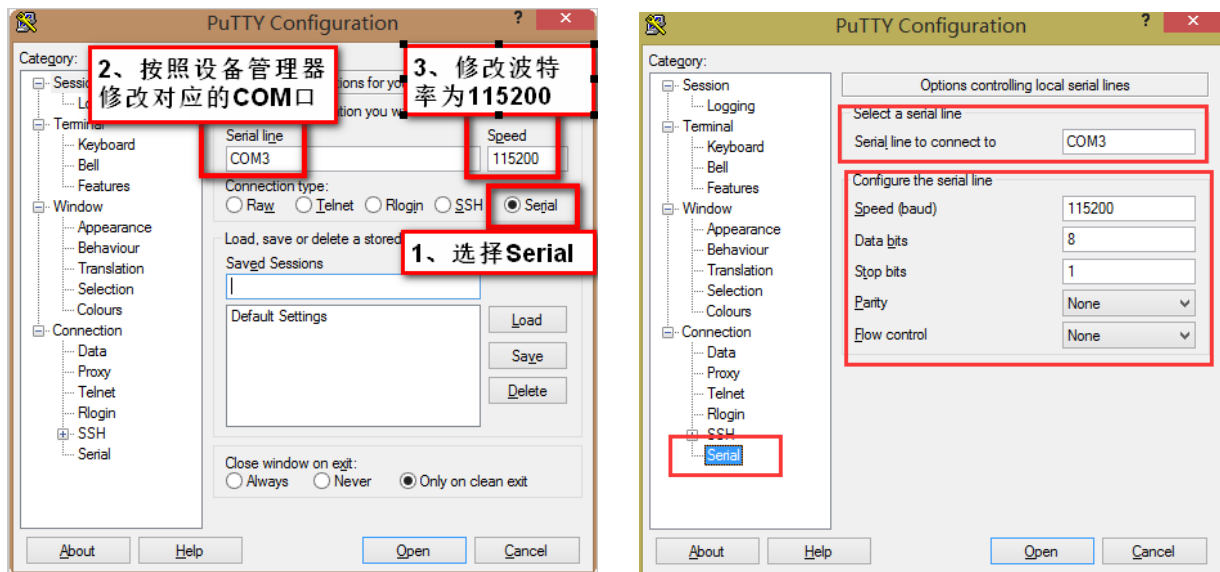
一般的开发平台都会留有系统调试的串口，目的是让用户可以通过串口和开发板进行通信，一般打开系统开关时，系统会向串口输出启动信息，此时我们接入相应的串口线（我们提供的串口线为 CH340USB 转串口线），让数据通过串口线传至 PC 机上，我们在 PC 上使用串口软件（putty）进行信息显示。Putty 在使用时需要首先配置一下，配置过程如下。

打开【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:工具软件\串口调试工具】文件。

打开设备管理器，确定当前 PC 机上的串口号。

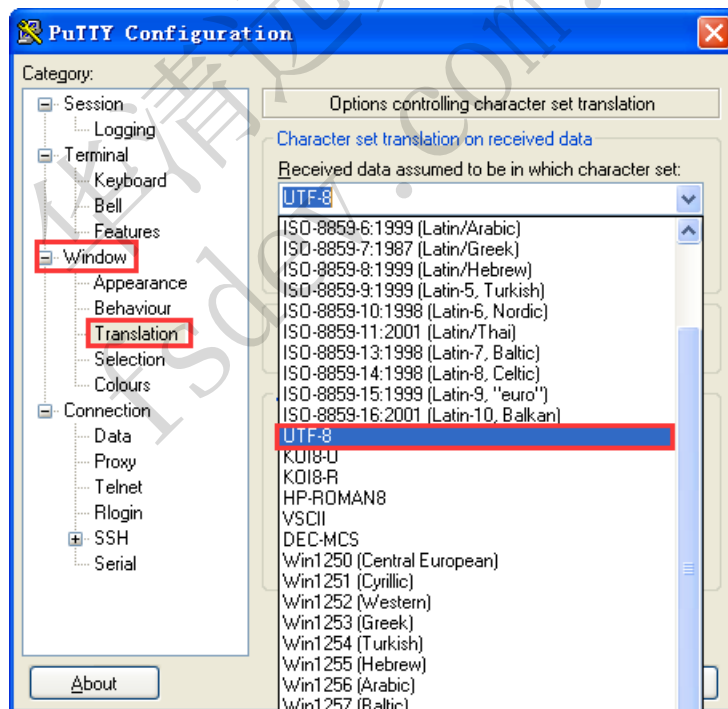


选择串口（Serial）连接方式：

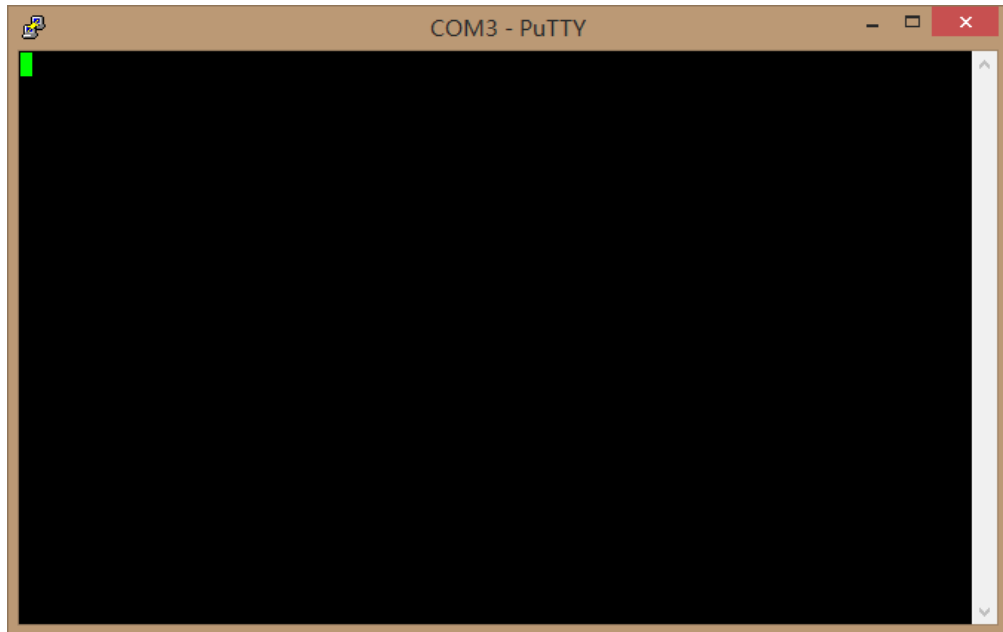


选中第一步方框内的 Serial,再点击第二步中的 Serial,进入串口设置的对话框（注意：这里如果使用台式机物理串口一般为 COM1；使用 USB 转串口线需要提前安装驱动程序，光盘中提供常用的 CH340 和 PL2303 两种驱动，路径为【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:工具软件\串口调试工具】，如果使用其他 USB 转串口线请自行安装驱动程序）。

再选中 Window 中的 Translation,在选项栏中选择文本编码格式为 UTF-8，防止出现中文乱码：



点击 open 打开串口。



打开串口后，不提示任何错误信息说明串口号是正确的，开发平台供电后，能正常显示启动信息，无乱码说明串口设置是正常的。

开发板供电之后的现象应和下面一致。

```
-----
Second Boot by Nexell Co. : Ver0.3.6 - Built on Jun 12 2015 20:06:08
-----
NSIH : Ver0.0.xx

DDR3 POR Init Start 0
phy init
Lock value   = 182
GATE CYC     = 0x00000492
GATE CODE    = 0x00000000
Read DQ      = 0x08080808
Write DQ     = 0x08080808
```

倒计时提示：

```
MIPI CLK: 420MHz
DSIM_ESCMODE 2 : 0x0
DSIM_STATUS : 0x10040f
## Skip BAT Animation.
## IRQ(0x48) : 0x00 0x00 0x00 0x00 0x00
## chg_type : ADP
## battery_vol : 0mV
## battery_cap : 100%
## Booting
Card did not respond to voltage select!
Hit any key to stop autoboot: 3 █
```

1.5.1 错误处理

据我们了解很多人在使用 putty 时会出现错误，连接串口后无任何输出信息，这里对常见的错误进行一些说明。

- 串口位置必须正确连接，应连接在 DEBUG_2 口，其他串口并不是调试串口，没有任何打印信息。
- 串口号并非一成不变，每一次插拔都会造成串口号的改变，每次打开串口需通过设备管理器确认设备号。



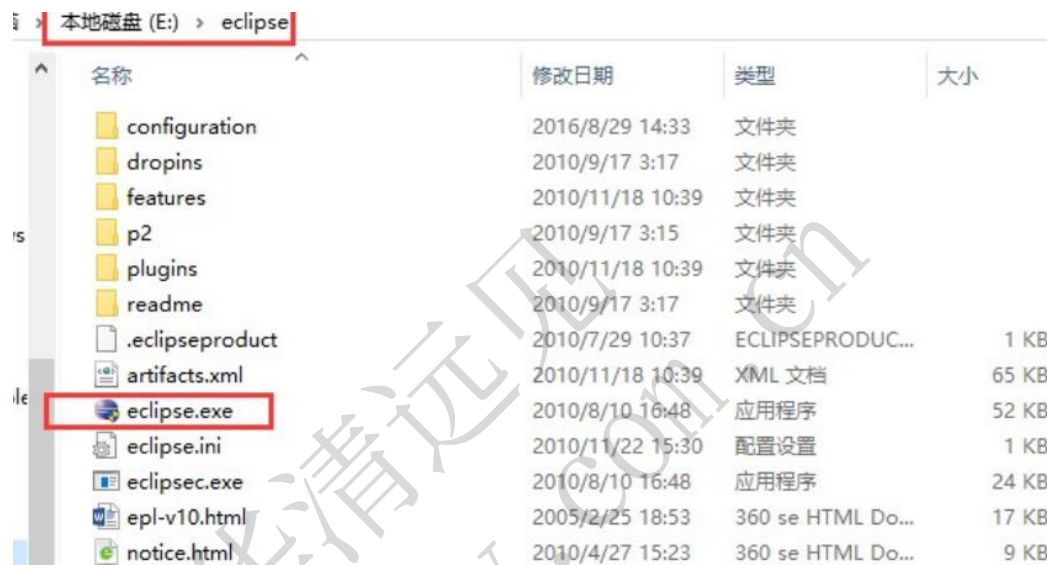
- 波特率、校验位等参数必须和图中参数一致，否则会出现乱码现象。
- 如果以上都确认是正常的，开发板供电时，确认电源指示灯是常亮状态，并确认当前的启动方式。

供电正常后，如再无启动信息，请确认核心板中是否有程序，如没有程序则可以通过制作 sd 卡启动盘并从 sd 卡启动。

1.6 Eclipse for ARM 工具使用

我们提供的 Eclipse 是一个绿色软件，并不需要用户进行安装，只需要将压缩的文件解压后即可使用。进入光盘【华清远见-嵌入式 ARM 实验箱资料-I-FS6818:工具软件\ARM 裸机开发工具\ eclipse】

拷贝 eclipse-cpp-helios-SR1-win32.zip 到用户的磁盘，例如 E 盘。选择解压到当前目录，解压后需要保证 eclipse 位于该盘的顶层目录下，如下图。



解压完成后，双击 eclipse.exe 即可打开。**(注意：在 Win7 以上的用户，使用管理员模式打开)**

指定一个工程存放目录，Eclipse for ARM 是一个标准的窗口应用程序，可以单击程序按钮开始运行。打开后必须先指定一个工程存放路径，如图所示。

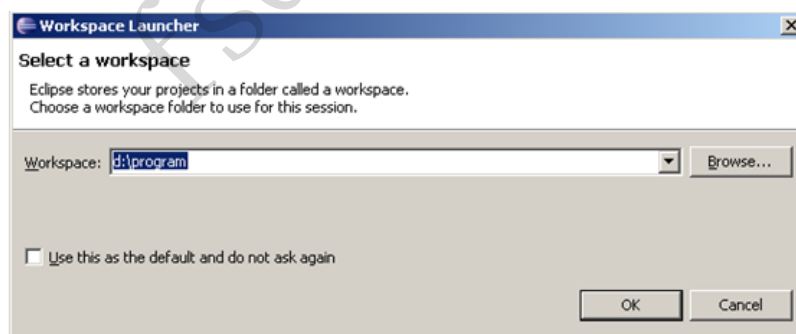


图 工程路径选择

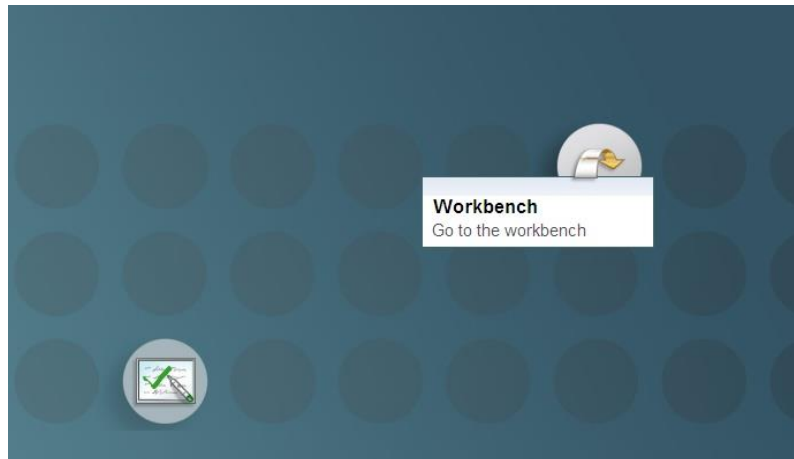


图 进入工程界面

第一次打开会出现这个界面，然后点击“Workbench”进入工程界面。弹出 工作界面。

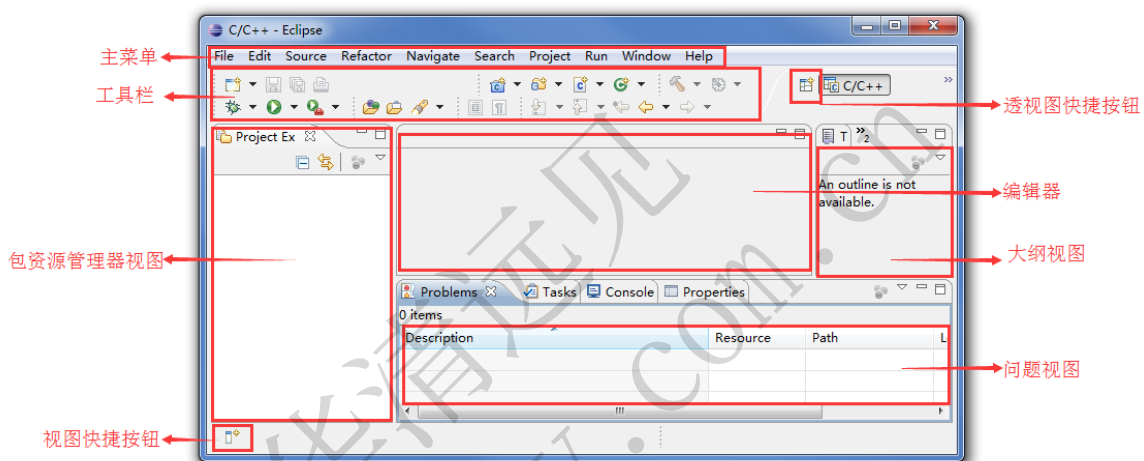
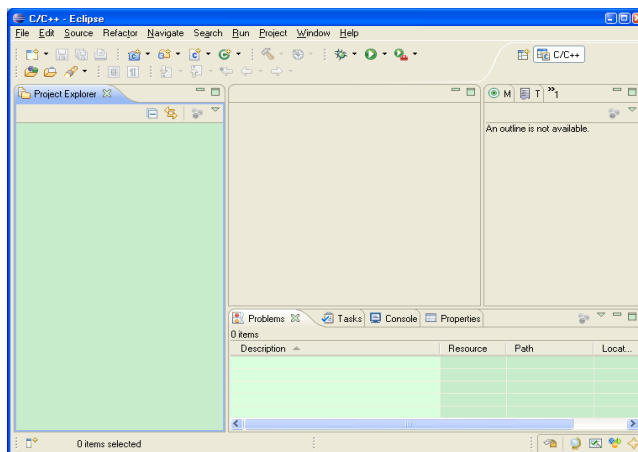


图 工程工作界面

- 主菜单：包括文件、编辑、源代码、重构、运行、窗口等菜单。大部分的向导和各种配置对话框都可以从主菜单中打开。
- 工具栏：包括文件工具栏、调试、运行等。工具栏中的按钮都是相应的菜单的快捷方式。
- 包资源管理器视图：用于显示项目中的源文件、引用的库等。
- 视图快捷按钮：用来切换到提供的其他视图。
- 透视图快捷按钮：用来切换到提供的各个透视图，提供 6 种透视图，常用的有：CVS 资源库研究、C/C++（缺省值）、调试等视图，后文调试项目即为调试视图，可通过点击此按钮返回主窗口。
- 编辑器：用于代码的编辑。
- 大纲视图：用于显示代码的纲要结构，单击结构树的各结点可以在编辑器中快速定位代码。
- 问题视图：用于显示代码或项目配置的错误，双击错误项可以快速定位代码。

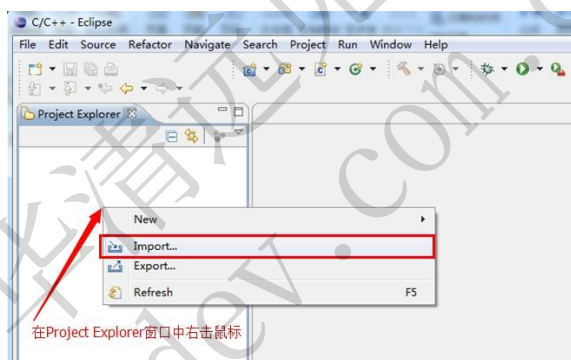


1.7 导入已有工程

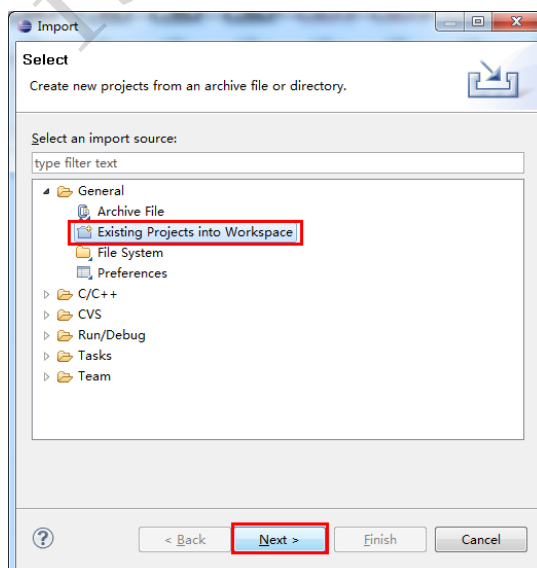
首先将我们的工程从光盘中拷贝出来，用户将【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:程序源码\ARM 裸机实验源码】下的所有代码都拷贝到工作目录下，如：D:\eclipse_projects\FS6818。

(注意：如果从光盘介质直接拷贝出来的目录可能会出现文件有只读属性的情况，建议查看文件夹的只读属性是否选中，确保在未勾选的状态下)

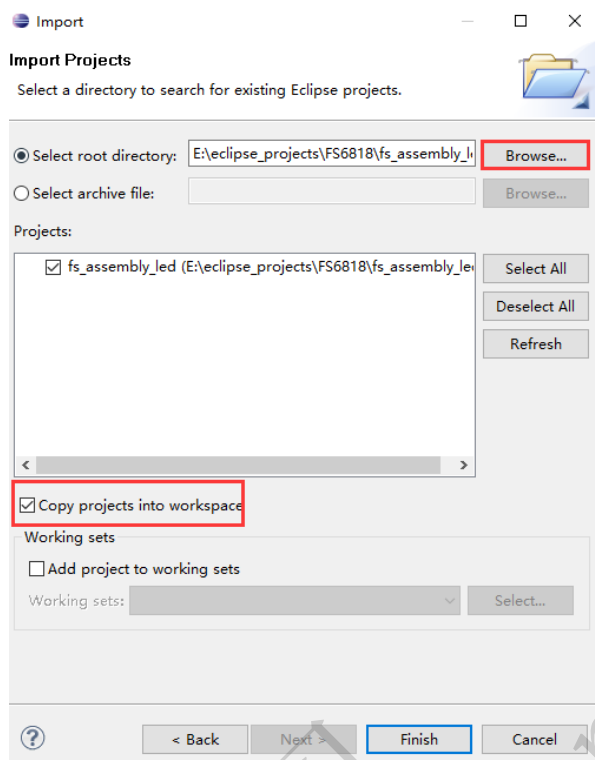
如图所示，在 Project Explorer 窗口中右击鼠标，选择“Import...”，如下图所示



选择“Import...”后，出现如下图所示的窗口，选中“Existing Project Into Workspace”，然后点击“Next”。



选择下一步之后：



点击 finish 后，工程就已经导入到 eclipse 中了，用户可以用这种方式导入所有的 ARM 工程。

1.8 编译工程

工程导入成功后，可以点击如下图所示的编译图标(或者快捷键Ctrl+B)这种方式会编译所有已经打开的工程。



如果要编译单独的工程，将鼠标放在工程名称上右键单击，菜单中会出现 Clean Project 和 Build Project 的选项，用户只需要选择即可编译当前工程。编译汇编点灯代码成功后的现象：

```
C-Build [fs_assembly_led]
**** Build of configuration Default for project fs_assembly_led ****

make all
arm-none-eabi-gcc -O0 -g -c -o start.o start.s
arm-none-eabi-ld start.o -Tmap.lds -o start.elf
arm-none-eabi-objcopy -O binary -S start.elf start.bin
```



第 2 章 嵌入式 ARM 技术概论

ARM 体系结构的处理器在嵌入式中的应用是非常广泛的，本章将向读者介绍 ARM 处理器的基本知识。通过阅读本章，读者将了解以下主要内容：

- ARM 体系结构的技术特征及发展。
- ARM 微处理器简介。
- ARM 微处理器结构。
- ARM 微处理器的应用选型。
- Cortex-A53 内部功能及特点。
- 数据类型。
- 存储系统。
- 流水线。
- 寄存器组织 S。
- 程序状态寄存器。

2.1 ARM 体系结构的技术特征及发展

ARM (Advanced RISC Machines) 有 3 种含义，它是一个公司的名称，是一类微处理器的通称，还是一种技术的名称。

2.1.1 ARM 公司简介

1991 年 ARM 公司 (Advanced RISC Machine Limited) 成立于英国剑桥，最早由 Arcon、Apple 和 VLSI 合资成立，主要出售芯片设计技术的授权，1985 年 4 月 26 日，第一个 ARM 原型在英国剑桥的 Acorn 计算机有限公司诞生 (在美国 VLSI 公司制造)。目前，ARM 架构处理器已在高性能、低功耗、低成本的嵌入式应用领域中占据了领先地位。

ARM 公司最初只有 12 人，经过多年的发展，ARM 公司已拥有近千名员工，在许多国家都设立了分公司，包括在中国上海的分公司。目前，采用 ARM 技术知识产权 (IP) 核的微处理器，即我们通常所说的 ARM 微处理器，已遍及工业控制、消费类电子产品、通信系统、网络系统、无线系统等各类产品市场，基于 ARM 技术的微处理器应用约占据了 32 位 RISC 微处理器 80% 以上的市场份额，其中，在手机市场，ARM 占有绝对的垄断地位。可以说，ARM 技术正在逐步渗入到人们生活中的各个方面，而且随着 32 位 CPU 价格的不断下降和开发环境的不断成熟，ARM 技术会应用得越来越广泛。

ARM 公司是专门从事基于 RISC 技术芯片设计开发的公司，作为嵌入式 RISC 处理器的知识产权 IP 供应商，公司本身并不直接从事芯片生产，而是靠转让设计许可由合作公司生产各具特色的芯片，世界各大半导体生产商从 ARM 公司购买其设计的 ARM 微处理器核，根据各自不同的应用领域，加入适当的外围电路，从而形成自己的 ARM 微处理器芯片进入市场，利用这种合伙关系，ARM 很快成为许多全球性 RISC 标准的缔造者。目前，全世界有几十家大的半导体公司都使用 ARM 公司的授权，其中包括 Intel、IBM、SAMSUNG、LG 半导体、NEC、SONY、PHILIP 等公司，这也使得 ARM 技术获得更多的第三方工具、制造厂商、软件的支持，又使整个系统成本降低，使产品更容易进入市场并被消费者所接受，更具有竞争力。



2.1.2 ARM 技术特征

ARM 的成功，一方面得益于它独特的公司运作模式，另一方面，当然来自于 ARM 处理器自身的优良性能。作为一种先进的 RISC 处理器，ARM 处理器有如下特点。

- 体积小、低功耗、低成本、高性能。
- 支持 Thumb（16 位）/ARM（32 位）双指令集，能很好地兼容 8 位/16 位器件。
- 大量使用寄存器，指令执行速度更快。
- 大多数数据操作都在寄存器中完成。
- 寻址方式灵活简单，执行效率高。
- 指令长度固定。

此处有必要解释一下 RISC 处理器的概念及其与 CISC 微处理器的区别。

1、嵌入式 RISC 微处理器

RISC（Reduced Instruction Set Computer）是精简指令集计算机，RISC 把着眼点放在如何使计算机的结构更加简单和如何使计算机的处理速度更加快速上。RISC 选取了使用频率最高的简单指令，抛弃复杂指令，固定指令长度，减少指令格式和寻址方式，不用或少用微码控制。这些特点使得 RISC 非常适合嵌入式处理器。

2、嵌入式 CISC 微处理器

传统的复杂指令级计算机（CISC）则更侧重于硬件执行指令的功能性，使 CISC 指令及处理器的硬件结构变得更复杂。这些会导致成本、芯片体积的增加，影响其在嵌入式产品中的应用。如表所示描述了 RISC 和 CISC 之间的主要区别。

表 RISC 和 CISC 之间主要的区别

指 标	RISC	CISC
指令集	一个周期执行一条指令，通过简单指令的组合实现复杂操作；指令长度固定	指令长度不固定，执行需要多个周期
流水线	流水线每周期前进一步	指令的执行需要调用微代码的一个微程序
寄存器	更多通用寄存器	用于特定目的的专用寄存器
Load/Store 结构	独立的 Load 和 Store 指令完成数据在寄存器和外部存储器之间的传输	处理器能够直接处理存储器中的数据

2.1.3 ARM 体系架构的发展

在讨论 ARM 体系结构前，先解释一下体系结构的定义。

体系架构定义了指令集（ISA）和基于这一体系结构下处理器的编程模型。基于同种体系结构可以有多种处理器，每个处理器性能不同，所面向的应用不同，每个处理器的实现都要遵循这一体系结构。



ARM 体系结构为嵌入系统发展商提供很高的系统性能，同时保持优异的功耗和面积效率。

ARM 体系结构为满足 ARM 合作者及设计领域的一般需求正稳步发展。目前，ARM 体系结构共定义了 7 个版本，从版本 1 到版本 7，ARM 体系的指令集功能不断扩大，不同系列的 ARM 处理器，性能差别很大，应用范围和对象也不尽相同，但是，如果是相同的 ARM 体系结构，那么基于它们的应用软件是兼容的。

1、v1 架构

V1 版本的 ARM 处理器并没有实现商品化，采用的地址空间是 26 位，寻址空间是 64MB，在目前的版本中已不再使用这种结构。

2、v2 架构

与 v1 结构的 ARM 处理器相比，v2 架构的 ARM 处理器的指令结构要有所完善，比如增加了乘法指令并且支持协处理器指令，该版本的处理器仍然采用 26 位的地址空间。

3、v3 架构

从 v3 结构开始，ARM 处理器的体系结构有了很大的改变，实现了 32 位的地址空间，指令结构相对前面的两种结构也有所完善。

4、v4 架构

v4 结构的 ARM 处理器增加了半字指令的读取和写入操作，增加了处理器系统模式，并且有了 T 变种——v4T，在 Thumb 状态下支持的是 16 位的 Thumb 指令集。属于 v4T（支持 Thumb 指令）体系结构的处理器（核）有 ARM7TDMI、ARM7TDMI-S（ARM7TDMI 综合版本）、ARM710T（ARM7TDMI 核的处理器）、ARM720T（ARM7TDMI 核的处理器）、ARM740T（ARM7TDMI 核的处理器）、ARM9TDMI、ARM910T（ARM9TDMI 核的处理器）、ARM920T（ARM9TDMI 核的处理器）、ARM940T（ARM9TDMI 核的处理器）和 StrongARM（Intel 公司的产品）。

5、v5 架构

v5 架构的 ARM 处理器提升了 ARM 和 Thumb 两种指令的交互工作能力，同时有了 DSP 指令（v5E 架构）、Java 指令（v5J 架构）的支持。属于 v5T（支持 Thumb 指令）体系结构的处理器（核）有 ARM10TDMI 和 ARM1020T（ARM10TDMI 核处理器）。

属于 v5TE（支持 Thumb、DSP 指令）体系结构的处理器（核）有 ARM9E、ARM9E-S（ARM9E 可综合版本）、ARM946（ARM9E 核的处理器）、ARM966（ARM9E 核的处理器）、ARM10E、ARM1020E（ARM10E 核处理器）、ARM1022E（ARM10E 核的处理器）和 Xscale（Intel 公司产品）。

属于 v5TEJ（支持 Thumb、DSP 指令、Java）体系结构的处理器（核）有 ARM9EJ、ARM9EJ-S（ARM9EJ 可综合版本）、ARM926EJ（ARM9EJ 核的处理器）和 ARM10EJ。

6、v6 架构

v6 架构是在 2001 年发布的，在该版本中增加了媒体指令，属于 v6 体系结构的处理器核有 ARM11



(2002 年发布)。v6 体系结构包含 ARM 体系结构中所有的 4 种特殊指令集：Thumb 指令 (T)、DSP 指令 (E)、Java 指令 (J) 和 Media 指令。

7、v7 架构

ARMv7 架构是在 ARMv6 架构的基础上诞生的。该架构采用了 Thumb-2 技术，它是在 ARM 的 Thumb 代码压缩技术的基础上发展起来的，并且保持了对现存 ARM 解决方案的完整的代码兼容性。Thumb-2 技术比纯 32 位代码少使用 31% 的内存，减小了系统开销，同时能够提供比已有的基于 Thumb 技术的解决方案高出 38% 的性能。ARMv7 架构还采用了 NEON 技术，将 DSP 和媒体处理能力提高了近 4 倍。并支持改良的浮点运算，满足下一代 3D 图形、游戏物理应用及传统嵌入式控制应用的需求。

Cortex 系列处理器是基于 ARMv7 架构的，分为 Cortex-M3、Cortex-R 和 Cortex-A3 类。

本章的 2.28 节将会例举一些 Cortex 的特性。

8、v8 架构

ARMv8 是在 32 位 ARM 架构上进行开发的，将被首先用于对扩展虚拟地址和 64 位数据处理技术有更高要求的产品领域，如企业应用、高档消费电子产品。ARMv8 架构包含两个执行状态：AArch64 和 AArch32。AArch64 执行状态针对 64 位处理技术，引入了一个全新指令集 A64，可以存取大虚拟地址空间；而 AArch32 执行状态将支持现有的 ARM 指令集。目前的 ARMv7 架构的主要特性都将在 ARMv8 架构中得以保留或进一步拓展，如 TrustZone 技术、虚拟化技术及 NEON advanced SIMD 技术等。

2.2 ARM 微处理器简介

ARM 处理器的产品系列非常广，包括 ARM7、ARM9、ARM9E、ARM10E、ARM11 和 SecurCore、Cortex 等。每个系列提供一套特定的性能来满足设计者对功耗、性能、体积的要求。SecurCore 是单独一个产品系列，是专门为安全设备而设计的。

表总结了 ARM 各系列处理器所包含的不同类型。

表 ARM 各系列处理器所包含的不同类型

ARM 系列	包含类型
ARM9/9E 系列	ARM920T
	ARM922T
	ARM926EJ-S
	ARM940T
	ARM946E-S
	ARM966E-S
	ARM968E-S



向量浮点运算（Vector Floating Point）系列	VFP9-S
	VFP10
ARM10E 系列	ARM1020E
	ARM1022E
	ARM1026EJ-S
ARM11 系列	ARM1136J-S
	ARM1136JF-S
	ARM1156T2(F)-S
	ARM1176JZ(F)-S
	ARM11 MPCore
Cortex 系列	Cortex-A
	Cortex-R
	Cortex-M
SecurCore 系列	SC100
	SC110
	SC200
	SC210
其他合作伙伴产品	StrongARM
	XScale
	MBX

本节简要介绍 ARM 各个系列处理器的特点。

2.2.1 ARM9 处理器系列

ARM9 系列于 1997 年问世。由于采用了 5 级指令流水线，ARM9 处理器能够运行在比 ARM7 更高的时钟频率上，改善了处理器的整体性能；存储器系统根据哈佛体系结构（程序和数据空间独立的体系结构）重新设计，区分了数据总线和指令总线。

ARM9 系列的第一个处理器是 ARM920T，它包含独立的数据指令 Cache 和 MMU（Memory Management Unit，存储器管理单元）。此处理器能够用在要求有虚拟存储器支持的操作系统上。该系列中的 ARM922T 是 ARM920T 的变种，只有一半大小的数据指令 Cache。



ARM940T 包含一个更小的数据指令 Cache 和一个 MPU (Micro Processor Unit, 微处理器)。它是针对不要求运行操作系统的应用而设计的。ARM920T、ARM940T 都执行 v4T 架构指令。

ARM9 系列处理器主要应用于下面一些场合:

- (1) 下一代无线设备, 包括视频电话和 PDA 等。
- (2) 数字消费品, 包括机顶盒、家庭网关、MP3 播放器和 MPEG-4 播放器。
- (3) 成像设备, 包括打印机、数码照相机和数码摄像机。
- (4) 汽车、通信和信息系统。

2.2.2 ARM9E 处理器系列

ARM9 系列的下一代处理器基于 ARM9E-S 内核, 这个内核是 ARM9 内核带有 E 扩展的一个可综合版本, 包括 ARM946E-S 和 ARM966E-S 两个变种。两者都执行 v5TE 架构指令。它们也支持可选的嵌入式跟踪宏单元, 支持开发者实时跟踪处理器指令和数据的执行。当调试对时间敏感的程序段时, 这种方法非常重要。

ARM946E-S 包括 TCM (Tightly Coupled Memory, 紧耦合存储器)、Cache 和一个 MPU。TCM 和 Cache 的大小可配置。该处理器是针对要求有确定的实时响应的嵌入式控制而设计的。ARM966E-S 有可配置的 TCM, 但没有 MPU 和 Cache 扩展。

ARM9 系列的 ARM926EJ-S 内核为可综合的处理器内核, 发布于 2000 年。它是针对小型便携式 Java 设备, 如 3G 手机和 PDA 应用而设计的。ARM926EJ-S 是第一个包含 Jazelle 技术, 可加速 Java 字节码执行的 ARM 处理器内核。它还有一个 MMU、可配置的 TCM 及具有零或非零等待存储器的数据/指令 Cache。

ARM9E 系列处理器主要应用于下面一些场合:

- (1) 下一代无线设备, 包括视频电话和 PDA 等。
- (2) 数字消费品, 包括机顶盒、家庭网关、MP3 播放器和 MPEG-4 播放器。
- (3) 成像设备, 包括打印机、数码照相机和数码摄像机。
- (4) 存储设备, 包括 DVD 或 HDD 等。
- (5) 工业控制, 包括电机控制等。
- (6) 汽车、通信和信息系统的 ABS 和车体控制。

网络设备, 包括 VoIP、WirelessLAN 等。

2.2.3 ARM11 处理器系列

ARM1136J-S 发布于 2003 年, 是针对高性能和高能效而设计的。ARM1136J-S 是第一个执行 ARMv6 架构指令的处理器。它集成了一条具有独立的 Load/Store 和算术流水线的 8 级流水线。ARMv6 指令包含了针对媒体处理的单指令流多数据流扩展, 采用特殊的设计改善视频处理能力。

2.2.4 SecurCore 处理器系列

SecurCore 系列处理器提供了基于高性能的 32 位 RISC 技术的安全解决方案。SecurCore 系列处理器除了具有体积小、功耗低、代码密度高等特点外, 还具有它自己的特别优势, 即提供了安全解决方案支



持。下面总结了 SecurCore 系列的主要特点：

- (1) 支持 ARM 指令集和 Thumb 指令集，以提高代码密度和系统性能。
- (2) 采用软内核技术以提供最大限度的灵活性，可以防止外部对其进行扫描探测。
- (3) 提供了安全特性，可以抵制攻击。
- (4) 提供面向智能卡和低成本的存储保护单元 MPU。
- (5) 可以集成用户自己的安全特性和其他的协处理器。

SecurCore 系列包含 SC100、SC110、SC200 和 SC210 四种类型。

SecurCore 系列处理器主要应用于一些安全产品及应用系统，包括电子商务、电子银行业务、网络、移动媒体和认证系统等。

2.2.5 StrongARM 和 Xscale 处理器系列

StrongARM 处理器最初是 ARM 公司与 Digital Semiconductor 公司合作开发的，现在由 Intel 公司单独许可，在低功耗、高性能的产品中应用很广泛。它采用哈佛架构，具有独立的数据和指令 Cache，有 MMU。StrongARM 是第一个包含 5 级流水线的高性能 ARM 处理器，但它不支持 Thumb 指令集。

Intel 公司的 Xscale 是 StrongARM 的后续产品，在性能上有显著改善。它执行 V5TE 架构指令，也采用哈佛结构，类似于 StrongARM 也包含一个 MMU。前面说过，Xscale 已经被 Intel 卖给了 Marvell 公司。

2.2.6 MPCore 处理器系列

MPCore 是在 ARM11 核心的基础上构建的，结构上仍属于 V6 指令体系。根据不同的需要，MPCore 可以被配置为 1 到 4 个处理器的组合方式，最高性能达到 2600 Dhrystone MIPS，运算能力几乎与 Pentium III 1GHz 处于同一水准（Pentium III 1GHz 的指令执行性能约为 2700 Dhrystone MIPS）。多核心设计的优点是在频率不变的情况下让处理器的性能获得明显提升，在多任务应用中表现尤其出色，这一点很适合未来家庭消费电子的需要。例如，机顶盒在录制多个频道电视节目的同时，还可通过互联网收看数字视频点播节目；车内导航系统在提供导航功能的同时，可以向后座乘客提供各类视频娱乐信息等。在这类应用环境下，多核心结构的嵌入式处理器将表现出极强的性能优势。

2.2.7 最新 ARM 应用处理器发展现状

(1) 从之前的 ARM 单核逐步向双核演变。作为对比，下面依次将近年来最尖端的芯片应用方案列举出来。

- NVIDIA（英伟达）的 Tegra 2 双核处理器及 Tegra 3 四核处理器，已经应用在摩托罗拉双核智能手机 ME860 及 LG Optimus 2X 手机上。
- 三星 Exynos 6818，基于 CORTEX-A53 的八核处理器。
- TI 的 OMAP4430 及 OMAP4460 双核 ARM 处理芯片，已应用在 LG Optimus 3D 手机。
- 高通 MSM8260、MSM8660（1.5G）、MSM8960（1.7G）双核处理器及 APQ8060（2.5G）四核心处理器。目前应用的代表有 HTC 的金字塔（Pyramid）双核智能手机，还有国内的小米手机。



- 苹果 A8 64 位处理器，典型代表是 iPhone6。
 - (2) 内嵌的图形显示芯片越来越强劲。
 - Mali 系列由 ARM 出品，Mali-400、Mali-T658 于 2011 年 11 月推出，支持 OpenGL ES 2.0 和 DirectX 接口，可从单核扩展到四核，可提供卓越的二维和三维加速性能。
 - PowerVR SGX 系列由 Imagination Technologies 公司出品，包括 PowerVR SGX530/535/540/543MP，支持 DirectX 9、SM3.0 和 OpenGL 2.0。
 - SGX535 被苹果公司的 iPhone4 和 iPad 采用，而 SGX540 性能更加强劲，在三星 Galaxy Tab 与魅族 M9 上采用。SGX543MP 作为新一代最强新品，目前已成为苹果 iPad 2 (SGX543MP2/双核) 和索尼 NGP (SGX543MP4/四核) 的图形内核。
 - Adreno 系列由高通公司出品，主要配合 Snapdragon CPU 使用。旗下典型方案有 Adreno200/205/220/300。
 - 在图形处理单元上，Tegra 3 从之前 Tegra 2 的 8 核心图形单元升级到 12 核心单元，NVIDIA 官方宣布将有 3 倍的图形性能提升。这 12 个处理核心的 GeForce GPU 专门为下一代移动游戏而打造 (完全兼容现有 Tegra 2 游戏)，支持更好的动态光影、物理效果和高分辨率环境。典型处理器方案有 NVIDIA Tegra 2 和 NVIDIA Tegra 3。
 - (2) 支持大 RAM，支持大数据量的存储介质。
- 现在诸多处理器已支持 DDR2、DDR3、LPDDR (mDDR) 等类型的内存。这些类型的内存高速度，高精度，并且容量也很高，已属于高速硬件之一。
- (3) 提升显示控制器性能。最高 2048×1536 分辨率液晶屏显示，如 Tegra 3 处理器。
 - (4) 提升 Camera 性能。最高支持 3200 万像素摄像头

2.3 ARM 微处理器结构

ARM 内核采用 RISC 体系结构。ARM 体系结构的主要特征如下：

- (1) 采用大量的寄存器，它们都可以用于多种用途。
- (2) 采用 Load/Store 体系结构。
- (3) 每条指令都条件执行。
- (4) 采用多寄存器的 Load/Store 指令。
- (5) 通过协处理器指令集来扩展 ARM 指令集，包括在编程模式中增加了新的寄存器和数据类型。
- (6) 通过协处理器指令集来扩展 ARM 指令集，包括在编程模式中增加了新的寄存器和数据类型。
- (7) 如果把 Thumb 指令集也当做 ARM 体系结构的一部分，那么在 Thumb 体系结构中还可以高密度 16 位压缩形式表示指令集。

2.4 ARM 微处理器的应用选型

随着国内嵌入式应用领域的发展，ARM 芯片必然会获得广泛的重视和应用。但是由于 ARM 芯片有多达几十种的芯核结构、70 多个芯片生产厂家及千变万化的内部功能配置组合，开发人员在选择方案时会有一定的困难。所以对 ARM 芯片做对比研究是十分必要的。



2.4.1 ARM 芯片选择的一般原则

1、功能

考虑处理器本身能够支持的功能，如 USB、网络、串口、液晶显示功能等。

2、性能

从处理器的功耗、速度、稳定可靠性等方面考虑。

3、价格

通常产品总是希望在完成功能要求的基础上，成本越低越好。在选择处理器时需要考虑处理的价格，及由处理器衍生出的开发价格。如开发板价格、处理器自身价格、外围芯片、开发工具、制版价格等。

4、熟悉程度及开发资源

通常公司对产品的开发周期都有严格的要求，选择一款自己熟悉的处理器可以大大降低开发风险。在自己熟悉的处理器都无法满足功能的情况下，可以尽量选择开发资源丰富的处理器。

5、操作系统支持

在选择嵌入式处理器时，如果最终的程序需要运行在操作系统上，那么还应该考虑处理器对操作系统的支持。

6、升级

很多产品在开发完成后都会面临升级的问题，正所谓人无远虑必有近忧。所以在选择处理器时必须要考虑升级的问题。如尽量选择具有相同封装的不同性能等级的处理器；考虑产品未来可能增加的功能。

7、供货稳定

供货稳定也是选择处理器时的一个重要参考因素，尽量选择大厂家，比较通用的芯片。

2.4.2 选择一款适合 ARM 教学的 CPU

在 ARM 教学中，在选择 CPU 作为学习目标时，主要从芯片功能、开发平台价格、开发资源等方面考虑。

1、ARM 芯核

如果希望学习使用 Windows CE 或 Linux 等操作系统，就需要选择 ARM720T 以上带有 MMU（Memory Management Unit）功能的 ARM 芯片，ARM720T、StrongARM、Cortex-A 系列处理器都带有 MMU 功能。而 ARM7TDMI 没有 MMU，不支持 Windows CE 和大部分的 Linux。目前，uClinux 及 Linux 2.6 内核等 Linux 系统不需要 MMU 的支持。



2、系统时钟速度

系统时钟决定了 ARM 芯片的处理速度。ARM7 的处理速度为 0.97MIPS/MHz，常见的 ARM7 芯片系统主时钟为 20~133MHz，ARM9 的处理速度为 1.1MIPS/MHz，常见的 ARM9 的系统主时钟为 100~233MHz。Cortex-A 系列的主时钟频率也越来越快，如 Cortex-A9 主频率可以达到 1.6GHz 以上，如果希望学习可以支持较为复杂的操作系统的芯片时，可以选择 ARM9 及 ARM9 以上的芯片。

3、支持内存访问的类型

支持内存访问的类型如表所示。

表 支持内存访问的类型

芯片名	是否有 SDRAM	是否有 DDR2	是否有 mDDR	是否有 DDR3
S3C2410	是	否	否	否
S3C2440	是	否	否	否
S5PV210	否	是	否	否
S5PV310	否	否	否	是
EXYNOS6818	否	否	否	是

4、USB 接口

USB 接口产品的使用越来越广泛，许多 ARM 芯片内置 USB 控制器，有些芯片甚至同时有 USB Host 和 USB Slave 控制器。表显示了内置 USB 控制器的 ARM 芯片。

表 内置 USB 控制器的 ARM 芯片

芯片型号	ARM 内核	供应商	USB (otg)	USB Host
S3C2410	ARM920T	SAMSUNG	1	2
S3C2440	ARM920T	SAMSUNG	1	2
EXYNOS210	CORTEX-A8	SAMSUNG	1	1
S5PV310	CORTEX-A9	SAMSUNG	1	1
EXYNOS6818	CORTEX-A53	SAMSUNG	1	3



5、GPIO 数量

在某些芯片供应商提供的说明书中，往往申明的是最大可能的 GPIO 数量，但是有许多引脚是和地址线、数据线、串口线等引脚复用的。这样在系统设计时需要计算实际可以使用的 GPIO 数量。

6、中断控制器

ARM 内核只提供快速中断（FIQ）和标准中断（IRQ）两个中断向量。但各个半导体厂家在设计芯片时加入了自己定义的中断控制器，以便支持诸如串行口、外部中断、时钟中断等硬件中断。外部中断控制是选择芯片时必须考虑的重要因素，合理的外部中断设计可以很大程度地减少任务调度工作量。例如 PHILIPS 公司的 SAA7750，所有 GPIO 都可以设置成 FIQ 或 IRQ，并且可以选择上升沿、下降沿、高电平和低电平 4 种中断方式。这使得红外线遥控接收、指轮盘和键盘等任务都可以作为背景程序运行。而 Cirrus Logic 公司的 EP7312 芯片只有 4 个外部中断源，并且每个中断源都只能是低电平或高电平中断，这样接收红外线信号的场合必须用查询方式，浪费大量 CPU 时间。

7、IIS（Integrate Interface of Sound）接口

IIS 接口即集成音频接口。如果设计音频应用产品，IIS 接口是必需的。

8、nWAIT 信号

这是一个外部总线速度控制信号。不是每个 ARM 芯片都提供这个信号引脚，利用这个信号与廉价 GAL 芯片就可以实现符合 PCMCIA 标准的 WLAN 卡和 BlueTooth 卡的接口，而不需要外加高成本的 PCMCIA 专用控制芯片。另外，当需要扩展外部 DSP 协处理器时，此信号也是必需的。

9、RTC（Real Time Clock）

很多 ARM 芯片都提供 RTC（实时时钟）功能，但方式不同。如 Cirrus Logic 公司的 EP7312 的 RTC 只是一个 32 位计数器，需要通过软件计算出年月日时分秒；而 SAA7750 和 S3C2410 等芯片的 RTC 直接提供年月日时分秒格式。

10、LCD 控制器

有些 ARM 芯片内置 LCD 控制器，有的甚至内置 64KB 彩色 TFT LCD 控制器。在设计 PDA 和手持式显示记录设备时，选用内置 LCD 控制器的 ARM 芯片（如 S3C2410）较为适宜。

11、PWM 输出

有些 ARM 芯片有 2~8 路 PWM 输出，可以用于电机控制或语音输出等场合。

12、ADC 和 DAC

有些 ARM 芯片内置 2~8 通道 8~12 位通用 ADC，可以用于电池检测、触摸屏和温度监测等。PHILIPS 的 SAA7750 更是内置了一个 16 位立体声音频 ADC 和 DAC，并且带耳机驱动。

13、扩展总线



大部分 ARM 芯片具有外部 SDRAM 和 SRAM 扩展接口，不同的 ARM 芯片可以扩展的芯片数量即片选线数量不同，外部数据总线有 8 位、16 位或 32 位。为某些特殊应用设计的 ARM 芯片（如德国 Micronas 的 PUC3030A）没有外部扩展功能。

14、 UART 和 IrDA

几乎所有的 ARM 芯片都具有 1~2 个 UART 接口，可以用于和 PC 通信或用 Angel 进行调试。一般的 ARM 芯片通信波特率为 115200bit/s，少数专为蓝牙技术应用设计的 ARM 芯片的 UART 通信波特率可以达到 920kbit/s，如 Linkup 公司 L7205。

15、 时钟计数器和看门狗

一般 ARM 芯片都具有 2~4 个 16 位或 32 位时钟计数器和一个看门狗计数器。

16、 电源管理功能

ARM 芯片的耗电量与工作频率成正比，一般 ARM 芯片都有低功耗模式、睡眠模式和关闭模式。

17、 DMA 控制器

有些 ARM 芯片内部集成 DMA（Direct Memory Access）接口，可以和硬盘等外部设备高速交换数据，同时减少数据交换时对 CPU 资源的占用。

另外，可以选择的内部功能部件还有 HDLC、SDLC、CD-ROM Decoder、Ethernet MAC、VGA controller 和 DC-DC。可以选择的内置接口有：IIC、SPDIF、CAN、SPI、PCI 和 PCMCIA。

18、 封装类型

最后需说明的是封装问题。ARM 芯片现在主要的封装有 QFP、TQFP、PQFP、LQFP、BGA、LBGA 等形式，BGA 封装具有芯片面积小的特点，可以减少 PCB 的面积，但是需要专用的焊接设备，无法手工焊接。另外，一般 BGA 封装的 ARM 芯片无法用双面板完成 PCB 布线，需要多层 PCB 板布线。

最后，院校的实际情况结合当前及未来一段时间的市场人才需求，经过综合考虑，本书教学选取的是三星公司的 Exynos6818 芯片。Exynos6818 是一款基于 Cortex-A53 核心的微处理器芯片。

2.5 数据类型

2.5.1 ARM 的基本数据类型

ARM 采用的是 32 位架构，ARM 的基本数据类型有以下 3 种。

- Byte：字节，8bit。
- Halfword：半字，16bit（半字必须与 2 字节边界对齐）。
- Word：字，32bit（字必须与 4 字节边界对齐）。

存储器可以看做是序号为 $0 \sim 2^{32}-1$ 的线性字节阵列。如图 2-1 所示为 ARM 存储器的组织结构。其中每一个字节都有唯一的地址。字节可以占用任一位置，图中给出了几个例子。长度为 1 个字

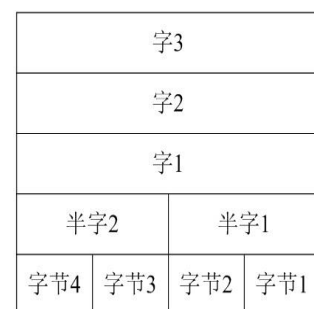


图 2-1 ARM 存储器组织结构



的数据项占用一组 4 字节的位置，该位置开始于 4 的倍数的字节地址（地址最末两位为 00）。半字占有两个字节的位置，该位置开始于偶数字节地址（地址最末一位为 0）。

注意：

- (1) ARM 系统结构 v4 以上版本支持以上 3 种数据类型，v4 以前版本仅支持字节和字。
- (2) 当将这些数据类型中的任意一种声明成 unsigned 类型时， n 位数据值表示范围为 $0 \sim 2^n - 1$ 的非负数，通常使用二进制格式。
- (3) 当将这些数据类型的任何一种声明成 signed 类型时， n 位数据值表示范围为 $-2^{n-1} \sim 2^{n-1} - 1$ 的整数，使用二进制的补码格式。
- (4) 所有数据类型指令的操作数都是字类型的，如“ADD r1, r0, #0x1”中的操作数“0x1”就是以字类型数据处理的。
- (5) Load/Store 数据传输指令可以从存储器存取传输数据，这些数据可以是字节、半字、字。加载时自动进行字节或半字的零扩展或符号扩展。对应的指令分别为 LDR/BSTRB（字节操作）、LDRH/STRH（半字操作）、LDR/STR（字操作）。详见后面的指令参考。

ARM 指令编译后是 4 个字节（与字边界对齐）。Thumb 指令编译后是 2 个字节（与半字边界对齐）。

2.5.2 浮点数据类型

浮点运算使用在 ARM 硬件指令集中未定义的数据类型。尽管如此，但 ARM 公司在协处理器指令空间定义了一系列浮点指令。通常这些指令全部可以通过未定义指令异常（此异常收集所有硬件协处理器不接受的协处理器指令）在软件中实现，但是其中的一小部分也可以由浮点运算协处理器 FPA10 以硬件方式实现。另外，ARM 公司还提供了用 C 语言编写的浮点库作为 ARM 浮点指令集的替代方法（Thumb 代码只能使用浮点指令集）。该库支持 IEEE 标准的单精度和双精度格式。C 编译器有一个关键字标志来选择这个历程。它产生的代码与软件仿真（通过避免中断、译码和浮点指令仿真）相比既快又紧凑。

2.5.3 存储器大/小端

从软件角度看，内存相对于一个大的字节数组，其中每个数组元素（字节）都是可寻址的。

ARM 支持大端模式（big-endian）和小端模式（little-endian）两种内存模式。

如图所示显示了大端模式和小端模式数据存放特点。



图 大小端模式存放数据的特点

下面的例子显示了使用内存大/小端（big/little endian）的存取格式。

程序执行前：



```
r0=0x11223344
```

执行指令：

```
r1=0x100
STR r0, [r1]
LDRB r2, [r1]
```

执行后：

```
小端模式下：r2=0x44
大端模式下：r2=0x11
```

上面的例子向我们提示了一个潜在的编程隐患。在大端模式下，一个字的高地址放的是数据的低位，而在小端模式下，数据的低位放在内存中的低地址。要小心对待存储器中一个字内字节的顺序。

2.6 Cortex-A53 内核工作模式

Cortex-A53 基于 ARMv8 架构，共有 8 种工作模式，如表所示。

表 S5P6818 处理器的工作模式

处理器工作模式	简 写	描 述
用户模式 (User)	usr	正常程序执行模式，大部分任务执行在这种模式下
快速中断模式 (FIQ)	fiq	当一个高优先级 (fast) 中断产生时将会进入这种模式，一般用于高速数据传输和通道处理
外部中断模式 (IRQ)	irq	当一个低优先级 (normal) 中断产生时将会进入这种模式，一般用于通常的中断处理
特权模式 (Supervisor)	svc	当复位或软中断指令执行时进入这种模式，是一种供操作系统使用的保护模式
数据访问中止模式 (Abort)	abt	当存取异常时将会进入这种模式，用于虚拟存储或存储保护
未定义指令中止模式 (Undef)	und	当执行未定义指令时进入这种模式，有时用于通过软件仿真协处理器硬件的工作方式
系统模式 (System)	sys	使用 and User 模式相同寄存器集的模式，用于运行特权级操作系统任务
监控模式 (Monitor)	mon	可以在安全模式与非安全模式之间进行转换

除用户模式外的其他 7 种处理器模式称为特权模式 (Privileged Modes)。在特权模式下，程序可以访问所有的系统资源，也可以任意地进行处理器模式切换。其中以下 6 种又称为异常模式：

- (1) 快速中断模式 (FIQ)。



- (2) 外部中断模式 (IRQ)。
- (3) 特权模式 (Supervior)。
- (4) 数据访问中止模式 (Abort)。
- (5) 未定义指令中止模式 (Undef)。
- (6) 监控模式 (Monitor)。

处理器模式可以通过软件控制进行切换, 也可以通过外部中断或异常处理过程进行切换。

大多数的用户程序运行在用户模式下。当处理器工作在用户模式时, 应用程序不能够访问受操作系统保护的一些系统资源, 应用程序也不能直接进行处理器模式切换。当需要进行处理器模式切换时, 应用程序可以产生异常处理, 在异常处理过程中进行处理器模式切换。这种体系结构可以使操作系统控制整个系统资源的使用。

当应用程序发生异常中断时, 处理器进入相应的异常模式。在每一种异常模式中都有一组专用寄存器以供相应的异常处理程序使用, 这样就可以保证在进入异常模式时用户模式下的寄存器 (保存程序运行状态) 不被破坏。

2.7 存储系统

ARM 存储系统有非常灵活的体系结构, 可以适应不同的嵌入式应用系统的需要。ARM 存储器系统可以使用简单的平板式地址映射机制 (就像一些简单的单片机一样, 地址空间的分配方式是固定的, 系统中各部分都使用物理地址), 也可以使用其他技术提供功能更为强大的存储系统。例如:

- (1) 系统可能提供多种类型的存储器件, 如 Flash、ROM、SRAM 等。
- (2) Cache 技术。
- (3) 写缓存技术 (Write Buffers)。
- (4) 虚拟内存和 I/O 地址映射技术。

大多数的系统通过下面的方法之一可实现对复杂存储系统的管理。

- (1) 使用 Cache, 缩小处理器和存储系统速度差别, 从而提高系统的整体性能。
- (2) 使用内存映射技术实现虚拟空间到物理空间的映射。这种映射机制对嵌入式系统非常重要。通常嵌入式系统程序存放在 ROM/Flash 中, 这样系统断电后程序能够得到保存。但是, 通常 ROM/Flash 与 SDRAM 相比, 速度慢很多, 而且基于 ARM 的嵌入式系统中通常把异常中断向量表放在 RAM 中。利用内存映射机制可以满足这种需要。在系统加电时, 将 ROM/Flash 映射为地址 0, 这样可以进行一些初始化处理; 当这些初始化处理完成后将 SDRAM 映射为地址 0, 并把系统程序加载到 SDRAM 中运行, 这样可很好地满足嵌入式系统的需要。

- (3) 引入存储保护机制, 增强系统的安全性。

- (4) 引入一些机制保证将 I/O 操作映射成内存操作后, 各种 I/O 操作能够得到正确的结果。在简单存储系统中, 不存在这样的问题。而当系统引入了 Cache 和 write buffer 后, 就需要一些特别的措施。

在 ARM 系统中, 要实现对存储系统的管理通常使用协处理器 CP15, 它通常也被称为系统控制协处理器 (System Control Coprocessor)。

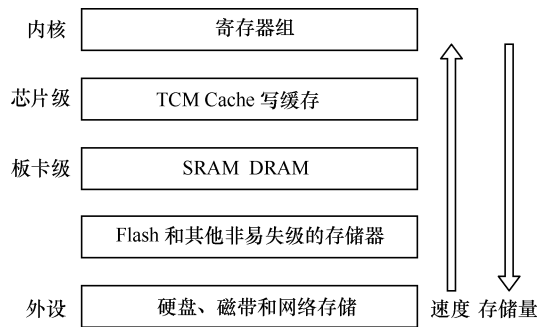


图 存储器的层次结构

每级都有特定的存储介质，下面对比各级系统中特定存储介质的存储性能。

- (1) 内核级的寄存器。处理器寄存器组可看做是存储器层次的顶层。这些寄存器被集成在处理器内核中，在系统中提供最快的存储器访问。典型的 ARM 处理器有多个 32 位寄存器，其访问时间为 ns 量级。
- (2) 芯片级的紧耦合存储器（TCM）是为弥补 Cache 访问的不确定性增加的存储器。TCM 是一种快速 SDRAM，它紧挨内核，并且保证取指和数据操作的时钟周期数，这一点对一些要求确定行为的实时算法是很重要的。TCM 位于存储器地址映射中，可作为快速存储器来访问。
- (3) 芯片级的片上 Cache 存储器的容量在 8KB~32KB 之间，访问时间大约为 10ns。高性能的 ARM 结构中，可能存在第二级片外 Cache，容量为几百 KB，访问时间为几十 ns。
- (4) 板卡级的 DRAM。主存储器可能是几 MB 到几十 MB 的动态存储器，访问时间大约为 100ns。
- (5) 外设级的后援存储器，通常是硬盘，可能从几百 MB 到几个 GB，访问时间为几十 ms。

2.7.1 协处理器（CP15）

ARM 处理器支持 16 个协处理器。在程序执行过程中，每个协处理器忽略属于 ARM 处理器和其他协处理器的指令。当一个协处理器硬件不能执行属于它的协处理器指令时，将产生一个未定义指令异常中断，在该异常中断处理程序中，可以通过软件模拟该硬件操作。例如，如果系统不包含向量浮点运算器，则可以选择浮点运算软件模拟包来支持向量浮点运算。CP15 即通常所说的系统控制协处理器（System Control Coprocessor），它负责完成大部分的存储系统管理。除了 CP15 外，在具体的各种存储管理机制中可能还会用到其他一些技术，如在 MMU 中除了 CP15 外，还使用了页表技术等。

在一些没有标准存储管理的系统中，CP15 是不存在的。在这种情况下，针对 CP15 的操作指令将被视为未定义指令，指令的执行结果不可预知。

CP15 包含 16 个 32 位寄存器，其编号为 0~15。实际上对于某些编号的寄存器可能对应多个物理寄存器，在指令中指定特定的标志位来区分这些物理寄存器。这种机制有些类似于 ARM 中的寄存器，当处于不同的处理器模式时，某些相同编号的寄存器对应于不同的物理寄存器。

CP15 中的寄存器可能是只读的，也可能是只写的，还有一些是可读/可写的。在对协处理器寄存器进行操作时，需要注意以下几个问题：

- (1) 寄存器的访问类型（只读/只写/可读可写）。
- (2) 不同的访问引发不同的功能。



- (3) 相同编号的寄存器是否对应不同的物理寄存器。
- (4) 寄存器的具体作用。

2.7.2 存储管理单元 (MMU)

MMU 提供的一个关键服务是使各个任务作为各自独立的程序在自己的私有存储空间中运行。在带 MMU 的操作系统控制下，运行的任务无须知道其他与之无关的任务的存储需求情况，这就简化了各个任务的设计。

MMU 提供了一些资源以允许使用虚拟存储器（将系统物理存储器重新编址，可将其看成一个独立于系统物理存储器的存储空间）。MMU 作为转换器，将程序和数据的虚拟地址（编译时的连接地址）转换成实际的物理地址，即在物理主存中的地址。这个转换过程允许运行的多个程序使用相同的虚拟地址，而各自存储在物理存储器的不同位置。

这样存储器就有两种类型的地址：虚拟地址和物理地址。虚拟地址由编译器和连接器在定位程序时分配；物理地址用来访问实际的主存硬件模块（物理上程序存在的区域）。

2.7.3 高速缓冲存储器 (Cache)

Cache 是一个容量小但存取速度非常快的存储器，它保存最近用到的存储器数据副本。对于程序员来说，Cache 是透明的。它自动决定保存哪些数据、覆盖哪些数据。现在 Cache 通常与处理器在同一芯片上实现。Cache 能够发挥作用是因为程序具有局部性。所谓局部性就是指在任何特定的时间，处理器趋于对相同区域的数据（如堆栈）多次执行相同的指令（如循环）。

Cache 经常与写缓存器（write buffer）一起使用。写缓存器是一个非常小的先进先出（FIFO）存储器，位于处理器核与主存之间。使用写缓存的目的是，将处理器核和 Cache 从较慢的主存写操作中解脱出来。当 CPU 向主存储器做写入操作时，它先将数据写入到写缓存区中，由于写缓存器的速度很高，这种写入操作的速度也将很高。写缓存区在 CPU 空闲时，以较低的速度将数据写入到主存储器中相应的位置。

通过引入 Cache 和写缓存区，存储系统的性能得到了很大的提高，但同时也带来了一些问题。例如，由于数据将存在于系统中不同的物理位置，可能造成数据的不一致性；由于写缓存区的优化作用，可能有些写操作的执行顺序不是用户期望的顺序，从而造成操作错误。

2.8 流水线

2.8.1 流水线的概念与原理

处理器按照一系列步骤来执行每一条指令，典型的步骤如下：

- (1) 从存储器读取指令（fetch）。
- (2) 译码以鉴别它属于哪一条指令（decode）。
- (3) 从指令中提取指令的操作数（这些操作数往往存在于寄存器 reg 中）。
- (4) 将操作数进行组合以得到结果或存储器地址（ALU）。
- (5) 如果需要，则访问存储器以存储数据（mem）。
- (6) 将结果写回到寄存器堆（res）。



并不是所有的指令都需要上述每一个步骤，但是，多数指令需要其中的多个步骤。这些步骤往往使用不同的硬件功能，如 ALU 可能只在第 4 步中用到。因此，如果一条指令不是在前一条指令结束之前就开始，那么在每一步骤内处理器只有少部分的硬件在使用。

有一种方法可以明显改善硬件资源的使用率和处理器的吞吐量，这就是在当前一条指令结束之前就开始执行下一条指令，即通常所说的流水线（Pipeline）技术。流水线是 RISC 处理器执行指令时采用的机制。使用流水线，可在取下一条指令的同时译码和执行其他指令，从而加快执行的速度。可以把流水线看做是汽车生产线，每个阶段只完成专门的处理器任务。

采用上述操作顺序，处理器可以这样来组织：当一条指令刚刚执行完步骤（1）并转向步骤（2）时，下一条指令就开始执行步骤（1）。从原理上说，这样的流水线应该比没有重叠的指令执行快 6 倍，但由于硬件结构本身的一些限制，实际情况会比理想状态差一些。

2.8.2 流水线的分类

1、3 级流水线 ARM 组织

到 ARM7 为止的 ARM 处理器使用简单的 3 级流水线，它包括下列流水线级。

- （1） 取指令（fetch）：从寄存器装载一条指令。
- （2） 译码（decode）：识别被执行的指令，并为下一个周期准备数据通路的控制信号。在这一级，指令占有译码逻辑，不占用数据通路。
- （3） 执行（execute）：处理指令并将结果写回寄存器。

如图所示为 3 级流水线指令的执行过程。



图 3 级流水线

当处理器执行简单的数据处理指令时，流水线使得平均每个时钟周期能完成 1 条指令。但 1 条指令需要 3 个时钟周期来完成，因此，有 3 个时钟周期的延时（latency），但吞吐率（throughput）是每个周期 1 条指令。

2、5 级流水线 ARM 组织

所有的处理器都要满足对高性能的要求，直到 ARM7 为止，在 ARM 核中使用的 3 级流水线的性价比是很高的。但是，为了得到更高的性能，需要重新考虑处理器的组织结构。有两种方法来提高性能。

- （1） 提高时钟频率。时钟频率的提高，必然引起指令执行周期的缩短，所以要求简化流水线每一级的逻辑，流水线的级数就要增加。
- （2） 减少每条指令的平均指令周期数 CPI。这就要求重新考虑 3 级流水线 ARM 中多于 1 个流水线周期的实现方法，以便使其占有较少的周期，或者减少因指令相关造成的流水线停顿，也可以将两者结合起来。

3 级流水线 ARM 核在每一个时钟周期都访问存储器，或者取指令，或者传输数据。只是抓紧存储器



不用的几个周期来改善系统性能，效果并不明显。为了改善 CPI，存储器系统必须在每个时钟周期中给出多于一个的数据。方法是在每个时钟周期从单个存储器中给出多于 32 位数据，或者为指令或数据分别设置存储器。

基于以上原因，较高性能的 ARM 核使用了 5 级流水线，而且具有分开的指令和数据存储器。把指令的执行分割为 5 部分而不是 3 部分，进而可以使用更高的时钟频率，分开的指令和数据存储器使核的 CPI 明显减少。

在 ARM9TDMI 中使用了典型的 5 级流水线，5 级流水线包括下面的流水线级。

(1) 取指令 (fetch): 从存储器中取出指令，并将其放入指令流水线。

(2) 译码 (decode): 指令被译码，从寄存器堆中读取寄存器操作数。在寄存器堆中有 3 个操作数读端口，因此，大多数 ARM 指令能在 1 个周期内读取其操作数。

(3) 执行 (execute): 将其中 1 个操作数移位，并在 ALU 中产生结果。如果指令是 Load 或 Store 指令，则在 ALU 中计算存储器的地址。

缓冲/数据 (buffer/data): 如果需要则访问数据存储器，否则 ALU 只是简单地缓冲 1 个时钟周期。

回写 (write-back): 将指令的结果回写到寄存器堆，包括任何从寄存器读出的数据。

如图所示列出了 5 级流水线指令的执行过程。



图 5 级流水线

在程序执行过程中，PC 值是基于 3 级流水线操作特性的。5 级流水线中提前 1 级来读取指令操作数，得到的值是不同的 (PC + 4 而不是 PC + 8)。这里产生代码不兼容是不容许的。但 5 级流水线 ARM 完全仿真 3 级流水线的行为。在取指级增加的 PC 值被直接送到译码级的寄存器，穿过两级之间的流水线寄存器。下一条指令的 PC + 4 等于当前指令的 PC + 8，因此，未使用额外的硬件便得到了正确的 R15。

3、13 级流水线

在 Cortex-A8 中有一条 13 级的流水线，但是由于 ARM 公司没有对其中的技术公开任何相关的细节，这里只能简单介绍一下，从经典 ARM 系列到现在的 Cortex 系列，ARM 处理器的结构在向复杂的阶段发展，但没改变的是 CPU 的取指指令和地址关系，不管是几级流水线，都可以按照最初的 3 级流水线的操作特性来判断其当前的 PC 位置。这样做主要还是为了软件兼容性上的考虑，由此可以判断的是，后面 ARM 所推出的处理核心都想满足这一特点，感兴趣的读者可以自行查阅相关资料。

2.8.3 影响流水线性能的因素

1、互锁

在典型的程序处理过程中，经常会遇到这样的情形，即一条指令的结果被用做下一条指令的操作数。例如，有如下指令序列：

```
LDR R0, [R0, #0]
ADD R0, R0, R1    ; 在 5 级流水线上产生互锁
```

从例子可以看出，流水线的操作产生中断，因为第 1 条指令的结果在第 2 条指令取数时还没有产



生。第 2 条指令必须停止，直到结果产生为止。

2、跳转指令

跳转指令也会破坏流水线的行为，因为后续指令的取指步骤受到跳转目标计算的影响，因而必须推迟。但是，当跳转指令被译码时，在它被确认是跳转指令之前，后续的取指操作已经发生。这样一来，已经被预取进入流水线的指令不得被丢弃。如果跳转目标的计算是在 ALU 阶段完成的，那么在得到跳转目标之前已经有两条指令按原有指令流读取。

显然，只有当所有指令都依照相似的步骤执行时，流水线的效率达到最高。如果处理器的指令非常复杂，每一条指令的行为都与下一条指令不同，那么就很难用流水线实现。

2.9 寄存器组织

ARM 处理器有如下 40 个 32 位长的寄存器。

33 个通用寄存器。

6 个状态寄存器：1 个 CPSR（Current Program Status Register，当前程序状态寄存器），6 个 SPSR（Saved Program Status Register，备份程序状态寄存器）。

1 个 PC（Program Counter，程序计数器）。

ARM 处理器共有 7 种不同的处理器模式，在每一种处理器模式中都有一组相应的寄存器组，如图 2-6 所示列出了 ARM 处理器的寄存器组织概要。



图 寄存器列表

当前处理器的模式决定着哪组寄存器可操作，任何模式都可以存取下列寄存器。

(1) 相应的 R0~R12。

相应的 R13（Stack Pointer, SP, 栈指向）和 R14（the Link Register, LR, 链路寄存器）。



相应的 R15 (PC)。

相应的 CPSR。

特权模式 (除 System 模式外) 还可以存取相应的 SPSR。

通用寄存器根据其分组与否可分为以下两类。

- (1) 未分组寄存器 (Unbanked Register), 包括 R0~R7。
- (2) 分组寄存器 (Banked Register), 包括 R8~R14。

1、未分组寄存器

未分组寄存器包括 R0~R7。顾名思义, 在所有处理器模式下对于每一个未分组寄存器来说, 指的都是同一个物理寄存器。未分组寄存器没有被系统用于特殊的用途, 任何可采用通用寄存器的应用场合都可以使用未分组寄存器。但由于其通用性, 在异常中断所引起的处理器模式切换时, 其使用的是相同的物理寄存器, 所以也就很容易使寄存器中的数据被破坏。

2、分组寄存器

R8~R14 是分组寄存器, 它们每一个访问的物理寄存器取决于当前的处理器模式。

对于分组寄存器 R8~R12 来说, 每个寄存器对应两个不同的物理寄存器。一组用于除 FIQ 模式外的所有处理器模式, 而另一组则专门用于 FIQ 模式。这样的结构设计有利于加快 FIQ 的处理速度。不同模式下寄存器的使用, 要使用寄存器名后缀加以区分。例如, 当使用 FIQ 模式下的寄存器时, 寄存器 R8 和寄存器 R9 分别记为 R8_fiq、R9_fiq; 当使用用户模式下的寄存器时, 寄存器 R8 和 R9 分别记为 R8_usr、R9_usr 等。在 ARM 体系结构中, R8~R12 没有任何指定的其他的用途, 所以当 FIQ 中断到达时, 不用保存这些通用寄存器, 也就是说, FIQ 处理程序可以不必执行保存和恢复中断现场的指令, 从而使中断处理过程非常迅速。所以 FIQ 模式常被用来处理一些时间紧急的任务, 如 DMA 处理。

对于分组寄存器 R13 和 R14 来说, 每个寄存器对应 6 个不同的物理寄存器。其中的一个是用户模式和系统模式公用的, 而另外 5 个分别用于 5 种异常模式。访问时需要指定它们的模式。名字形式如下:

R13_<mode>

R14_<mode>

其中, <mode>可以是以下几种模式之一: usr、svc、abt、und、irp、fiq 及 mon。

R13 寄存器在 ARM 处理器中常用做堆栈指针, 称为 SP。当然, 这只是一种习惯用法, 并没有任何指令强制性的使用 R13 作为堆栈指针, 用户完全可以使用其他寄存器作为堆栈指针。而在 Thumb 指令集中, 有一些指令强制性地使用 R13 作为堆栈指针, 如堆栈操作指令。

每一种异常模式拥有自己的 R13。异常处理程序负责初始化自己的 R13, 使其指向该异常模式专用的栈地址。在异常处理程序入口处, 将用到的其他寄存器的值保存在堆栈中, 返回时, 重新将这些值加载到寄存器。通过这种保护程序现场的方法, 异常不会破坏被其中断的程序现场。

寄存器 R14 又被称为连接寄存器 (Link Register, LR), 在 ARM 体系结构中具有下面两种特殊的作用。

每一种处理器模式用自己的 R14 存放当前子程序的返回地址。当通过 BL 或 BLX 指令调用子程序时, R14 被设置成该子程序的返回地址。在子程序返回时, 把 R14 的值复制到程序计数器 (PC)。典型的做



法是使用下列两种方法之一。

- a) 执行下面任何一条指令。

```
MOV PC, LR
BX LR
```

- b) 在子程序入口处使用下面的指令将 PC 保存到堆栈中。

```
STMFD SP!, {<register>,LR}
```

在子程序返回时，使用如下相应的配套指令返回。

```
LDMFD SP!, {<register>,PC}
```

当异常中断发生时，该异常模式特定的物理寄存器 R14 被设置成该异常模式的返回地址，对于有些模式 R14 的值可能与返回地址有一个常数的偏移量（如数据异常使用 SUB PC, LR, #8 返回）。具体的返回方式与上面的子程序返回方式基本相同，但使用的指令稍微有些不同，以保证当异常出现时正在执行的程序的状态被完整保存。

R14 也可以被用做通用寄存器使用。

2.10 程序状态寄存器

当前程序状态寄存器（Current Program Status Register, CPSR）可以在任何处理器模式下被访问，它包含下列内容：

ALU（Arithmetic Logic Unit，算术逻辑单元）状态标志的备份。

当前的处理器模式。

中断使能标志。

设置处理器的状态。

每一种处理器模式下都有一个专用的物理寄存器做备份程序状态寄存器（Saved Program Status Register, SPSR）。当特定的异常中断发生时，这个物理寄存器负责存放当前程序状态寄存器的内容。当异常处理程序返回时，再将其内容恢复到当前程序状态寄存器。

CPSR 寄存器（和保存它的 SPSR 寄存器）中的位分配如图所示。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
N	Z	C	V	Q	IT [1:0]	J	保留	GE[3:0]	IT[7:2]	E	A	I	F	T	M[4:0]																

图 程序状态寄存器格式

下面给出各个状态位的定义。

1、标志位

N（Negative）、Z（Zero）、C（Carry）和 V（oVerflow）通称为条件标志位。这些条件标志位会根据程序中的算术指令或逻辑指令的执行结果进行修改，而且这些条件标志位可由大多数指令检测以决定指令是否执行。

在 ARM 4T 架构中，所有的 ARM 指令都可以条件执行，而 Thumb 指令却不能。

各条件标志位的具体含义如下。



1) N

本位设置成当前指令运行结果的 bit[31] 的值。当两个由补码表示的有符号整数运算时， $N = 1$ 表示运算的结果为负数， $N = 0$ 表示结果为正数或零。

2) Z

$Z = 1$ 表示运算的结果为零， $Z = 0$ 表示运算的结果不为零。

3) C

下面分 4 种情况讨论 C 的设置方法。

(1) 在加法指令中（包括比较指令 CMN），当结果产生了进位，则 $C = 1$ ，表示无符号数运算发生上溢出；其他情况下 $C = 0$ 。

在减法指令中（包括比较指令 CMP），当运算中发生错位（即无符号数运算发生下溢出），则 $C = 0$ ；其他情况下 $C = 1$ 。

对于在操作数中包含移位操作的运算指令（非加/减法指令），C 被设置成被移位寄存器最后移出去的位。

对于其他非加/减法运算指令，C 的值通常不受影响。

4) V

下面分两种情况讨论 V 的设置方法。

(2) 对于加/减运算指令，当操作数和运算结果都是以二进制的补码表示的带符号的数时，且运算结果超出了有符号运算的范围是溢出。 $V = 1$ 表示符号位溢出。

对于非加/减法指令，通常不改变标志位 V 的值（具体可参照 ARM 指令手册）。

尽管以上 C 和 V 的定义看起来颇为复杂，但使用时在大多数情况下用一个简单的条件测试指令即可，不需要程序员计算出条件码的精确值即可得到需要的结果。

2、Q 标志位

在带 DSP 指令扩展的 ARM v5 及更高版本中，bit[27] 被指定用于指示增强的 DAP 指令是否发生了溢出，因此也就被称为 Q 标志位。同样，在 SPSR 中 bit[27] 也被称为 Q 标志位，用于在异常中断发生时保存和恢复 CPSR 中的 Q 标志位。

在 ARM v5 以前的版本及 ARM v5 的非 E 系列处理器中，Q 标志位没有被定义，属于待扩展的位。

3、控制位

CPSR 的低 8 位（I、F、T 及 M[4:0]）统称为控制位。当异常发生时，这些位的值将发生相应的变化。另外，如果在特权模式下，也可以通过软件编程来修改这些位的值。

1) 中断禁止位

$I = 1$ ，IRQ 被禁止。

$F = 1$ ，FIQ 被禁止。

2) 状态控制位

T 位是处理器的状态控制位。



T = 0，处理器处于 ARM 状态（即正在执行 32 位的 ARM 指令）。

T = 1，处理器处于 Thumb 状态（即正在执行 16 位的 Thumb 指令）。

当然，T 位只有在 T 系列的 ARM 处理器上才有效，在非 T 系列的 ARM 版本中，T 位将始终为 0。

3) 模式控制位

M[4:0]作为位模式控制位，这些位的组合确定了处理器处于哪种状态。如表所示列出了其具体含义。

只有表中列出的组合是有效的，其他组合无效。

表 状态控制位 M[4:0]

M[4 : 0]	处理器模式	可以访问的寄存器
0b10000	User	PC, R14~R0, CPSR
0b10001	FIQ	PC, R14_fiq~R8_fiq, R7~R0, CPSR, SPSR_fiq
0b10010	IRQ	PC, R14_irq~R13_irq, R12~R0, CPSR, SPSR_irq
0b10011	Supervisor	PC, R14_svc~R13_svc, R12~R0, CPSR, SPSR_svc
0b10111	Abort	PC, R14_abt~R13_abt, R12~R0, CPSR, SPSR_abt
0b11011	Undefined	PC, R14_und~R13_und, R12~R0, CPSR, SPSR_und
0b11111	System	PC, R14~R0, CPSR (ARM v4 及更高版本)
0b10110	Secure monitor	PC,R0-R12, CPSR,SPSR_mon,r13_mon,r14_mon

1、IF-THEN 标志位

CPSR 中的 bits[15:10,26:25]称为 if-then 标志位，它用于对 thumb 指令集中 if-then-else 这一类语句块的控制。

其中 IT[7:5]定义为基本条件，如图所示。

IT[4:0]被定义为 IF-THEN 语句块的长度。

	[7:5]	[4]	[3]	[2]	[1]	[0]	
控制基础	P1	P2	P3	P4	1		4 指令IT块入口点
控制基础	P1	P2	P3	1	0		3 指令IT块入口点
控制基础	P1	P2	1	0	0		2 指令IT块入口点
控制基础	P1	1	0	0	0		1 指令IT块入口点
000	0	0	0	0	0		普通执行状态，无IT块入口点

图 IF-THEN 标志位[7:5]的定义



2、E 位/A 位/GE[19-16]位的定义

A 表示异步异常禁止位。

E 表示大小端控制位，0 表示小端操作，1 表示大端操作。注意，该位在预取阶段是被忽略的。

GE[19-16]用于表示在 SIMD 指令集中的大于、等于标志。在任何模式下该位可读可写。

华清远见
fsdev.com.cn



第 3 章 ARM 微处理器指令系统

ARM 指令集可以分为跳转指令、数据处理指令、程序状态寄存器传输指令、Load/Store 指令、协处理器指令和异常中断产生指令。根据使用的指令类型不同，指令的寻址方式分为数据处理指令寻址方式和内存访问指令寻址方式。

本章主要介绍 ARM 汇编语言。主要内容如下：

➤ ARM 处理器的寻址方式。

ARM 处理器的指令集。

3.1 ARM 处理器的寻址方式

ARM 指令的寻址方式分为数据处理指令寻址方式和内存访问指令寻址方式。

3.1.1 数据处理指令寻址方式

数据处理指令的基本语法格式如下：

```
<opcode> {<cond>} {S} <Rd>, <Rn>, <shifter_operand>
```

其中，<shifter_operand>有 11 种形式，如表所示。

表 <shifter_operand>的寻址方式

	语 法	寻 址 方 式
1	#<immediate>	立即数寻址
2	<Rm>	寄存器寻址
3	<Rm>, LSL #<shift_imm>	立即数逻辑左移
4	<Rm>, LSL <Rs>	寄存器逻辑左移
5	<Rm>, LSR #<shift_imm>	立即数逻辑右移
6	<Rm>, LSR <Rs>	寄存器逻辑右移
7	<Rm>, ASR #<shift_imm>	立即数算术右移
8	<Rm>, ASR <Rs>	寄存器算术右移
9	<Rm>, ROR #<shift_imm>	立即数循环右移
10	<Rm>, ROR <Rs>	寄存器循环右移
11	<Rm>, RRX	寄存器扩展循环右移



数据处理指令寻址方式可以分为以下几种。

- (1) 立即数寻址方式。
- (2) 寄存器寻址方式。
- (3) 寄存器移位寻址方式。

1、立即数寻址方式

指令中的立即数是由一个 8bit 的常数移动 4bit 偶数位 (0, 2, 4, ..., 26, 28, 30) 得到的。所以，每一条指令都包含一个 8bit 的常数 X 和移位值 Y，得到的立即数 = X 循环右移 (2×Y)，如图所示。



图 立即数表示方法

下面列举了一些有效的立即数：

0xFF、0x104、0xFF0、0xFF00、0xFF000、0xFF000000、0xF000000F

下面是一些无效的立即数：

0x101、0x102、0xFF1、0xFF04、0xFF003、0xFFFFFFFF、0xF000001F

下面是一些应用立即数的指令：

```
MOV R0, #0           ;送 0 到 R0
ADD R3, R3, #1        ;R3 的值加 1
CMP R7, #1000         ;将 R7 的值和 1000 比较
BIC R9, R8, #0xFF00   ;将 R8 中 8~15 位清零，结果保存在 R9 中
```

2、寄存器寻址方式

寄存器的值可以被直接用于数据操作指令，这种寻址方式是各类处理器经常采用的一种方式，也是一种执行效率较高的寻址方式，如：

```
MOV R2, R0           ;R0 的值送 R2
ADD R4, R3, R2        ;R2 加 R3，结果送 R4
CMP R7, R8            ;比较 R7 和 R8 的值
```

3、寄存器移位寻址方式

寄存器的值在被送到 ALU 之前，可以事先经过桶形移位寄存器的处理。预处理和移位发生在同一周期内，所以有效地使用移位寄存器，可以增加代码的执行效率。

下面是一些在指令中使用了移位操作的例子：

```
ADD R2, R0, R1, LSR #5
MOV R1, R0, LSL #2
RSB R9, R5, R5, LSL #1
SUB R1, R2, R0, LSR #4
MOV R2, R4, ROR R0
```



3.1.2 内存访问指令寻址方式

内存访问指令的寻址方式可以分为以下几种。

- (1) 字及无符号字节的 Load/Store 指令的寻址方式。
- (2) 杂类 Load/Store 指令的寻址方式。
- (3) 批量 Load/Store 指令的寻址方式。
- (4) 协处理器 Load/Store 指令的寻址方式。

1、字及无符号字节的 Load/Store 指令的寻址方式

字及无符号字节的 Load/Store 指令语法格式如下：

`LDR|STR{<cond>}{B}{T} <Rd>, <addressing_mode>`

其中，<addressing_mode>共有 9 种寻址方式，如表所示。

表 字及无符号字节的 Load/Store 指令的寻址方式

	格 式	模 式
1	<code>[Rn, #±<offset_12>]</code>	立即数偏移寻址 (Immediate offset)
2	<code>[Rn, ±Rm]</code>	寄存器偏移寻址 (Register offset)
3	<code>[Rn, Rm, <shift>#<offset_12>]</code>	带移位的寄存器偏移寻址 (Scaled register offset)
4	<code>[Rn, #±<offset_12>]!</code>	立即数前索引寻址 (Immediate pre-indexed)
5	<code>[Rn, ±Rm]!</code>	寄存器前索引寻址 (Register post-indexed)
6	<code>[Rn, Rm, <shift>#<offset_12>]!</code>	带移位的寄存器前索引寻址 (Scaled register pre-indexed)
7	<code>[Rn], #±<offset_12></code>	立即数后索引寻址 (Immediate post-indexed)
8	<code>[Rn], ±<Rm></code>	寄存器后索引寻址



		(Register post-indexed)
9	[Rn], ±<Rm>, <shift>#< offset_12>	带移位的寄存器后索引寻址 (Scaled register post-indexed)

上表中，“!”表示完成数据传输后要更新基址寄存器。

1、杂类 Load/Store 指令的寻址方式

使用该类寻址方式的指令的语法格式如下：

LDR|STR{<cond>}H|SH|SB|D <Rd>, <addressing_mode>

使用该类寻址方式的指令包括（有符号/无符号）半字 Load/Store 指令、有符号字节 Load/Store 指令和双字 Load/Store 指令。

该类寻址方式分为 6 种类型，如表所示。

表 杂类 Load/Store 指令的寻址方式

	格 式	模 式
1	[Rn, #±<offset_8>]	立即数偏移寻址 (Immediate offset)
2	[Rn, ±Rm]	寄存器偏移寻址 (Register offset)
3	[Rn, #±<offset_8>]!	立即数前索引寻址 (Immediate pre-indexed)
4	[Rn, ±Rm]!	寄存器前索引寻址 (Register post-indexed)
5	[Rn], #±<offset_8>	立即数后索引寻址 (Immediate post-indexed)
6	[Rn], ±<Rm>	寄存器后索引寻址 (Register post-indexed)

2、批量 Load/Store 指令寻址方式

批量 Load/Store 指令将一片连续内存单元的数据加载到通用寄存器组中或将一组通用寄存器的数据存储在内存单元中。

批量 Load/Store 指令的寻址模式产生一个内存单元的地址范围，指令寄存器和内存单元的对应关系满足这样的规则，即编号低的寄存器对应于内存中低地址单元，编号高的寄存器对应于内存中的高地址



单元。

该类指令的语法格式如下：

```
LDM|STM{<cond>}<addressing_mode> <Rn>{!},<registers><^>
```

该类指令的寻址方式如表所示。

表 批量 Load/Store 指令的寻址方式

	格 式	模 式
1	IA (Increment After)	后递增方式
2	IB (Increment Before)	先递增方式
3	DA (Decrement After)	后递减方式
4	DB (Decrement Before)	先递减方式

3、堆栈操作寻址方式

堆栈操作寻址方式和批量 Load/Store 指令寻址方式十分类似。但对于堆栈的操作，数据写入内存和从内存中读出要使用不同的寻址模式，因为进栈操作（pop）和出栈操作（push）要在不同的方向上调整堆栈。

下面详细讨论如何使用合适的寻址方式实现数据的堆栈操作。

根据不同的寻址方式，将堆栈分为以下 4 种。

- (1) Full 栈：堆栈指针指向栈顶元素（last used location）。
- (2) Empty 栈：堆栈指针指向第一个可用元素（the first unused location）。
- (3) 递减栈：堆栈向内存地址减小的方向生长。
- (4) 递增栈：堆栈向内存地址增加的方向生长。

根据堆栈的不同种类，将其寻址方式分为以下 4 种。

- (5) 满递减 FD (Full Descending)。
- (6) 空递减 ED (Empty Descending)。
- (7) 满递增 FA (Full Ascending)。
- (8) 空递增 EA (Empty Ascending)。

如表所示列出了堆栈的寻址方式和批量 Load/Store 指令寻址方式的对应关系。

表 堆栈寻址方式和批量 Load/Store 指令寻址方式的对应关系

批量数据寻址方式	堆栈寻址方式	L 位	P 位	U 位
LDMDA	LDMFA	1	0	0
LDMIA	LDMFD	1	0	1



LDMDB	LDMEA	1	1	0
LDMIB	LDMED	1	1	1
STMDA	STMED	0	0	0
STMIA	STMEA	0	0	1
STMDB	STMFD	0	1	0
STMIB	STMFA	0	1	1

1、协处理器 Load/Store 寻址方式

协处理器 Load/Store 指令的语法格式如下：

```
<opcode>{<cond>}{L} <coproc>,<CRd>,<addressing_mode>
```

3.2 ARM 处理器的指令集

3.2.1 数据操作指令

数据操作指令是指对存放在寄存器中的数据进行操作的指令。主要包括数据传送指令、算术指令、逻辑指令、比较与测试指令及乘法指令。

如果在数据处理指令前使用 S 前缀，指令的执行结果将会影响 CPSR 中的标志位。数据处理指令如表所示。

表 数据处理指令列表

助 记 符	操 作	行 为
MOV	数据传送	
MVN	数据取反传送	
AND	逻辑与	$Rd: =Rn \text{ AND } op2$
EOR	逻辑异或	$Rd: =Rn \text{ EOR } op2$
SUB	减	$Rd: =Rn - op2$
RSB	翻转减	$Rd: =op2 - Rn$
ADD	加	$Rd: =Rn + op2$



ADC	带进位的加	$Rd: =Rn + op2 + C$
SBC	带进位的减	$Rd: =Rn - op2 + C - 1$
RSC	带进位的翻转减	$Rd: =op2 - Rn + C - 1$
TST	测试	$Rn \text{ AND } op2$ 并更新标志位
TEQ	测试相等	$Rn \text{ EOR } op2$ 并更新标志位
CMP	比较	$Rn - op2$ 并更新标志位
CMN	负数比较	$Rn + op2$ 并更新标志位
ORR	逻辑或	$Rd: =Rn \text{ OR } op2$
BIC	位清 0	$Rd: =Rn \text{ AND NOT } (op2)$
BIC	位清 0	$Rd: =Rn \text{ AND NOT } (op2)$

1、MOV 指令

MOV 指令是最简单的 ARM 指令，执行的结果就是把一个数 N 送到目标寄存器 Rd，其中 N 可以是寄存器，也可以是立即数。

MOV 指令多用于设置初始值或者在寄存器间传送数据。

MOV 指令将移位码（shifter_operand）表示的数据传送到目的寄存器 Rd，并根据操作的结果更新 CPSR 中相应的条件标志位。

(1) 指令的语法格式：

```
MOV{<cond>} {S} <Rd>, <shifter_operand>
```

(2) 指令举例：

```
MOV    R0, R0          ; R0 = R0... NOP 指令
MOV    R0, R0, LSL#3   ; R0 = R0 * 8
```

如果 R15 是目的寄存器，将修改程序计数器或标志。这用于被调用的子函数结束后返回到调用代码，方法是把连接寄存器的内容传送到 R15。

```
MOV    PC, R14          ; 退出到调用者，用于普通函数返回，PC 即是 R15
MOVS   PC, R14          ; 退出到调用者并恢复标志位，用于异常函数返回
```

(3) 指令的使用如下。

MOV 指令主要完成以下功能。

- 将数据从一个寄存器传送到另一个寄存器。
- 将一个常数值传送到寄存器中。
- 实现无算术和逻辑运算的单纯移位操作，操作数乘以 2^n 可以用左移 n 位来实现。



d) 当 PC (R15) 用做目的寄存器时, 可以实现程序跳转。如 “MOV PC, LR”, 所以这种跳转可以实现子程序调用及从子程序返回, 代替指令 “B, BL”。

当 PC 作为目标寄存器且指令中 S 位被设置时, 指令在执行跳转操作的同时, 将当前处理器模式的 SPSR 寄存器的内容复制到 CPSR 中。这种指令 “MOVS PC LR” 可以实现从某些异常中断中返回。

2、MVN 指令

MVN 是反相传送 (Move Negative) 指令。它将操作数的反码传送到目的寄存器。

MVN 指令多用于向寄存器传送一个负数或生成位掩码。

MVN 指令将 shifter_operand 表示的数据的反码传送到目的寄存器 Rd, 并根据操作结果更新 CPSR 中相应的条件标志位。

(1) 指令的语法格式:

```
MNV{<cond>}{S} <Rd>,<shifter_operand>
```

(2) 指令举例如下。

MVN 指令和 MOV 指令相同, 也可以把一个数 N 送到目标寄存器 Rd, 其中 N 可以是立即数, 也可以是寄存器。这是逻辑非操作而不是算术操作, 这个取反的值加 1 才是它的取负的值。

```
MVN    R0, #4           ; R0 = -5
MVN    R0, #0           ; R0 = -1
```

(3) 指令的使用如下。

MVN 指令主要完成以下功能:

- (a) 向寄存器中传送一个负数。
- (b) 生成位掩码 (Bit Mask)。
- (c) 求一个数的反码。

3、AND 指令

AND 指令将 shifter_operand 表示的数值与寄存器 Rn 的值按位 (bitwise) 做逻辑与操作, 并将结果保存到目标寄存器 Rd 中, 同时根据操作的结果更新 CPSR 寄存器。

(1) 指令的语法格式:

```
AND{<cond>}{S} <Rd>,<Rn>,<shifter_operand>
```

(2) 指令举例如下。

- (a) 保留 R0 中的 0 位和 1 位, 丢弃其余的位。

```
AND    R0, R0, #3
```

- (b) R2 = R1&R3。

```
AND    R2,R1,R3
```

- (c) R0 = R0&0x01, 取出最低位数据。

```
ANDS   R0,R0,#0x01
```

4、EOR 指令

EOR (Exclusive OR) 指令将寄存器 Rn 中的值和 shifter_operand 的值执行按位 “异或” 操作, 并将



执行结果存储到目的寄存器 **Rd** 中，同时根据指令的执行结果更新 **CPSR** 中相应的条件标志位。

(1) 指令的语法格式:

```
EOR{<cond>}{S} <Rd>,<Rn>,<shifter_operand>
```

(2) 指令举例:

(a) 反转 **R0** 中的位 0 和 1。

```
EOR R0, R0, #3
```

(b) 将 **R1** 的低 4 位取反。

```
EOR R1, R1, #0x0F
```

(c) $R2 = R1 \wedge R0$ 。

```
EOR R2, R1, R0
```

(d) 将 **R5** 和 **0x01** 进行逻辑异或，结果保存到 **R0**，并根据执行结果设置标志位。

```
EORS R0, R5, #0x01
```

5、SUB 指令

SUB (**S**ubtract) 指令从寄存器 **Rn** 中减去 **shifter_operand** 表示的数值，并将结果保存到目标寄存器 **Rd** 中，并根据指令的执行结果设置 **CPSR** 中相应的标志位。

(1) 指令的语法格式:

```
SUB{<cond>}{S} <Rd>,<Rn>,<shifter_operand>
```

(2) **SUB** 指令举例:

(a) $R0 = R1 - R2$ 。

```
SUB R0, R1, R2
```

(b) $R0 = R1 - 256$ 。

```
SUB R0, R1, #256
```

(c) $R0 = R2 - (R3 \ll 1)$ 。

```
SUB R0, R2, R3, LSL#1
```

6、RSB 指令

RSB (**R**everse **S**ubtract) 指令从寄存器 **shifter_operand** 中减去 **Rn** 表示的数值，并将结果保存到目标寄存器 **Rd** 中，并根据指令的执行结果设置 **CPSR** 中相应的标志位。

(1) 指令的语法格式:

```
RSB{<cond>}{S} <Rd>,<Rn>,<shifter_operand>
```

(2) **RSB** 指令举例:

下面的指令序列可以求一个 64 位数值的负数。64 位数放在寄存器 **R0** 与 **R1** 中，其负数放在 **R2** 和 **R3** 中。其中 **R0** 与 **R2** 中放低 32 位值。

```
RSBS R2, R0, #0
```

```
RSC R3, R1, #0
```

7、ADD 指令

ADD 指令将寄存器 **shifter_operand** 的值加上 **Rn** 表示的数值，并将结果保存到目标寄存器 **Rd** 中，并



根据指令的执行结果设置 CPSR 中相应的标志位。

(1) 指令的语法格式:

```
ADD{<cond>}{S} <Rd>,<Rn>,<shifter_operand>
```

(2) ADD 指令举例:

```
ADD    R0, R1, R2          ; R0 = R1 + R2
ADD    R0, R1, #256        ; R0 = R1 + 256
ADD    R0, R2, R3, LSL#1   ; R0 = R2 + (R3 << 1)
```

8、ADC 指令

ADC 指令将寄存器 shifter_operand 的值加上 Rn 表示的数值, 再加上 CPSR 中的 C 条件标志位的值, 将结果保存到目标寄存器 Rd 中, 并根据指令的执行结果设置 CPSR 中相应的标志位。

(1) 指令的语法格式:

```
ADC{<cond>}{S} <Rd>,<Rn>,<shifter_operand>
```

(2) ADC 指令举例:

ADC 指令将把两个操作数加起来, 并把结果放置到目的寄存器中。它使用一个进位标志位, 这样就可以做比 32 位大的加法。下面的例子将加两个 128 位的数。

128 位结果: 寄存器 R0、R1、R2 和 R3。

第一个 128 位数: 寄存器 R4、R5、R6 和 R7。

第二个 128 位数: 寄存器 R8、R9、R10 和 R11。

```
ADDS    R0, R4, R8          ;加低端的字
ADCS    R1, R5, R9          ;加下一个字, 带进位
ADCS    R2, R6, R10         ;加第三个字, 带进位
ADCS    R3, R7, R11         ;加高端的字, 带进位
```

9、SBC 指令

SBC (Subtract with Carry) 指令用于执行操作数大于 32 位时的减法操作。该指令从寄存器 Rn 中减去 shifter_operand 表示的数值, 再减去寄存器 CPSR 中 C 条件标志位的反码 [NOT (Carry flag)], 并将结果保存到目标寄存器 Rd 中, 并根据指令的执行结果设置 CPSR 中相应的标志位。

(1) 指令的语法格式:

```
SBC{<cond>}{S} <Rd>,<Rn>,<shifter_operand>
```

(2) SBC 指令举例:

下面的程序使用 SBC 实现 64 位减法, (R1, R0) - (R3, R2), 结果存放到 (R1, R0)。

```
SUBS    R0, R0, R2
SBCS    R1, R1, R3
```

10、RSC 指令

RSC (Reverse Subtract with Carry) 指令从寄存器 shifter_operand 中减去 Rn 表示的数值, 再减去寄存器 CPSR 中 C 条件标志位的反码 [NOT (Carry Flag)], 并将结果保存到目标寄存器 Rd 中, 并根据指令的执行结果设置 CPSR 中相应的标志位。

(1) 指令的语法格式:



```
RSC{<cond>} {S} <Rd>, <Rn>, <shifter_operand>
```

(2) RSC 指令举例:

下面的程序使用 RSC 指令实现求 64 位数值的负数。

```
RSBS    R2, R0, #0
RSC     R3, R1, #0
```

11、 TST 测试指令

TST (Test) 测试指令用于将一个寄存器的值和一个算术值进行比较。条件标志位根据两个操作数做“逻辑与”后的结果设置。

(1) 指令的语法格式:

```
TST{<cond>} <Rn>, <shifter_operand>
```

(2) TST 指令举例:

TST 指令类似于 CMP 指令，不产生放置到目的寄存器中的结果。而是在给出的两个操作数上进行操作并把结果反映到状态标志上。使用 TST 指令来检查是否设置了特定的位。操作数 1 是要测试的数据字而操作数 2 是一个位掩码。经过测试后，如果匹配则设置 Zero 标志，否则清除它。与 CMP 指令一样，该指令不需要指定 S 后缀。

下面的指令测试在 R0 中是否设置了位 0。

```
TST     R0, #1
```

12、 TEQ 指令

TEQ (Test Equivalence) 指令用于将一个寄存器的值和一个算术值做比较。条件标志位根据两个操作数做“逻辑异或”后的结果设置。以便后面的指令根据相应的条件标志来判断是否执行。

(1) 指令的语法格式:

```
TEQ{<cond>} <Rn>, <shifter_operand>
```

(2) TEQ 指令举例:

下面的指令是比较 R0 和 R1 是否相等，该指令不影响 CPSR 中的 V 位和 C 位。

```
TEQ     R0, R1
```

TST 指令与 EORS 指令的区别在于 TST 指令不保存运算结果。使用 TEQ 进行相等测试，常与 EQ 和 NE 条件码配合使用，当两个数据相等时，条件码 EQ 有效；否则条件码 NE 有效。

13、 CMP 指令

CMP (Compare) 指令使用寄存器 Rn 的值减去 operand2 的值，根据操作的结果更新 CPSR 中相应的条件标志位，以便后面的指令根据相应的条件标志来判断是否执行。

(1) 指令的语法格式:

```
CMP{<cond>} <Rn>, <shifter_operand>
```

(2) CMP 指令举例:

CMP 指令允许把一个寄存器的内容与另一个寄存器的内容或立即值进行比较，更改状态标志来允许进行条件执行。它进行一次减法，但不存储结果，而是正确地更改标志位。标志位表示的是操作数 1 与操作数 2 比较的结果（其值可能为大于、小于、相等）。如果操作数 1 大于操作数 2，则此后的有 GT 后



缀的指令将可以执行。

显然，CMP 不需要显式地指定 S 后缀来更改状态标志。

a) 下面的指令是比较 R1 和立即数 10 并设置相关的标志位。

```
CMP    R1, #10
```

b) 下面的指令是比较寄存器 R1 和 R2 中的值并设置相关的标志位。

```
CMP    R1, R2
```

通过上面的例子可以看出，CMP 指令与 SUBS 指令的区别在于 CMP 指令不保存运算结果，在进行两个数据大小判断时，常用 CMP 指令及相应的条件码来进行操作。

14、 CMN 指令

CMN (Compare Negative) 指令使用寄存器 Rn 的值减去 operand2 的负数值 (加上 operand2)，根据操作的结果更新 CPSR 中相应的条件标志位，以便后面的指令根据相应的条件标志来判断是否执行。

(1) 指令的语法格式：

```
CMN{<cond>} <Rn>, <shifter_operand>
```

(2) CMN 指令举例：

CMN 指令将寄存器 Rn 中的值加上 shifter_operand 表示的数值，根据加法的结果设置 CPSR 中相应的条件标志位。寄存器 Rn 中的值加上 shifter_operand 的操作结果对 CPSR 中条件标志位的影响，与寄存器 Rn 中的值减去 shifter_operand 的操作结果的相反数对 CPSR 中条件标志位的影响有细微差别。当第 2 个操作数为 0 或者为 0x80000000 时两者结果不同。比如下面两条指令。

```
CMP    Rn, #0
CMN    Rn, #0
```

第 1 条指令使标志位 C 值为 1，第 2 条指令使标志位 C 值为 0。

下面的指令使 R0 值加 1，判断 R0 是否为 1 的补码，若是，则 Z 置位。

```
CMN    R0, #1
```

15、 ORR 指令

ORR (Logical OR) 为逻辑或操作指令，它将第 2 个源操作数 shifter_operand 的值与寄存器 Rn 的值按位做“逻辑或”操作，结果保存到 Rd 中。

(1) 指令的语法格式：

```
ORR{<cond>}{S} <Rd>, <Rn>, <shifter_operand>
```

(2) ORR 指令举例：

(a) 设置 R0 中位 0 和 1。

```
ORR    R0, R0, #3
```

(b) 将 R0 的低 4 位置 1。

```
ORR    R0, R0, #0x0F
```

(c) 使用 ORR 指令将 R2 的高 8 位数据移入到 R3 的低 8 位中。

```
MOV    R1, R2, LSR #4
ORR    R3, R1, R3, LSL #8
```

16、 BIC 位清零指令



BIC (Bit Clear) 位清零指令，将寄存器 Rn 的值与第 2 个源操作数 shifter_operand 的值的反码按位做“逻辑与”操作，结果保存到 Rd 中。

(1) 指令的语法格式：

```
BIC{<cond>}{S} <Rd>, <Rn>, <shifter_operand>
```

(2) BIC 指令举例：

(a) 清除 R0 中的位 0、1 和 3，保持其余的不变。

```
BIC R0, R0, #0x1011
```

(b) 将 R3 的反码和 R2 做“逻辑与”操作，结果保存到 R1 中。

```
BIC R1, R2, R3
```

3.2.2 乘法指令

ARM 乘法指令完成两个数据的乘法。两个 32 位二进制数相乘的结果是 64 位的积。在有些 ARM 的处理器版本中，将乘积的结果保存到两个独立的寄存器中。另外一些版本只将最低有效 32 位存放到一个寄存器中。无论是哪种版本的处理器，都有乘—累加的变型指令，将乘积连续累加得到总和。而且有符号数和无符号数都能使用。对于有符号数和无符号数，结果的最低有效位是一样的。因此，对于只保留 32 位结果的乘法指令，不需要区分有符号数和无符号数这两种情况。

如表 3-7 所示为各种形式乘法指令的功能。

表 3-7 各种形式乘法指令

操作码[23:21]	助 记 符	意 义	操 作
000	MUL	乘（保留 32 位结果）	$Rd: = (Rm \times Rs) [31:0]$
001	MLA	乘—累加（保留 32 位结果）	$Rd: = (Rm \times Rs + Rn) [31:0]$
100	UMULL	无符号数长乘	$RdHi: RdLo: = Rm \times Rs$
101	UMLAL	无符号长乘—累加	$RdHi: RdLo: += Rm \times Rs$
110	SMULL	有符号数长乘	$RdHi: RdLo: = Rm \times Rs$
111	SMLAL	有符号数长乘—累加	$RdHi: RdLo: += Rm \times Rs$

其中：

- (1) “RdHi: RdLo” 是由 RdHi（最高有效 32 位）和 RdLo（最低有效 32 位）连接形成的 64 位数，“[31:0]” 只选取结果的最低有效 32 位。
- (2) 简单的赋值由“:=”表示。
- (3) 累加（将右边加到左边）是由“+=”表示。

各个乘法指令中的位 S（参考下文具体指令的语法格式）控制条件码的设置会产生以下结果。

- a) 对于产生 32 位结果的指令形式，将标志位 N 设置为 Rd 的第 31 位的值；对于产生长结果的指令形式，将其设置为 RdHi 的第 31 位的值。
- b) 对于产生 32 位结果的指令形式，如果 Rd 等于零，则标志位 Z 置位；对于产生长结果的指令形式，RdHi 和 RdLo 同时为零时，标志位 Z 置位。
- c) 将标志位 C 设置成无意义的值。
- d) 标志位 V 不变。

1、MUL 指令



MUL (Multiply) 32 位乘法指令将 Rm 和 Rs 中的值相乘，结果的最低 32 位保存到 Rd 中。

(1) 指令的语法格式：

```
MUL{<cond>}{S} <Rd>, <Rm>, <Rs>
```

(2) 指令举例：

(a) $R1 = R2 \times R3$ 。

```
MUL R1, R2, R3
```

(b) $R0 = R3 \times R7$ ，同时设置 CPSR 中的 N 位和 Z 位。

```
MULS R0, R3, R7
```

2、MLA 指令

MLA (Multiply Accumulate) 32 位乘—累加指令将 Rm 和 Rs 中的值相乘，再将乘积加上第 3 个数，结果的最低 32 位保存到 Rd 中。

(1) 指令的语法格式：

```
MLA{<cond>}{S} <Rd>, <Rm>, <Rs>, <Rn>
```

(2) 指令举例：

下面的指令完成 $R1 = R2 \times R3 + 10$ 的操作。

```
MOV R0, #0x0A
```

```
MLA R1, R2, R3, R0
```

3、UMULL 指令

UMULL (Unsigned Multiply Long) 为 64 位无符号乘法指令。它将 Rm 和 Rs 中的值做无符号数相乘，结果的低 32 位保存到 RsLo 中，高 32 位保存到 RdHi 中。

(1) 指令的语法格式：

```
UMULL{<cond>}{S} <RdLo>, <RdHi>, <Rm>, <Rs>
```

(2) 指令举例：

下面的指令完成 $(R1, R0) = R5 \times R8$ 操作。

```
UMULL R0, R1, R5, R8;
```

4、UMLAL 指令

UMLAL (Unsigned Multiply Accumulate Long) 为 64 位无符号长乘—累加指令。指令将 Rm 和 Rs 中的值做无符号数相乘，64 位乘积与 RdHi、RdLo 相加，结果的低 32 位保存到 RsLo 中，高 32 位保存到 RdHi 中。

(1) 指令的语法格式：

```
UMLAL{<cond>}{S} <RdLo>, <RdHi>, <Rm>, <Rs>
```

(2) 指令举例：

下面的指令完成 $(R1, R0) = R5 \times R8 + (R1, R0)$ 操作。

```
UMLAL R0, R1, R5, R8;
```

5、SMULL 指令



SMULL (Signed Multiply Long) 为 64 位有符号长乘法指令。指令将 Rm 和 Rs 中的值做有符号数相乘, 结果的低 32 位保存到 RsLo 中, 高 32 位保存到 RdHi 中。

(1) 指令的语法格式:

```
SMULL{<cond>}{S} <RdLo>, <RdHi>, <Rm>, <Rs>
```

(2) 指令举例:

下面的指令完成 $(R3, R2) = R7 \times R6$ 操作。

```
SMULL R2, R3, R7, R6;
```

6、SMLAL 指令

SMLAL (Signed Multiply Accumulate Long) 为 64 位有符号长乘—累加指令。指令将 Rm 和 Rs 中的值做有符号数相乘, 64 位乘积与 RdHi、RdLo 相加, 结果的低 32 位保存到 RsLo 中, 高 32 位保存到 RdHi 中。

(1) 指令的语法格式:

```
SMLAL{<cond>}{S} <RdLo>, <RdHi>, <Rm>, <Rs>
```

(2) 指令举例:

下面的指令完成 $(R3, R2) = R7 \times R6 + (R3, R2)$ 操作。

```
SMLAL R2, R3, R7, R6;
```

3.2.3 Load/Store 指令

Load/Store 内存访问指令在 ARM 寄存器和存储器之间传送数据。ARM 指令中有 3 种基本的数据传送指令。

(1) 单寄存器 Load/Store 指令 (Single Register), 这些指令在 ARM 寄存器和存储器之间提供更灵活的单数据项传送方式。数据项可以是字节、16 位半字或 32 位字。

(2) 多寄存器 Load/Store 内存访问指令。这些指令的灵活性比单寄存器传送指令差, 但可以使大量的数据更有效地传送。它们用于进程的进入和退出、保存和恢复工作寄存器及复制存储器中的一块数据。

(3) 单寄存器交换指令 (Single Register Swap)。这些指令允许寄存器和存储器中的数值进行交换, 在一条指令中有效地完成 Load/Store 操作。它们在用户级编程中很少用到。它的主要用途是在多处理器系统中实现信号量 (Semaphores) 的操作, 以保证不会同时访问公用的数据结构。

(4) 单寄存器的 Load/Store 指令, 这种指令用于把单一的数据传入或者传出一个寄存器。支持的数据类型有字节 (8 位)、半字 (16 位) 和字 (32 位)。

1、单寄存器的 Load/Store 指令

如表所示列出了所有单寄存器的 Load/Store 指令。

表 单寄存器 Load/Store 指令

指 令	作 用	操 作
LDR	把存储器中的一个字装入一个寄存器	$Rd \leftarrow mem32[address]$
STR	将寄存器中的字保存到存储器	$Rd \rightarrow mem32[address]$



LDRB	把一个字节装入一个寄存器	$Rd \leftarrow \text{mem8}[\text{address}]$
STRB	将寄存器中的低 8 位字节保存到存储器	$Rd \rightarrow \text{mem8}[\text{address}]$
LDRH	把一个半字装入一个寄存器	$Rd \leftarrow \text{mem16}[\text{address}]$
STRH	将寄存器中的低 16 位半字保存到存储器	$Rd \rightarrow \text{mem16}[\text{address}]$
LDRBT	用户模式下将一个字节装入寄存器	$Rd \leftarrow \text{mem8}[\text{address}]$ under user mode
STRBT	用户模式下将寄存器中的低 8 位字节保存到存储器	$Rd \rightarrow \text{mem8}[\text{address}]$ under user mode
LDRT	用户模式下把一个字装入一个寄存器	$Rd \leftarrow \text{mem32}[\text{address}]$ under user mode
STRT	用户模式下将存储器中的字保存到寄存器	$Rd \leftarrow \text{mem32}[\text{address}]$ under user mode
LDRSB	把一个有符号字节装入一个寄存器	$Rd \leftarrow \text{sign}\{\text{mem8}[\text{address}]\}$
LDRSH	把一个有符号半字装入一个寄存器	$Rd \leftarrow \text{sign}\{\text{mem16}[\text{address}]\}$

1> LDR 指令

LDR 指令用于从内存中将一个 32 位的字读取到目标寄存器。

(1) 指令的语法格式:

```
LDR{<cond>} <Rd>, <addr_mode>
```

(2) 指令举例:

```
LDR R1, [R0, #0x12] ;将 R0+12 地址处的数据读出, 保存到 R1 中 (R0 的值不变)
LDR R1, [R0] ;将 R0 地址处的数据读出, 保存到 R1 中 (零偏移)
LDR R1, [R0, R2] ;将 R0+R2 地址的数据读出, 保存到 R1 中 (R0 的值不变)
LDR R1, [R0, R2, LSL #2] ;将 R0+R2×4 地址处的数据读出, 保存到 R1 中 (R0、R2 的值不变)
LDR Rd, label ;label 为程序标号, label 必须是当前指令的-4~4KB 范围内
LDR Rd, [Rn], #0x04 ;Rn 的值用做传输数据的存储地址。在数据传送后, 将偏移量 0x04 与 Rn 相加, 结果写回到 Rn 中。Rn 不允许是 R15
```

2> STR 指令

STR 指令用于将一个 32 位的字数据写入到指令中指定的内存单元。

(1) 指令的语法格式:

```
STR{<cond>} <Rd>, <addr_mode>
```

(2) 指令举例: LDR/STR 指令用于对内存变量的访问、内存缓冲区数据的访问、查表、外围部件的控制操作等, 若使用 LDR 指令加载数据到 PC 寄存器, 则实现程序跳转功能, 这样也就实现了程序散转。

a) 变量访问。

```
NumCount .equ 0x40003000 ;定义变量 NumCount
LDR R0, =NumCount ;使用 LDR 伪指令装载 NumCount 的地址到 R0
LDR R1, [R0] ;取出变量值
ADD R1, R1, #1 ;NumCount=NumCount+1
STR R1, [R0] ;保存变量
```

b) GPIO 设置。

```
GPIO—BASE .equ 0xe0028000 ;定义 GPIO 寄存器的基地址
...
LDR R0, =GPIO—BASE
LDR R1, =0x00ffff00 ;将设置值放入寄存器
STR R1, [R0, #0x0C] ;IODIR=0x00ffff00, IOSET 的地址为 0xE0028004
```



c) 程序散转。

```
...
MOV  R2,R2,LSL #2           ;功能号乘以 4，以便查表
LDR  PC,[PC,R2]             ;查表取得对应功能子程序地址并跳转
NOP
FUN-TAB .word  FUN-SUB0
        .word  FUN-SUB1
        .word  FUN-SUB2
...
```

3> LDRB 指令

LDRB 指令根据 `addr_mode` 所确定的地址模式将一个 8 位字节读取到指令中的目标寄存器 `Rd`。

指令的语法格式：

```
LDR{<cond>}B <Rd>, <addr_mode>
```

4> STRB 指令

STRB 指令从寄存器中取出指定的 8 位字节放入寄存器的低 8 位，并将寄存器的高位补 0。指令的语法格式：

```
STR{<cond>}B <Rd>,<addr_mode>
```

5> LDRH 指令

LDRH 指令用于从内存中将一个 16 位的半字读取到目标寄存器。

如果指令的内存地址不是半字节对齐的，指令的执行结果不可预知。

指令的语法格式：

```
LDR{<cond>}H <Rd>,<addr_mode>
```

6> STRH 指令

STRH 指令从寄存器中取出指定的 16 位半字放入寄存器的低 16 位，并将寄存器的高位补 0。

指令的语法格式：

```
STR{<cond>}H <Rd>,<addr_mode>
```

2、多寄存器的 Load/Store 内存访问指令

多寄存器的 Load/Store 内存访问指令也叫批量加载/存储指令，它可以实现在一组寄存器和一块连续的内存单元之间传送数据。LDM 用于加载多个寄存器，STM 用于存储多个寄存器。多寄存器的 Load/Store 内存访问指令允许一条指令传送 16 个寄存器的任何子集或所有寄存器。多寄存器的 Load/Store 内存访问指令主要用于现场保护、数据复制和参数传递等。如表所示列出了多寄存器的 Load/Store 内存访问指令。

表 多寄存器的 Load/Store 内存访问指令

指 令	作 用	操 作
LDM	装载多个寄存器	$\{Rd\}^*N \leftarrow \text{mem32}[\text{start address}+4*N]$
STM	保存多个寄存器	$\{Rd\}^*N \rightarrow \text{mem32}[\text{start address}+4*N]$

1> LDM 指令



LDM 指令将数据从连续的内存单元中读取到指令中指定的寄存器列表中的各寄存器中。当 PC 包含在 LDM 指令的寄存器列表中时，指令从内存中读取的字数据将被作为目标地址值，指令执行后程序将从目标地址处开始执行，从而实现了指令的跳转。

指令的语法格式：

```
LDM{<cond>}<addressing_mode> <Rn>{!}, <registers>
```

寄存器 R0~R15 分别对应于指令编码中 bit[0]~bit[15]位。如果 Ri 存在于寄存器列表中，则相应的位等于 1，否则为 0。LDM 指令将数据从连续的内存单元中读取到指令中指定的寄存器列表中的各寄存器中。

指令的语法格式：

```
LDM{<cond>}<addressing_mode><Rn>,<registers_without_pc>
```

2> STM 指令

STM 指令将指令中寄存器列表中的各寄存器数值写入到连续的内存单元中。主要用于块数据的写入、数据栈操作及进入子程序时保存相关寄存器的操作。

指令的语法格式：

```
STM{<cond>}<addressing_mode> <Rn>{!}, <registers>
```

STM 指令将指令中寄存器列表中的各寄存器数值写入到连续的内存单元中。主要用于块数据的写入、数据栈操作及进入子程序时保存相关寄存器等操作。

指令的语法格式：

```
STM{<cond>}<addressing_mode> <Rn>,<registers>^
```

3> 数据传送指令应用

LDM/STM 批量加载/存储指令可以实现在一组寄存器和一块连续的内存单元之间传输数据。LDM 为加载多个寄存器，STM 为存储多个寄存器。允许一条指令传送 16 个寄存器的任何子集或所有寄存器。指令格式如下：

```
LDM{cond}<模式> Rn{!},regist{^}  
STM{cond}<模式> Rn{!},regist{^}
```

LDM/STM 的主要用途有现场保护、数据复制和参数传递等。其模式有 8 种，其中前面 4 种用于数据块的传输，后面 4 种是堆栈操作，如下所示。

- (1) IA：每次传送后地址加 4。
- (2) IB：每次传送前地址加 4。
- (3) DA：每次传送后地址减 4。
- (4) DB：每次传送前地址减 4。
- (5) FD：满递减堆栈。
- (6) ED：空递增堆栈。
- (7) FA：满递增堆栈。
- (8) EA：空递增堆栈。

其中，寄存器 Rn 为基址寄存器，装有传送数据的初始地址，Rn 不允许为 R15；后缀“!”表示最后



的地址写回到 Rn 中；寄存器列表 reglist 可包含多于一个寄存器或寄存器范围，使用“,” 分开，如{R1, R2, R6~R9}，寄存器排列由小到大排列；“^” 后缀不允许在用户模式下使用，只能在系统模式下使用。若在 LDM 指令用寄存器列表中包含有 PC 时使用，那么除了正常的多寄存器传送外，将 SPSR 复制到 CPSR 中，这可用于异常处理返回；使用“^” 后缀进行数据传送且寄存器列表不包含 PC 时，加载/存储的是用户模式寄存器，而不是当前模式寄存器。

```
LDMIA R0!, {R3~R9}      ;加载 R0 指向的地址上的多字数据，保存到 R3~R9 中，R0 值更新
STMIA R1!, {R3~R9}      ;将 R3~R9 的数据存储到 R1 指向的地址上，R1 值更新
STMFD SP!, {R0~R7, LR}  ;现场保存，将 R0~R7、LR 入栈
LDMFD SP!, {R0~R7, PC}^ ;恢复现场，异常处理返回
```

在进行数据复制时，先设置好源数据指针，然后使用块复制寻址指令 LDMIA/STMIA、LDMIB/STMIB、LDMDA/STMDA、LDMDB/STMDB 进行读取和存储。而进行堆栈操作时，则要先设置堆栈指针，一般使用 SP 然后使用堆栈寻址指令 STMFD/LDMFD、STMED/LDMED、STMEA/LDMEA 实现堆栈操作。数据是存储在基址寄存器的地址之上还是之下，地址是存储第一个值之前还是之后、增加还是减少，如表 3-10 所示。

表 3-10 多寄存器的 Load/Store 内存访问指令映射

		向 上 生 长		向 下 生 长	
		满	空	满	空
增加	之前	STMIB			LDMIB
		STMFA			LDMED
	之后		STMIA	LDMIA	
			STMEA	LDMFD	
增加	之前		LDMDB	STMDB	
			LDMEA	STMFD	
	之后	LDMDA			STMDA
		LDMFA			STMED

【举例】 使用 LDM/STM 进行数据复制。

```
LDR R0, =SrcData      ;设置源数据地址
LDR R1, =DstData      ;设置目标地址
LDMIA R0, {R2~R9}     ;加载 8 字数据到寄存器 R2~R9
STMIA R1, {R2~R9}     ;存储寄存器 R2~R9 到目标地址
```

【举例】 使用 LDM/STM 进行现场寄存器保护，常在子程序或异常处理使用。

```
SENDBYTE:
    STMFD SP!, {R0~R7, LR} ;寄存器压栈保护
    ...
    BL DELAY               ;调用 DELAY 子程序
    ...
    LDMFD SP!, {R0~R7, PC} ;恢复寄存器，并返回
```

3、单数据交换指令

交换指令是 Load/Store 指令的一种特例，它把一个寄存器单元的内容与寄存器内容交换。交换指令是一个原子操作（Atomic Operation），也就是说，在连续的总线操作中读/写一个存储单元，在操作期间



阻止其他任何指令对该存储单元的读/写。交换指令如表所示。

表 交换指令 SWP

指 令	作 用	操 作
SWP	字交换	tmp=mem32[Rn] mem32[Rn]=Rm Rd=tmp
SWPB	字节交换	tmp=mem8[Rn] mem8[Rn]=Rm Rd=tmp

1> SWP 字交换指令

SWP 指令用于将内存中的一个字单元和一个指定寄存器的值相交换。操作过程如下：假设内存单元地址存放在寄存器<Rn>中，指令将<Rn>中的数据读取到目的寄存器 Rd 中，同时将另一个寄存器<Rm>的内容写入到该内存单元中。

当<Rd>和<Rm>为同一个寄存器时，指令交换该寄存器和内存单元的内容。

指令的语法格式：

```
SWP{<cond>} <Rd>, <Rm>, [<Rn>]
```

2> SWPB 字节交换指令

SWPB 指令用于将内存中的一个字节单元和一个指定寄存器的低 8 位值相交换，操作过程如下：假设内存单元地址存放在寄存器<Rn>中，指令将<Rn>中的数据读取到目的寄存器 Rd 中，寄存器 Rd 的高 24 位设为 0，同时将另一个寄存器<Rm>的低 8 位内容写入到该内存字节单元中。当<Rd>和<Rm>为同一个寄存器时，指令交换该寄存器低 8 位内容和内存字节单元的内容。

指令的语法格式：

```
SWP{<cond>}B <Rd>, <Rm>, [<Rn>]
```

3> 交换指令 SWP 应用

SWP 指令用于将一个内存单元（该单元地址放在寄存器 Rn 中）的内容读取到一个寄存器 Rd 中，同时将另一个寄存器 Rm 的内容写到该内存单元中，使用 SWP 可实现信号量操作。

指令的语法格式：

```
SWP{cond}B Rd, Rm, [Rn]
```

其中，B 为可选后缀，若有 B，则交换字节；否则交换 32 位字。Rd 为目的寄存器，存储从存储器中加载的数据，同时，Rm 中的数据将会被存储到存储器中。若 Rm 与 Rn 相同，则为寄存器与存储器内容进行交换。Rn 为要进行数据交换的存储器地址，Rn 不能与 Rd 和 Rm 相同。

SWP 指令举例：

```
SWP R1, R1, [R0] ; 将 R1 的内容与 R0 指向的存储单元内容进行交换
```

```
SWPB R1, R2, [R0] ; 将 R0 指向的存储单元内容读取一字节数据到 R1 中（高 24 位清零），并
```



将 R2 的内容 写入到该内存单元中（最低字节有效），使用 SWP 指令可以方便地进行信号量操作

```

12C_SEM      .equ      0x40003000
...
12C_SEM_WAIT:
    MOV      R0, #0
    LDR      R0, =12C_SEM
    SWP      R1, R1, [R0]      ;取出信号量，并将其设为 0
    CMP      R1, #0            ;判断是否有信号
    BEQ      12C_SEM_WAIT      ;若没有信号则等待
    
```

3.2.4 跳转指令

跳转（B）和跳转连接（BL）指令是改变指令执行顺序的标准方式。ARM 一般按照字地址顺序执行指令，需要时使用条件执行跳过某段指令。只要程序必须偏离顺序执行，就要使用控制流指令来修改程序计数器。尽管在特定情况下还有其他几种方式实现这个目的，但转移和转移连接指令是标准的方式。跳转指令改变程序的执行流程或者调用子程序。这种指令使得一个程序可以使用子程序、if-then-else 结构及循环。执行流程的改变迫使程序计数器（PC）指向一个新的地址，ARMv5 架构指令集包含的跳转指令如表所示。

表 ARMv5 架构跳转指令

助 记 符	说 明	操 作
B	跳转指令	$pc \leftarrow label$
BL	带返回的连接跳转	$pc \leftarrow label (lr \leftarrow BL \text{ 后面的第一条指令})$
BX	跳转并切换状态	$pc \leftarrow Rm \& 0xffffffe, T \leftarrow Rm \& 1$
BLX	带返回的跳转并切换状态	$pc \leftarrow lable, T \leftarrow 1$ $pc \leftarrow Rm \& 0xffffffe, T \leftarrow Rm \& 1$ $lr \leftarrow BL \text{ 后面的第一条指令}$

另一种实现指令跳转的方式是通过直接向 PC 寄存器中写入目标地址值，实现在 4GB 地址空间中任意跳转，这种跳转指令又称为长跳转。如果在长跳转指令之前使用“MOV LR”或“MOV PC”等指令，可以保存将来返回的地址值，也就实现了在 4GB 的地址空间中的子程序调用。

1、跳转指令 B 及带连接的跳转指令 BL

跳转指令 B 使程序跳转到指定的地址执行程序。带连接的跳转指令 BL 将下一条指令的地址复制到 R14（即返回地址连接寄存器 LR）寄存器中，然后跳转到指定地址运行程序。需要注意的是，这两条指令和目标地址处的指令都要属于 ARM 指令集。两条指令都可以根据 CPSR 中的条件标志位的值决定指令是否执行。

（1） 指令的语法格式：

```
B{L}{<cond>} <target_address>
```

BL 指令用于实现子程序调用。子程序的返回可以通过将 LR 寄存器的值复制到 PC 寄存器来实现。

下面 3 种指令可以实现子程序返回。

a) BX R14（如果体系结构支持 BX 指令）。



- b) MOV PC, R14。
- c) 当子程序在入口处使用了压栈指令:

```
STMFD R13!,{<registers>,R14}
```

可以使用指令:

```
LDMFD R13!,{<registers>,PC}
```

将子程序返回地址放入 PC 中。

ARM 汇编器通过以下步骤计算指令编码中的 signed_immed_24。

- (a) 将 PC 寄存器的值作为本跳转指令的基地址值。
- (b) 从跳转的目标地址中减去上面所说的跳转的基地址,生成字节偏移量。由于 ARM 指令是字对齐的,该字节偏移量为 4 的倍数。
- (c) 当上面生成的字节偏移量超过-33 554 432~+33 554 430 时,不同的汇编器使用不同的代码产生策略。否则,将指令编码字中的 signed_immed_24 设置成上述字节偏移量的 bits[25:2]。

(2) 程序举例:

- a) 程序跳转到 LABEL 标号处。

```
B LABEL ;
ADD R1,R2,#4
ADD R3,R2,#8
SUB R3,R3,R1
LABEL:
SUB R1,R2,#8
```

- b) 跳转到绝对地址 0x1234 处。

```
B 0x1234
```

- c) 跳转到子程序 func 处执行,同时将当前 PC 值保存到 LR 中。

```
BL func
```

- d) 条件跳转:当 CPSR 寄存器中的 C 条件标志位为 1 时,程序跳转到标号 LABEL 处执行。

```
BCC LABEL
```

- e) 通过跳转指令建立一个无限循环。

```
LOOP:
ADD R1,R2,#4
ADD R3,R2,#8
SUB R3,R3,R1
B LOOP
```

- f)通过使用跳转使程序体循环 10 次。

```
MOV R0,#10
LOOP:
SUBS R0,#1
BNE LOOP
```

- g) 条件子程序调用示例。

```
...
CMP R0,#5 ;如果 R0<5
BLLT SUB1 ;则调用
```



```
BLGE SUB2 ; 否则调用 SUB2
```

2、带状态切换的跳转指令 BX

带状态切换的跳转指令（BX）使程序跳转到指令中指定的参数 Rm 指定的地址执行程序，Rm 的第 0 位复制到 CPSR 中 T 位，bit[31:1]移入 PC。若 Rm 的 bit[0]为 1，则跳转时自动将 CPSR 中的标志位 T 置位，即把目标地址的代码解释为 Thumb 代码；若 Rm 的位 bit[0]为 0，则跳转时自动将 CPSR 中的标志位 T 复位，即把目标地址代码解释为 ARM 代码。

(1) 指令的语法格式：

```
BX{<cond>} <Rm>
```

(a) 当 Rm[1:0]=0b10 时，指令的执行结果不可预知。因为在 ARM 状态下，指令是 4 字节对齐的。

(b) PC 可以作为 Rm 寄存器使用，但这种用法不推荐使用。当 PC 作为<Rm>使用时，指令“BX PC”将程序跳转到当前指令下面第二条指令处执行。虽然这样跳转可以实现，但最好使用下面的指令完成这种跳转。

```
MOV PC, PC
```

或

```
ADD PC, PC, #0
```

(2) 指令举例：

(a) 转移到 R0 中的地址，如果 R0[0]=1，则进入 Thumb 状态。

```
BX R0;
```

(b) 跳转到 R0 指定的地址，并根据 R0 的最低位来切换处理器状态。

```
ADRL R0, ThumbFun+1 ;
```

```
BX R0;
```

3、带连接和状态切换的连接跳转指令 BLX

带连接和状态切换的跳转指令（Branch with Link Exchange, BLX）使用标号，用于使程序跳转到 Thumb 状态或从 Thumb 状态返回。该指令为无条件执行指令，并用分支寄存器的最低位来更新 CPSR 中的 T 位，将返回地址写入到连接寄存器 LR 中。

(1) 语法格式：

```
BLX <target_add>
```

其中，<target_add>为指令的跳转目标地址。该地址根据以下规则计算。

- 将指令中指定的 24 位偏移量进行符号扩展，形成 32 位立即数。
- 将结果左移两位。
- 位 H (bit[24]) 加到结果地址的第一位 (bit[1])。
- 将结果累加进程序计数器 (PC) 中。

计算偏移量的工作一般由 ARM 汇编器来完成。这种形式的跳转指令只能实现-32~32MB 空间的跳转。左移两位形成字偏移量，然后将其累加进程序计数器 (PC) 中。这时，程序计数器的内容为 BX 指令地址加 8 字节。位 H (bit[24]) 也加到结果地址的第一位 (bit[1])，使目标地址成为半字地址，以执行



接下来的 Thumb 指令。计算偏移量的工作一般由 ARM 汇编器来完成。这种形式的跳转指令只能实现 -32~32MB 空间的跳转。

(2) 指令的使用

(a) 从 Thumb 状态返回到 ARM 状态，使用 BX 指令。

```
BX R14
```

(b) 可以在子程序的入口和出口增加栈操作指令。

```
PUSH {<registers>,R14}
```

```
POP {<registers>,PC}
```

3.2.5 状态操作指令

ARM 指令集提供了两条指令，可直接控制程序状态寄存器（Program State Register，PSR）。MRS 指令用于把 CPSR 或 SPSR 的值传送到一个寄存器；MSR 与之相反，把一个寄存器的内容传送到 CPSR 或 SPSR。这两条指令相结合，可用于对 CPSR 和 SPSR 进行读/写操作。程序状态寄存器指令如表所示。

表 程序状态寄存器指令

指 令	作 用	操 作
MRS	把程序状态寄存器的值送到一个通用寄存器	Rd=SPR
MSR	把通用寄存器的值送到程序状态寄存器或把一个立即数送到程序状态字	PSR[field]=Rm 或 PSR[field]=immediate

在指令语法中可看到一个称为 fields 的项，它可以是控制（C）、扩展（X）、状态（S）及标志（F）的组合。

1、MRS

MRS 指令用于将程序状态寄存器的内容传送到通用寄存器中。

在 ARM 处理器中，只有 MRS 指令可以将状态寄存器 CPSR 或 SPSR 读出到通用寄存器中。

(1) 指令的语法格式：

```
MRS{cond} Rd, PSR
```

其中，Rd 为目标寄存器，Rd 不允许为程序计数器（PC）。PSR 为 CPSR 或 SPSR。

(2) 指令举例：

```
MRS R1,CPSR ;将 CPSR 状态寄存器读取，保存到 R1 中
```

```
MRS R2,SPSR ;将 SPSR 状态寄存器读取，保存到 R1 中
```

MRS 指令读取 CPSR，可用来判断 ALU 的状态标志及 IRQ/FIQ 中断是否允许等；在异常处理程序中，读 SPSR 可指定进入异常前的处理器状态等。MRS 与 MSR 配合使用，实现 CPSR 或 SPSR 寄存器的读—修改—写操作，可用来进行处理器模式切换，允许/禁止 IRQ/FIQ 中断等设置。另外，进程切换或允许异常中断嵌套时，也需要使用 MRS 指令读取 SPSR 状态值并保存起来。

2、MSR

在 ARM 处理器中，只有 MSR 指令可以直接设置状态寄存器 CPSR 或 SPSR。

(3) 指令的语法格式：

```
MSR{cond} PSR_field,#immed_8r
```



```
MSR{cond} PSR_field,Rm
```

其中，PSR 是指 CPSR 或 SPSR。<fields>设置状态寄存器中需要操作的位。状态寄存器的 32 位可以分为 4 个 8 位的域（field）。bits[31:24]为条件标志位域，用 f 表示；bits[23:16]为状态位域，用 s 表示；bits[15:8]为扩展位域，用 x 表示；bits[7:0]为控制位域，用 c 表示；immed_8r 为要传送到状态寄存器指定域的立即数，8 位；Rm 为要传送到状态寄存器指定域的数据源寄存器。

(4) 指令举例：

```
MSR CPSR_c, #0xD3      ;CPSR[7:0]=0xD3, 切换到管理模式
MSR CPSR_cxsf, R3      ;CPSR=R3
```

注意：

只有在特权模式下才能修改状态寄存器。

程序中不能通过 MSR 指令直接修改 CPSR 中的 T 位控制位来实现 ARM 状态/Thumb 状态的切换，必须使用 BX 指令来完成处理器状态的切换（因为 BX 指令属转移指令，它会打断流水线状态，实现处理器状态的切换）。MRS 与 MSR 配合使用，实现 CPSR 或 SPSR 寄存器的读—修改—写操作，可用来进行处理器模式切换及允许/禁止 IRQ/FIQ 中断等设置。

3、程序状态寄存器指令的应用

【举例】 使能 IRQ 中断。

```
ENABLE_IRQ:
MRS    R0,CPSR
BIC    R0,R0,#0x80
MSR    CPSR_c,R0
MOV    PC,LR
```

【举例】 禁止 IRQ 中断。

```
DISABLE_IRQ:
MRS    R0,CPSR
ORR    R0,R0,#0x80
MSR    CPSR_c,R0
MOV    PC,LR
```

【举例】 堆栈指令初始化。

```
INITSTACK:
MOV    R0,LR      ;保存返回地址
```

设置管理模式堆栈：

```
MSR    CPSR_c, #0xD3
LDR    SP,StackSvc
```

设置中断模式堆栈：

```
MSR    CPSR_c, #0xD2
LDR    SP,StackSvc
```

3.2.6 协处理器指令

ARM 体系结构允许通过增加协处理器来扩展指令集。最常用的协处理器是用于控制片上功能的系统



协处理器。例如，控制 Cache 和存储管理单元的 cp15 寄存器。此外，还有用于浮点运算的浮点 ARM 协处理器，各生产商还可以根据需要开发自己的专用协处理器。

ARM 协处理器具有自己专用的寄存器组，它们的状态由控制 ARM 状态的指令的镜像指令来控制。程序的控制流指令由 ARM 处理器来处理，所有协处理器指令只能同数据传递和数据传送有关。按照 RISC 的 Load/Store 体系原则，数据的处理和传送指令是被清楚分开的，所以它们有不同的指令格式。ARM 处理器支持 16 个协处理器，在程序执行过程中，每个协处理器忽略 ARM 和其他协处理器指令。当一个协处理器硬件不能执行属于它的协处理器指令时，将产生一个未定义指令异常中断，在该异常中断处理过程中，可以通过软件仿真该硬件操作。如果一个系统中不包含向量浮点运算器，则可以选择浮点运算软件包来支持向量浮点运算。

ARM 协处理器可以部分地执行一条指令，然后产生中断。如除法运算除数为 0 和溢出，这样可以更好地处理运行时产生（run-time-generated）的异常。但是，指令的部分执行是由协处理器完成的，此过程对 ARM 来说是透明的。当 ARM 处理器重新获得执行时，它将从产生异常的指令处开始执行。对某一个协处理器来说，并不一定用到协处理器指令中的所有的域。具体协处理器如何定义和操作完全由协处理器的制造商自己决定，因此，ARM 协处理器指令中的协处理器寄存器的标识符及操作助记符也有各种不同的实现定义。程序员可以通过宏定义这些指令的语法格式。

ARM 协处理器指令可分为以下 3 类。

(1) 协处理器数据操作。协处理器数据操作完全是协处理器内部操作，它完成协处理器寄存器的状态改变。如浮点加运算，在浮点协处理器中两个寄存器相加，结果放在第 3 个寄存器中。这类指令包括 CDP 指令。

(2) 协处理器数据传送指令。这类指令从寄存器读取数据装入协处理器寄存器，或将协处理器寄存器的数据装入存储器。因为协处理器可以支持自己的数据类型，所以每个寄存器传送的字数与协处理器有关。ARM 处理器产生存储器地址，但传送的字节由协处理器控制。这类指令包括 LDC 指令和 STC 指令。

(3) 协处理器寄存器传送指令。在某些情况下，需要 ARM 处理器和协处理器之间传送数据。如一个浮点运算协处理器，FIX 指令从协处理器寄存器取得浮点数据，将它转换为整数，并将整数传送到 ARM 寄存器中。经常需要用浮点比较产生的结果来影响控制流，因此，比较结果必须传送到 ARM 的 CPSR 中。这类协处理器寄存器传送指令包括 MCR 和 MRC。

如表所示列出了所有协处理器处理指令。

表 协处理器指令

助 记 符	操 作
CDP	协处理器数据操作
LDC	装载协处理器寄存器
MCR	从 ARM 寄存器传数据到协处理器寄存器
MRC	从协处理器寄存器传数据到 ARM 寄存器
STC	存储协处理器寄存器

下面简单介绍一下比较常用的 MCR 及 MRC 命令的用法：



1、ARM 寄存器到协处理器寄存器的数据传送指令 MCR

1> 指令编码格式

ARM 寄存器到协处理器寄存器的数据传送指令 MCR（Move to Coprocessor from ARM Register）将 ARM 寄存器<Rd> 的值传送到协处理器寄存器 cp_num 中。如果没有协处理器执行指定操作，将产生未定义指令异常。指令的编码格式如图 3-2 所示。

31	28	27	24	23	21	20	19	16	15	12	11	8	7	5	4	3	0
cond	1110	opcode_1	0	CRn	Rd	cp_num	opcode_2	1	CRm								

图 3-2 MCR 指令编码格式

2> 指令的语法格式

```
MCR{<cond>} <coproc>, <opcode_1>, <Rd>, <CRn>, <CRm> {<opcode_2>}
```

① <cond>

为指令编码中的条件域。它指示指令在什么条件下执行。当<cond>忽略时，指令为无条件执行（cond=AL（Alway））。

② <coproc>

指定协处理器的编号，标准的协处理器的名字为 p0、p1、…、p15。

③ <opcode_1>

指定协处理器执行的操作码，确定哪一个协处理器指令将被执行。

④ <Rd>

确定哪一个 ARM 寄存器的数值将被传送。如果程序计数器 PC 的值被传送，指令的执行结果不可预知。

⑤ <CRn>

确定包含第一个操作数的协处理器寄存器。

⑥ <CRm>

确定包含第二个操作数的协处理器寄存器。

⑦ <opcode_2>

指定协处理器执行的操作码，确定哪一个协处理器指令将被执行。通常与<opcode_1>配合使用。

3> 指令举例

将 ARM 寄存器 r7 中的值传送到协处理器 p14 的寄存器 c7 中，第一操作数 opcode_1=1，第二操作数 opcode_2=6。

```
MCR p14, 1, r7, c7, c12, 6
```

4> 指令的使用

指令的编码格式中，bits[31：24]、bit[20]、bits[15：8]和 bit[4]为 ARM 体系结构定义。其他域由各生产商定义。硬件协处理器支持与否完全由生产商定义，某款 ARM 芯片中，是否支持协处理器或支持哪个协处理器与 ARM 版本无关。生产商可以选择实现部分协处理器指令或者完全不支持协处理器。



2、协处理器寄存器到 ARM 寄存器的数据传送指令 MRC

1> 指令编码格式

协处理器寄存器到 ARM 寄存器的数据传送指令 MRC（Move to ARM register from Coprocessor）将协处理器 `cp_num` 的寄存器的值传送到 ARM 寄存器中。如果没有协处理器执行指定操作，将产生未定义指令异常。指令的编码格式如图所示。

31	28	27	24	23	21	20	19	16	15	12	11	8	7	5	4	3	0
cond		1110		opcode_1		1	CRn	Rd		cp_num		opcode_2		1	CRm		

图 MRC 指令编码格式

2> 指令的语法格式

```
MRC{<cond>} <coproc>, <opcode_1>, <Rd>, <CRn>, <CRm>{, <opcode_2>}
```

为指令编码中的条件域。它指示指令在什么条件下执行。当<cond>忽略时，指令为无条件执行

（cond=AL（Alway））。

① <coproc>

指定协处理器的编号，标准的协处理器的名字为 p0、p1、…、p15。

② <opcode_1>

指定协处理器执行的操作码，确定哪一个协处理器指令将被执行。

③ <Rd>

确定哪一个 ARM 寄存器接受协处理器传送的数值。如果程序计数器 PC 被用做目的寄存器，指令的执行结果不可预知。

④ <CRn>

确定包含第一个操作数的协处理器寄存器。

⑤ <CRm>

确定包含第二个操作数的协处理器寄存器。

⑥ <opcode_2>

指定协处理器执行的操作码，确定哪一个协处理器指令将被执行。通常与<opcode_1>配合使用。

3> 指令举例

协处理器源寄存器为 c0 和 c2，目的寄存器为 ARM 寄存器 r4，第一操作数 opcode_1=5，第二操作数 opcode_2=3。

```
MRC p15, 5, r4, c0, c2, 3
```

4> 指令的使用

如果目的寄存器为程序计数器 r15，则程序状态字条件标准位根据传送数据的前 4bit 确定，后 28bit 被忽略。指令的编码格式中，bits[31:24]、bit[20]、bits[15:8]和 bit[4]为 ARM 体系结构定义。其他域由各生产商定义。

硬件协处理器支持与否完全由生产商定义，某款 ARM 芯片中，是否支持协处理器或支持哪个协处理



器与 ARM 版本无关。生产商可以选择实现部分协处理器指令或者完全不支持协处理器。

如果协处理器必须完成一些内部工作来准备一个 32 位数据向 ARM 传送（例如，浮点 FIX 操作必须将浮点值转换为等效的定点值），那么这些工作必须在协处理器提交传送前进行。因此，在准备数据时经常需要协处理器握手信号处于“忙—等待”状态。ARM 可以在忙—等待时间内产生中断。如果它确实得以中断，那么它将暂停握手以服务中断。当它从中断服务程序返回时，将可能重试协处理器指令，但也不可能不重试。例如，中断可能导致任务切换，无论哪种情况，协处理器必须给出一致结果，因此，在握手提交阶段之前的准备工作不允许改变处理器的可见状态。

如图所示列出了 cp15 的各个寄存器的目的。

寄存器编号	基本作用	在 MMU 中的作用	在 PU 中的作用
0	ID 编码（只读）	ID 编码和 cache 类型	
1	控制位（可读写）	各种控制位	
2	存储保护和控制	地址转换表基地址	Cachability 的控制位
3	存储保护和控制	域访问控制位	Bufferability 控制位
4	存储保护和控制	保留	保留
5	存储保护和控制	内存失效状态	访问权限控制位
6	存储保护和控制	内存失效地址	保护区域控制
7	高速缓存和写缓存	高速缓存和写缓存控制	
8	存储保护和控制	TLB 控制	保留
9	高速缓存和写缓存	高速缓存锁定	
10	存储保护和控制	TLB 锁定	保留
11	TCM ACCESS	NULL	NULL
12	异常向量表基地址	NULL	NULL
13	进程标识符	进程标识符	
14	保留		
15	因设计而异	因设计而异	因设计而异

图 cp15 寄存器列表

3.2.7 异常产生指令

ARM 指令集中提供了两条产生异常的指令，通过这两条指令可以用软件的方法实现异常。如表所示为 ARM 异常产生指令。

表 ARM 异常产生指令

助 记 符	含 义	操 作
SWI	软中断指令	产生软中断，处理器进入管理模式
BKPT	断点中断指令	处理器产生软件断点

软件中断指令（Software Interrupt, SWI）用于产生软中断，从而实现从用户模式变换到管理模式，CPSR 保存到管理模式的 SPSR 中，执行转移到 SWI 向量，在其他模式下也可以使用 SWI 指令，处理器同样切换到管理模式。

（1） 指令的语法格式。

```
SWI{<cond>} <immed_24>
```

（2） 指令举例。

① 下面指令产生软中断，中断立即数为 0。

```
SWI 0;
```



② 产生软中断，中断立即数为 0x123456。

```
SWI    0x123456;
```

③ 使用 SWI 指令时，通常使用以下两种方法进行参数传递。

a.指令 24 位的立即数指定了用户请求的类型，中断服务程序的参数通过寄存器传递。

下面的程序产生一个中断号为 12 的软中断。

```
MOV    R0, #34                ;设置功能号为 34
SWI     12                    ;产生软中断，中断号为 12
```

b.另一种情况，指令中的 24 位立即数被忽略，用户请求的服务类型由寄存器 R0 的值决定，参数通过其他寄存器传递。

下面的例子通过 R0 传递中断号，R1 传递中断的子功能号。

```
MOV    R0, #12                ;设置 12 号软中断
MOV    R1, #34                ;设置功能号为 34
SWI     0
```

3.2.8 其他指令介绍

1、特殊指令介绍

Fmxr /Fmrx 指令是 NEON 下的扩展指令，在做浮点运算的时候，要先打开 vfp，因此需要用到 Fmxr 指令。

Fmxr: 由 arm 寄存器将数据转移到协处理器中。

Fmrx: 由协处理器转移到 arm 寄存器中。

如图所示为浮点异常寄存器格式。



图 浮点异常寄存器格式

如表所示为 FPEXC 的位定义。

表 FPEXC 的位定义

位	域	功能描述
[31]	EX	异常位，该位指定了有多少信息需要存储记录 SIMD/VFP 协处理器的状态
[30]	EN	NEON/VFP 使能位，设置 EN 位 1 则开启 NEON/VFP 协处理器，复位会将 EN 置 0
[29:0]		保留

FPEXC<浮点异常寄存器>，该寄存器是一个可控制 SIMD 及 VFP 的全局使能寄存器，并指定了这些扩展技术是如何记录的。

如果要打开 VFP 协处理器的话，可以用以下指令：



```
mov r0, #0x40000000
fmxr    fpexc, r0    @ enable NEON and VFP coprocessor
```

2、CLZ 计算前导零数目

(1) 语法格式:

```
CLZ {cond} Rd,Rm
```

其中:

- cond 是一个可选的条件代码。
- Rd 是目标寄存器。
- Rm 是操作数寄存器。

(1) 用法: CLZ 指令对 Rm 中的值的前导零进行计数, 并将结果返回到 Rd 中, 如果未在源寄存器中设置任何位, 则该结果值为 32, 如果设置了位 31, 则结果值为 0。

(2) 条件标记: 该指令不会更改标记。

(3) 体系结构: ARMv5 以上。

(4) 示例如图所示。

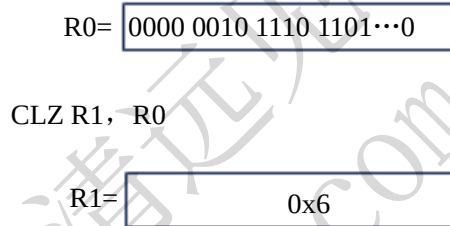


图 CLZ 例子

3、饱和指令介绍

这是用来设计饱和算法的一组指令, 所谓饱和是指出现下列 3 种情况:

- (1) 对于有符号饱和运算, 如果结果小于 -2^n , 则返回结果将为 -2^n 。
- (2) 对于无符号饱和运算, 如果整数结果是负值, 那么返回的结果将为 0。
- (3) 对于结果大于 2^n-1 的情况, 则返回结果将为 2^n-1 。

只要出现这情况, 就称为饱和, 并且饱和指令会设置 Q 标记, 下面简单介绍一下 QADD 带符号加法。

QSUB: 带符号减法。

QDADD: 带符号加倍加法。

QDSUB: 带符号加倍减法。

将结果饱和导入符号范围 $(-2^{31} \leq x \leq 2^{31}-1)$ 内。

(1) 语法格式:

```
op{cond} {Rd} ,Rm,Rn
```

其中:

- op 是 QADD, QSUB, QDADD, QDSUB 之一。



- `cond` 是一个可选的条件代码。
- `Rd` 是目标寄存器。
- `Rm, Rn` 是存放操作数的寄存器（注：不要将 `r15` 用做 `Rd, Rm` 或 `Rn`）。

（2） 用法如下：

- `QADD` 指令可将 `Rm` 和 `Rn` 中的值相加。
- `QSUB` 指令可从 `Rm` 中的值减去 `Rn` 中的值。
- `QDADD/QDSUB` 指令涉及并行指令，因此这里不多做讨论。

条件标记：如果发生饱和，则这些指令设置 `Q` 标记，若要读取 `Q` 标记的状态，需要使用 `MRS` 指令。

体系结构：该指令可用于 `v5T-E` 及 `v6` 或者更高版本的体系中。

（3） 示例如下：

```
QADD r0 ,r1,r9
QSUBLT r9,r0,r1
```

华清远见
fsdev.com.cn



第 4 章 ARM 汇编语言程序设计

在第 2、3 章中阐述的体系结构及指令集理论的基础上，本章主要介绍利用 ARM 汇编语言进行编程。ARM 编译器可以支持汇编语言、C/C++、汇编语言与 C/C++ 的混合编程等，本章将介绍汇编、C 相关的编程方法。

本章主要内容：

- GNU ARM 汇编伪操作。
- GNU ARM 汇编支持的伪指令。
- 汇编语言与 C 的混合编程。

4.1 GNU ARM 汇编器支持的伪操作

4.1.1 伪操作概述

在 ARM 汇编语言程序中，有一些特殊指令助记符，这些助记符与指令系统的助记符不同，没有相对应的操作码，通常称这些特殊指令助记符为伪操作标识符（directive），它们所完成的操作称为伪操作。伪操作在源程序中的作用是为了完成汇编程序做各种准备工作的，这些伪操作仅在汇编过程中起作用，一旦汇编结束，伪操作的使命就完成。

在 ARM 的汇编程序中，伪操作主要有符号定义伪操作、数据定义伪操作、汇编控制伪操作及其杂项伪操作等。

4.1.2 数据定义（Data Definition）伪操作

数据定义伪操作一般用于为特定的数据分配存储单元，同时可完成已分配存储单元的初始化。常见的数据定义伪操作有 .byte、.short、.long、.quad、.float、.string、.asciz、.ascii 和 .rept。数据定义伪操作如下。

（1）伪指令名：

```
.byte
```

用途：单字节定义。

用法：

```
.byte 1,2,0b01,0x34,072,'s' ;
```

（2）伪指令名：

```
.short
```

用途：定义双字节数据，

用法：

```
.short 0x1234,60000 ;
```

（3）伪指令名：

```
.long
```

用途：定义 4 字节数据。

用法：

```
.long 0x12345678,23876565
```

（4）伪指令名：



```
.quad:
```

用途：定义 8 字节。

用法：

```
.quad 0x1234567890abcd
```

(5) 伪指令名：

```
.float
```

用途：定义浮点数。

用法：

```
.float 0f311971.693993751E-40
```

(6) 伪指令名：

```
.string/.asciz/.ascii:
```

用途：定义多个字符串。

用法：

```
.string "abcd", "efgh", "hello!"
.asciz "qwer", "sun", "world!"
.ascii "welcome\0" (需要注意的是：.ascii 伪操作定义的字符串需要在每行末尾添加结尾
字符'\0')
```

(7) 伪指令名：

```
.rept/.endr
```

用途：重复定义伪操作。

用法：

```
.rept 3
.byte 0x23
.endr
```

(8) 伪指令名：

```
.equ/.set
```

用途：赋值语句，.equ(.set) 变量名，表达式。

用法：

```
.equ abc 3 @ abc=3
```

4.1.3 汇编控制伪操作

汇编控制伪操作用于控制汇编程序的执行流程，常用的汇编控制伪操作包括以下几条。

1、.if、.else、.endif

a) 语法格式

.if、.else、.endif 伪操作能根据条件的成立与否决定是否执行某个指令序列。当.if 后面的逻辑表达式为真，则执行.if 后的指令序列，否则执行.else 后的指令序列。其中，.else 及其后指令序列可以没有，此时，当.if 后面的逻辑表达式为真，则执行指令序列，否则继续执行后面的指令。

提示：

.if、.else、.endif 伪指令可以嵌套使用。



语法格式如下:

```
.if logical-expressing
...
{.else
...}
.endif logical-expression:
```

用于决定指令执行流程的逻辑表达式。

b) 使用说明

当程序中有一段指令需要在满足一定条件时执行, 使用该指令。该操作还有另一种形式。

```
.if logical-expression
    Instruction
.elseif logical-expression2
    Instructions
.endif
```

该形式避免了 if-else 形式的嵌套, 使程序结构更加清晰、易读。

2、.macro、.endm

a) 语法格式

.macro 伪操作可以将一段代码定义为一个整体, 称为宏指令, 然后就可以在程序中通过宏指令多次调用该段代码。其中, \$ 标号在宏指令被展开时, 标号会被替换为用户定义的符号。

宏操作可以使用一个或多个参数, 当宏操作被展开时, 这些参数被相应的值替换。

宏操作的使用方式和功能与子程序有些相似, 子程序可以提供模块化的程序设计、节省存储空间并提高运行速度。但在使用子程序结构时需要保护现场, 从而增加了系统的开销, 因此, 在代码较短且需要传递的参数较多时, 可以使用宏操作代替子程序。

包含在 .macro 和 .endm 之间的指令序列称为宏定义体, 在宏定义体的第一行应声明宏的原型 (包含宏名、所需的参数), 然后就可以在汇编程序中通过宏名来调用该指令序列。在源程序被编译时, 汇编器将宏调用展开, 用宏定义中的指令序列代替程序中的宏调用, 并将实际参数的值传递给宏定义中的形式参数。

提示:

.macro、.endm 伪操作可以嵌套使用。

语法格式如下:

```
.macro
{ $label } macroname { $parameter { , $parameter } ... }
; code
.endm
```

- (1) { \$label }。\$ 标号在宏指令被展开时, 标号会被替换为用户定义的符号。通常, 在一个符号前使用 “\$” 表示该符号被汇编器编译时, 使用相应的值代替该符号。
- (2) **Macroname**: 所定义的宏的名称。
- (3) **Parameter**: 宏指令的参数。当宏指令被展开时将被替换成相应的值, 类似于函数中的参数。



b) 使用说明

在子程序代码比较短，而需要传递的参数比较多的情况下可以使用宏汇编技术。

首先通过 `.macro` 和 `.endm` 伪操作定义宏，包括宏定义体代码。在 `.macro` 伪操作之后的第一行声明宏的原型，其中包含该宏定义的名称及需要的参数。在汇编中可以通过该宏定义的名称来调用它。当源程序被编译时，汇编器将展开每个宏调用，用宏定义体代替源程序中宏定义的名称，并用实际参数值代替宏定义时的形式参数。

c) 示例

示例如下：

```
.macro SHIFTLLEFT a, b
.if \b < 0
MOV \a, \a, ASR #-\b
.exitm
.endif
MOV \a, \a, LSL #\b
.endm
```

3、.mexit

a) 语法格式

`.mexit` 用于从宏定义中跳转出去。

b) 用法

只需要在宏定义的代码中插入该指令即可。

```
.macro SHIFTLLEFT a, b
.if \b < 0
mov \a, \a, ASR #-\b
.exitm
.endif
mov \a, \a, LSL #\b
.endm
```

4.1.4 杂项伪操作

ARM 汇编中还有一些其他的伪操作，在汇编程序中经常会被使用，包括以下几条。

<code>.arm</code>	<code>.arm</code>	@ 定义以下代码使用 ARM 指令集编译
<code>.code 32</code>	<code>.code 32</code>	@作用同 <code>.arm</code>
<code>.code 16</code>	<code>.code 16</code>	@作用同 <code>.thumb</code>
<code>.thumb</code>	<code>.thumb</code>	@定义以下代码使用 Thumb 指令集编译
<code>.section</code>	<code>.section expr</code>	@定义域中包含的段。expr 可以使 <code>.text</code> , <code>.data</code> , <code>.bss</code>
<code>.text</code>	<code>.text {subsection}</code>	@将定义符开始的代码编译到代码段或代码子段 (subsection)
<code>.data</code>	<code>.data {subsection}</code>	@将定义符开始的代码编译到数据段或数据子段 (subsection)
<code>.bss</code>	<code>.bss {subsection}</code>	@将变量存放到 <code>.bss</code> 段或 <code>.bss</code> 的子段 (subsection)
<code>.align</code>	<code>.align{alignment}{,fill}{,max}</code>	@通过用零或指定的数据进行填充来使当前位置



与指定边界对齐

`.org            .org offset{,expr}    @指定从当前地址加上 offset 开始存放代码，并且从当前地址到当前地址加上 offset 之间的内存单元，用零或指定的数据进行填充`

4.2 ARM 汇编器支持的伪指令

ARM 汇编器支持 ARM 伪指令，这些伪指令在汇编阶段被翻译成 ARM 或者 Thumb（或 Thumb-2）指令（或指令序列）。ARM 伪指令包含 ADR、ADRL、LDR 等。

4.2.1 ADR 伪指令

1、语法格式

ADR 伪指令为小范围地址读取伪指令。ADR 伪指令将基于 PC 相对偏移地址或基于寄存器相对偏移地址值读取到寄存器中，当地址值是字节对齐时，取值范围为-255~255，当地址值是字对齐时，取值范围为-1020~1020。当地址值是 16 字节对齐时其取值范围更大。

语法格式如下：

```
ADR{cond}{.W} register,label
```

- a) **cond**
可选的指令执行条件。
- b) **.W**
可选项。指定指令宽度（Thumb-2 指令集支持）。
- c) **register**
目标寄存器。
- d) **label**
基于 PC 或具有寄存器的表达式。

2、使用说明

ADR 伪指令被汇编器编译成一条指令。汇编器通常使用 ADD 指令或 SUB 指令来实现伪操作的地址装载功能。如果不能用一条指令来实现 ADR 伪指令的功能，汇编器将报告错误。

3、示例

示例如下。

```
LDR    R4,=data+4*n      ;n 是汇编时产生的变量
; code
MOV    pc,lr
data   .word    value0
; n-1 条 DCD 伪操作
.word    valuen          ;所要装载入 R4 的值
;更多 DCD 伪操作
```



4.2.2 ADRL 伪指令

1、语法格式

ADRL 伪指令为中等范围地址读取伪指令。ADRL 伪指令将基于 PC 相对偏移的地址或基于寄存器相对偏移的地址值读取到寄存器中，当地址值是字节对齐时，取值范围为-64~64KB；当地址值是字对齐时，取值范围为-256~256KB。当地址值是 16 字节对齐时，其取值范围更大。在 32 位的 Thumb-2 指令中，地址取值范围到达-1~1MB。

语法格式如下：

```
ADRL{cond} register, label
```

a) cond

可选的指令执行条件。

b) register

目标寄存器。

c) label

基于 PC 或具体寄存器的表达式。

2、使用说明

ADRL 伪指令与 ADR 伪指令相似，用于将基于 PC 相对偏移的地址或基于寄存器相对偏移的地址值读取到寄存器中。所不同的是，ADRL 伪指令比 ADR 伪指令可以读取更大范围的地址。这是因为在编译阶段，ADRL 伪指令被编译器换成两条指令。即使一条指令可以完成该操作，编译器也将产生两条指令，其中一条为多余指令。如果汇编器不能在两条指令内完成操作，将报告错误，中止编译。

4.2.3 LDR 伪指令

1、语法格式

LDR 伪指令装载一个 32 位的常数和地址到寄存器。

语法格式如下：

```
LDR{cond}{.W} register,=[expr|label-expr]
```

(1) Cond

可选的指令执行条件。

(2) .W

可选项。指定指令宽度（Thumb-2 指令集支持）。

(3) register

目标寄存器。

(4) expr

32 位常量表达式。汇编器根据 expr 的取值情况，对 LDR 伪指令做如下处理。

① 当 expr 表示的地址值没有超过 MOV 指令或 MVN 指令的地址取值范围时，汇编器用一对 MOV 和 MVN 指令代替 LDR 指令。



② 当 `expr` 表示的指令地址值超过了 `MOV` 指令或 `MVN` 指令的地址范围时，汇编器将常数放入数据缓存池，同时用一条基于 `PC` 的 `LDR` 指令读取该常数。

(5) label-expr

一个程序相关或声明为外部的表达式。汇编器将 `label-expr` 表达式的值放入数据缓存池，使用一条程序相关 `LDR` 指令将该值取出放入寄存器。

当 `label-expr` 被声明为外部的表达式时，汇编器将在目标文件中插入链接重定位伪操作，由链接器在链接时生成该地址。

2、使用说明

当要装载的常量超出了 `MOV` 指令或 `MVN` 指令的范围时，使用 `LDR` 指令。

由 `LDR` 指令装载的地址是绝对地址，即 `PC` 相关地址。

当要装载的数据不能由 `MOV` 指令或 `MVN` 指令直接装载时，该值要先放入数据缓存池，此时 `LDR` 伪指令处的 `PC` 值到数据缓存池中目标数据所在地址的偏移量有一定限制。`ARM` 或 32 位的 `Thumb-2` 指令中该范围是 -4~4KB，`Thumb` 或 16 位的 `Thumb-2` 指令中该范围是 0~1KB。

3、示例

(1) 将常数 `0xff0` 读到 `R1` 中。

```
LDR R3,=0xff0 ;
```

相当于下面的 `ARM` 指令：

```
MOV R3,#0xff0
```

(2) 将常数 `0xffff` 读到 `R1` 中。

```
LDR R1,=0xffff ;
```

相当于下面的 `ARM` 指令：

```
LDR R1,[pc,offset_to_litpool]
...
litpool .word 0xffff
```

(3) 将 `place` 标号地址读入 `R1` 中。

```
LDR R2,=place ;
```

相当于下面的 `ARM` 指令：

```
LDR R2,[pc,offset_to_litpool]
...
litpool .word place
```

4.3 GNU ARM 汇编语言的语句格式

在汇编语言程序设计中，经常使用各种符号代替地址（`addresses`）、变量（`variables`）和常量（`constants`）等，以增加程序的灵活性和可读性。尽管符号的命名由编程者决定，但并不是任意的，必须遵循以下的约定。

(1) 符号区分大小写，同名的大、小写符号会被编译器认为是两个不同的符号。



(2) 符号在其作用范围内必须唯一。

(3) 自定义的符号名不能与系统的保留字相同。其中保留字包括系统内部变量（built in variable）和系统预定义（predefined symbol）的符号。

(4) 符号名不应与指令或伪指令同名。如果要使用和指令或伪指令同名的符号要用双斜杠“//”将其括起来，如“//SSERT//”。

注意：虽然符号被双斜杠括起来，但双斜杠并非符号名的一部分。

(5) 局部标号以数字开头，其他的符号都不能以数字开头。

1、变量（variable）

程序中的变量是指其值在程序的运行过程中可以改变的量。ARM（Thumb）汇编程序所支持的变量有 3 种。

- 数字变量（numeric）。
- 逻辑变量（logical）。
- 字符串变量（string）。

数字变量用于在程序的运行中保存数字值，但注意数字值的大小不应超出数字变量所能表示的范围。

逻辑变量用于在程序的运行中保存逻辑值，逻辑值只有两种取值情况：真（{TURE}）和假（{FALSE}）。

字符串变量用于在程序的运行中保存一个字符串，注意字符串的长度不应超出字符串变量所能表示的范围。

在 ARM（Thumb）汇编语言程序设计中，可使用 GBLA、GBLL、GBLS 伪指令声明全局变量，使用 LCLA、LCLL、LCLS 伪指令声明局部变量，可使用 SETA、SETL 和 SETS 对其进行初始化。

2、常量（constants）

程序中的常量是指其值在程序的运行过程中不能被改变的量。ARM（Thumb）汇编程序所支持的常量有数字常量、逻辑常量和字符串常量。

数字常量一般为 32 位的整数，当作为无符号数时，其取值范围为 $0 \sim 2^{32}-1$ ，当作为有符号数时，其取值范围为 $-2^{31} \sim 2^{31}-1$ 。汇编器认为 $-n$ 和 $2^{32}-n$ 是相等的。对于关系操作，如比较两个数的大小，汇编器将其操作数看做无符号的数，也就是说“ $0 > -1$ ”，对汇编器来说取值为“假（{FLASE}）”。

逻辑常量只有两种取值情况，真或假。

字符串常量为一个固定的字符串，一般用于程序运行时的信息提示。

3、程序中的变量代换

汇编语言中的变量可以作为一整行出现在汇编程序中，也可以作为行的一部分使用。如果在数字变量前面有一个代换操作符“\$”，编译器会将该数字变量的值转换为十六进制的字符串，并将该十六进制的字符串代换“\$”后的数字变量。

如果在逻辑变量前面有一个代换操作符“\$”，编译器会将该逻辑变量代换为它的取值（真或假）。如



果在字符串变量前面有一个代换操作符“\$”，编译器会将该字符串变量的值代换“\$”后的字符串变量。如果程序中需要字符“\$”，则可以用“\$\$”来表示。汇编器将不进行变量替换，而是将“\$\$”作为“\$”。

4、程序标号（label）

在 ARM 汇编中，标号代表一个地址，段内标号的地址在汇编时确定，而段外标号地址值在链接时确定。根据标号的生成方式，程序标号分为以下 3 种。

- 程序相关标号（Program-relative labels）。
- 寄存器相关标号（Register-relative labels）。
- 绝对地址（Absolute address）。

（1）程序相关标号

程序相关标号指位于目标指令前的标号或程序中的数据定义伪操作前的标号。这种标号在汇编时将被处理成 PC 值加上或减去一个数字常量。它常用于表示跳转指令的目标地址或代码段中所嵌入的少量数据。

（2）寄存器相关地址

这种标号在汇编时将被处理成寄存器的值加上或减去一个数字常量。它常被用于访问数据段中的数据。这种基于寄存器的标号通常用 MAP 和 FIELD 伪操作定义，也可以用 EQU 伪操作定义。

（3）绝对地址

绝对地址是一个 32 位的数字量，使用它可以直接寻址整个内存空间。

5、局部标号

局部标号是一个 0~99 之间的十进制数字，可重复定义。局部标号后面可以紧接一个通常表示该局部变量作用范围的符号。局部变量的作用范围为当前段，也可以用伪操作 ROUT 来定义局部标号的作用范围。

局部标号在子程序或程序循环中常被用到，也可以配合宏定义伪操作（.MACRO 和.MEND）来使程序结构更加合理。在同一个段中，可以使用相同的数字命名不同的局部变量。默认情况下，汇编器会寻址最近的变量。也可以通过汇编器命令选项来改变搜索顺序。

4.4 ARM 汇编语言的程序结构

4.4.1 汇编语言的程序格式

在 ARM（Thumb）汇编语言程序中可以使用.section 来进行分段，其中每一个段用段名或者文件结尾为结束，这些段使用默认的标志，如 a 为允许段，w 为可写段，x 为执行段。

在一个段中，我们可以定义下列的子段：

- .text
- .data
- .bss
- .sdata



➤ .sbss

由此我们可知道，段可以分为代码段、数据段及其他存储用的段，.text（正文段）包含程序的指令代码；.data(数据段)包含固定的数据，如常量、字符串；.bss（未初始化数据段）包含未初始化的变量、数组等，当程序较长时，可以分割为多个代码段和数据段，多个段在程序编译链接时最终形成一个可执行的映像文件。

```
.section.data
< initialized data here>
.section .bss
< uninitialized data here>
.section .text
.globl _start
_start:
<instruction code goes here>
```

4.4.2 汇编语言子程序调用

在 ARM 汇编语言程序中，子程序的调用一般是通过 BL 指令来实现的。在程序中，使用指令“BL 子程序”名即可完成子程序的调用。

该指令在执行时完成如下操作：将子程序的返回地址存放在连接寄存器 LR 中，同时将程序计数器 PC 指向子程序的入口点。当子程序执行完毕需要返回调用处时，只需要将存放在 LR 中的返回地址重新复制给程序计数器 PC 即可。在调用子程序的同时，也可以完成参数的传递和从子程序返回运算的结果，通常可以使用寄存器 R0~R3 完成。

注意：

同编译器编译的代码间的相互调用，要遵循 AAPCS（ARM Architecture）。详见 ARM 编译工具手册。

以下是使用 BL 指令调用子程序的汇编语言源程序的基本结构：

```
.text
.global _start
_start:
LDR      R0, =0x3FF5000
LDR      R1, 0xFF
STR      R1, [R0]
LDR      R0, =0x3FF5008
LDR      R1, 0x01
STR      R1, [R0]
BL       PRINT_TEXT
...
PRINT_TEXT:
...
        MOV     PC, BL
...
```



4.4.3 过程调用标准 AAPCS

为了使不同编译器编译的程序之间能够相互调用，必须为子程序间的调用规定一定的规则。AAPCS 就是这样一个标准。所谓 AAPCS，其英文全称为 Procedure Call Standard for the ARM Architecture (AAPCS)，即 ARM 体系结构过程调用标准。它是 ABI (Application Binary Interface (ABI) for the ARM Architecture (base standard) [BSABI]) 标准的一部分。

可以使用 “--apcs” 选项告诉编译器将源代码编译成符号 AAPCS 调用标准的目标代码。

注意：

使用 “--apcs” 选项并不影响代码的产生，编译器只是在各段中放置相应的属性，标识用户选定的 AAPCS 属性。

1、AAPCS 相关的编译/汇编选项

- none: 指定输入文件不使用 AAPCS 规则。
- /interwork: 指定输入文件符合 ARM/Thumb 交互标准。
- /nointerwork: 指定输入文件不能使用 ARM/Thumb 交互。这是编译器默认选项。
- /ropi: 指定输入文件是位置无关只读文件。
- /noropi: 指定输入文件是非位置无关只读文件。这是编译器默认选项。
- /pic: 同/ropi。
- /nopi: 同/noropi。
- /rwpi: 指定输入文件是位置无关可读可写文件。
- /norwpi: 指定输入文件是非位置无关可读可写文件。
- /pid: 同/rwpi。
- /nopid: 同/norwpi。
- /fpic: 指定输入文件编译成位置无关只读代码。代码中地址是 FPIC 地址。
- /swstackcheck: 编译过程中对输入文件使用堆栈检测。
- /noswstackcheck: 编译过程中对输入文件不使用堆栈检测。这是编译器默认选项。
- /swstna: 如果汇编程序对于是否进行数据栈检查无所谓，而与该汇编程序连接的其他程序指定了选项/swst 或选项/noswst，这时该汇编程序使用选项/swstna。

2、ARM 寄存器使用规则

AAPCS 中定义了 ARM 寄存器使用规则如下：

子程序间通过寄存器 R0、R1、R2、R3 来传递参数。如果参数多于 4 个，则多出的部分用堆栈传递。被调用的子程序在返回前无须恢复寄存器 R0-R3 的内容。

在子程序中，使用寄存器 R4-R11 来保存局部变量。如果在子程序中使用到了寄存器 R4-R11 中的某些寄存器，子程序进入时必须保存这些寄存器的值，在返回前必须恢复这些寄存器的值；对于子程序中沒有用到的寄存器则不必进行这些操作。在 Thumb 程序中，通常只能使用寄存器 R4-R7 来保存局部变量。



寄存器 R12 用做子程序间 scratch 寄存器（用于保存 SP，在函数返回时使用该寄存器出栈），记作 ip。在子程序间的连接代码段中常有这种使用规则。

寄存器 R13 用做数据栈指针，记作 sp。在子程序中寄存器 R13 不能用做其他用途。寄存器 sp 在进入子程序时的值和退出子程序时的值必须相等。

寄存器 R14 称为连接寄存器，记作 lr。它用于保存子程序的返回地址。如果在子程序中保存了返回地址，寄存器 R14 则可以用做其他用途。

寄存器 R15 是程序计数器，记作 pc。它不能用做其他用途。

ARM 寄存器在函数调用过程中的保护规则，如图 4-1 所示。

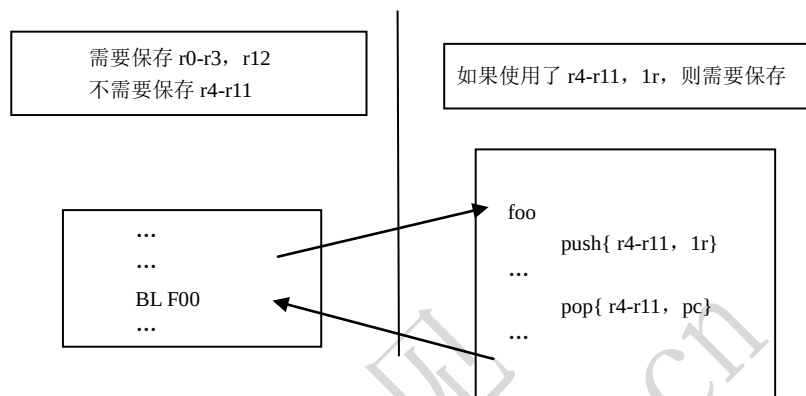


图 4-1 ARM 寄存器在函数调用中的保护规则

4.4.4 汇编语言程序设计举例

通过组合使用条件执行和条件标志设置，可简单地实现分支语句，不需要任何分支指令。这样可以改善性能，因为分支指令会占用较多的周期数；同时这样做也可以减小代码尺寸，提高代码密度。

下面是一段 C 语言程序，该程序实现了著名的 Euclid 最大公约数算法：

```
int gcd(int a, int b)
{
    while (a != b)
    {
        if (a > b)
            a = a - b;
        else
            b = b - a;
    }
    return a;
}
```

用 ARM 汇编语言重写来重写这个例子，如下所示。

```
Code1:
Gcd:
    CMP    r0, r1
    BEQ    end
    BLT    less
    SUB    r0, r0, r1
```



```

        B        gcd
Less:
        SUB      r1, r1, r0
        B        gcd
    
```

充分地利用条件执行修改上面的例子，得到 Code2。

```

Code2:
Gcd:
        CMP      r0, r1
        SUBGT    r0, r0, r1
        SUBLT    r1, r1, r0
        BNE      gcd
    
```

两段代码的比较如下。

- Code1: 仅使用了分支指令。
- Code2: 充分利用了 ARM 指令条件执行的特点，仅使用了 4 条指令就完成了全部算法。这对提供程序的代码密度和执行速度十分有帮助。

事实上，分支指令十分影响处理器的速度。每次执行分支指令，处理器都会排空流水线，重新装载指令。

4.5 汇编语言与 C 语言的混合编程

在 C 代码中实现汇编语言的方法有内联汇编和内嵌汇编两种，使用它们可以在 C 程序中实现 C 语言不能完成的一些工作。例如，在下面几种情况中必须使用内联汇编或嵌入型汇编。

4.5.1 GNU ARM 内联汇编

1、内联汇编语法

本小节简单介绍 GNU 风格的 ARM 内联汇编语法要点：

(1) 格式

格式如下：

```
asm volatile ("asm code": output: input: changed);
```

必须以 “;” 结尾，不管有多长对 C 都只是一条语句。

(2) asm 内嵌汇编关键字

volatile: 告诉编译器不要优化内嵌汇编，如果想优化可以不加。

(3) ANSI C 规范的关键字

ANSI C 规范的关键字如下。

```
__asm__
__volatile__ //前面和后面都有两个下画线，它们之间没有空格
```

如果后面部分没有内容，“:” 可以省略，前面或中间的不能省略 “:”，没有 asm code 也不可以省略 “””，没有 changed 必须省略 “:”。

2、汇编代码



汇编必须放在一个字符串内，但是字符串中间是不能直接按回车键换行的，可以写成多个字符串，只要字符串之间不加任何符号编译完后就会变成一个字符串：

```
"mov r0,r0\n\t" //指令之间必须要换行，\t 可以不加，只是为了在汇编文件中的指令格式
对齐
"mov r1,r1\n\t"
"mov r2,r2"
```

字符串内不是只能放指令，可以放一些标签、变量、循环、宏等，还可以把内嵌汇编放在 C 函数外面，用内嵌汇编定义函数、变量、段等，总之就跟直接在写汇编文件一样在 C 函数外面定义内嵌汇编时不能加 `volatile`、`output`、`input`、`changed`。

注意：编译器不检查 `asm code` 的内容是否合法，直接交给汇编器

3、output (ASM --> C) 和 input (C --> ASM)

(1) 指定输出值：

```
__asm__ __volatile__ (
    "asm code"
    : "constraint" (variable)
);
```

① `constraint` 定义 `variable` 的存放位置：

```
r :    使用任何可用的通用寄存器
m :    使用变量的内存地址
```

② `output` 修饰符：

```
+ :    可读可写
= :    只写
& :    该输出操作数不能使用输入部分使用过的寄存器，只能 +& 或 =& 方式使用
```

(2) 指定输入值：

```
__asm__ __volatile__ (
    "asm code"
    :
    : "constraint" (variable / immediate)
);
```

`constraint` 定义 `variable / immediate` 的存放位置：

```
r :    使用任何可用的通用寄存器（变量和立即数都可以）
m :    使用变量的内存地址（不能用立即数）
i :    使用立即数（不能用变量）
```

(3) 使用占位符：

```
int a = 100,b = 200;
int result;
__asm__ __volatile__ (
    "mov %0,%3\n\t" //mov    r3,#123    %0 代表 result, %3 代表 123 (编译器会自动加 #
号)
    "ldr r0,%1\n\t" //ldr    r0,[fp, #-12]    %1 代表 a 的地址
```



```
"ldr r1,%2\n\t" //ldr    r1,[fp, #-16]    %2 代表 b 的地址
"str r0,%2\n\t" /*str    r0,[fp, #-16]因为%1 和%2 是地址所以只能用 ldr 或 str 指令
*/
"str r1,%1\n\t" /*str    r1,[fp, #-12]如果用错指令编译时不会报错，要到汇编时才会报错*/
: "=r" (result), "+m" (a), "+m" (b) /*out1 是%0, out2 是%1, ..., outN 是%N-1*/
: "i" (123)                s /*in1 是%N, in2 是%N+1, ...*/);
```

(4) 引用占位符:

```
int num = 100;
__asm__ __volatile__ (
    "add    %0,%1,#100\n\t"
    : "=r" (a)
    : "0" (a)    //"0"是零，即%0，引用时不可以加 %，只能 input 引用 output);
    //引用是为了更能分清输出输入部分
```

(5) & 修饰符:

```
int num;
__asm__ __volatile__ (    //mov    r3, #123    //编译器自动加的指令
    "mov    %0,%1\n\t"    //mov    r3,r3    //输入和输出使用相同的寄存器
    : "=r" (num)
    : "r" (123)
);
int num;
__asm__ __volatile__ (
    //mov    r3, #123
    "mov    %0,%1\n\t"    //mov    r2,r3    //加了&后输入和输出的寄存器不一样了
    : "&r" (num)          //mov    r3, r2    //编译器自动加的指令
    : "r" (123)
);
```

4、内联汇编示例

下面通过一个例子进一步了解内联汇编的语法。该例子实现了位交换。

```
#include <stdio.h>
unsigned long ByteSwap(unsigned long val)
{
    int ch;
    asm volatile (
        "eor r3, %1, %1, ror #16\n\t"
        "bic r3, r3, #0x00FF0000\n\t"
        "mov %0, %1, ror #8\n\t"
        "eor %0, %0, r3, lsr #8"
        : "=r" (val)
        : "0" (val)
        : "r3"
    );
}
```



```

}
int main(void)
{
    unsigned long test_a = 0x1234,result;
    result = ByteSwap(test_a);
    printf("Result:%d\r\n", result);
    return 0;
}

```

4.5.2 混合编程调用举例

汇编程序、C 程序相互调用时，要特别注意遵守相应的 AAPCS 规则。下面一些例子具体说明了在这些混合调用中应注意遵守的 AAPCS 规则。

1、从 C 程序调用汇编语言

下面的程序显示了如何在 C 程序中调用汇编语言子程序，该段代码实现了将一个字符串复制到另一个字符串。

```

#include <stdio.h>
extern void strcpy(char *d, const char *s);
int main()
{
    const char *srcstr = "First string - source ";
    char dststr[] = "Second string - destination ";
    /* 下面将 dststr 作为数组进行操作 */
    printf("Before copying:\n");
    printf(" %s\n %s\n",srcstr,dststr);
    strcpy(dststr,srcstr);
    printf("After copying:\n");
    printf(" %s\n %s\n",srcstr,dststr);
    return(0);
}

```

下面为调用的汇编程序。

```

.global strcpy
strcpy:                                ;R0 指向目的字符串
                                        ;R1 指向源字符串
    LDRB R2, [R1],#1                  ;加载字节并更新源字符串指针地址
    STRB R2, [R0],#1                  ;存储字节并更新目的字符串指针地址
    CMP R2, #0                        ;判断是否为字符串结尾
    BNE strcpy                        ;如果不是，程序跳转到 strcpy 继续复制
    MOV pc,lr                         ;程序返回

```

2、从汇编语言调用 C 程序

下面的例子显示了如何从汇编语言调用 C 程序。

下面的子程序段定义了 C 语言函数。



```
int g(int a, int b, int c, int d, int e)
{
    return a + b + c + d + e;
}
```

下面的程序段显示了汇编语言调用。假设程序进入 f 时，R0 中的值为 i。

```
; int f(int i) { return g(i, 2*i, 3*i, 4*i, 5*i); }
.text
.global _start
_start:
    STR lr, [sp, #-4]!           // 保存返回地址 lr
    ADD R1, R0, R0               // 计算 2*i (第 2 个参数)
    ADD R2, R1, R0               // 计算 3*i (第 3 个参数)
    ADD R3, R1, R2               // 计算 5*i
    STR R3, [sp, #-4]!           // 第 5 个参数通过堆栈传递
    ADD R3, R1, R1               // 计算 4*i (第 4 个参数)
    BL g                         // 调用 C 程序
    ADD sp, sp, #4               // 从堆栈中删除第 5 个参数
    LDR pc, [sp], #4             // 返回
```



第 5 章 裸机实验

5.1 汇编点灯实验

5.1.1 【实验目的】

- (1) 熟悉 eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 掌握裸机实验的基本操作步骤
- (4) 通过本实验了解汇编的基本操作指令
- (5) 如何使用 IO 口控制 LED 的亮灭

5.1.2 【实验环境】

- (1) FS6818 开发平台
- (2) Eclipse 集成开发环境
- (3) PC 机 windows 操作系统

5.1.3 【实验内容】

编写汇编程序，通过汇编程序控制 FS6818 开发板上 RGB 红灯点亮，在此过程中了解如何使用简单的汇编指令，如何在汇编语言中配置 S5P6818 的 GPIO 寄存器为 GPIO 模式以及输入输出功能。

5.1.4 【实验原理】

下图是底板 RGB 灯的电路原理图。

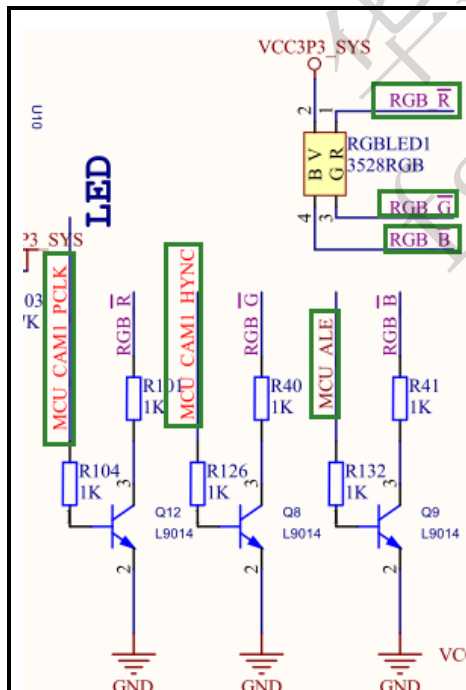


图 底板 RGB 连接

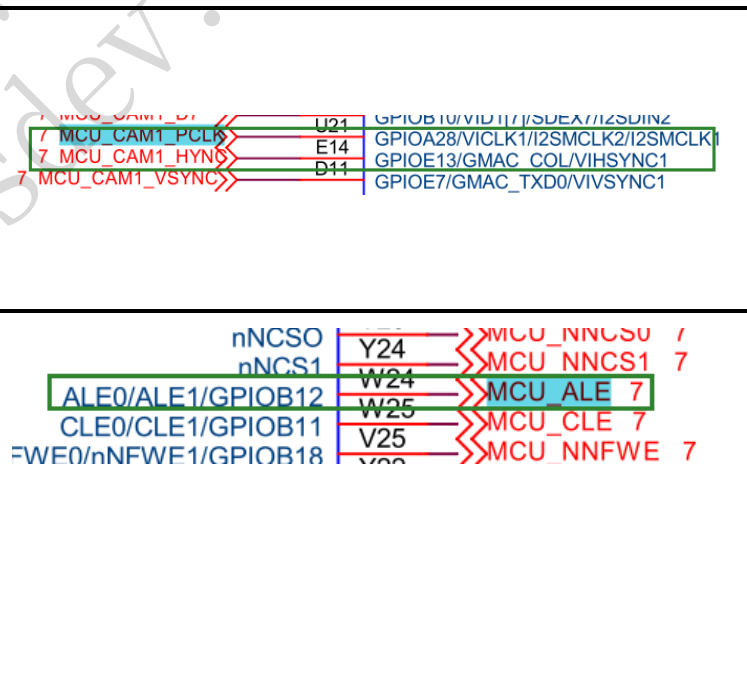


图 底板与核心板连接引脚

通过如上的底板 RGB 连接原理分析得知，灯的亮灭是由三极管是否导通控制的，当



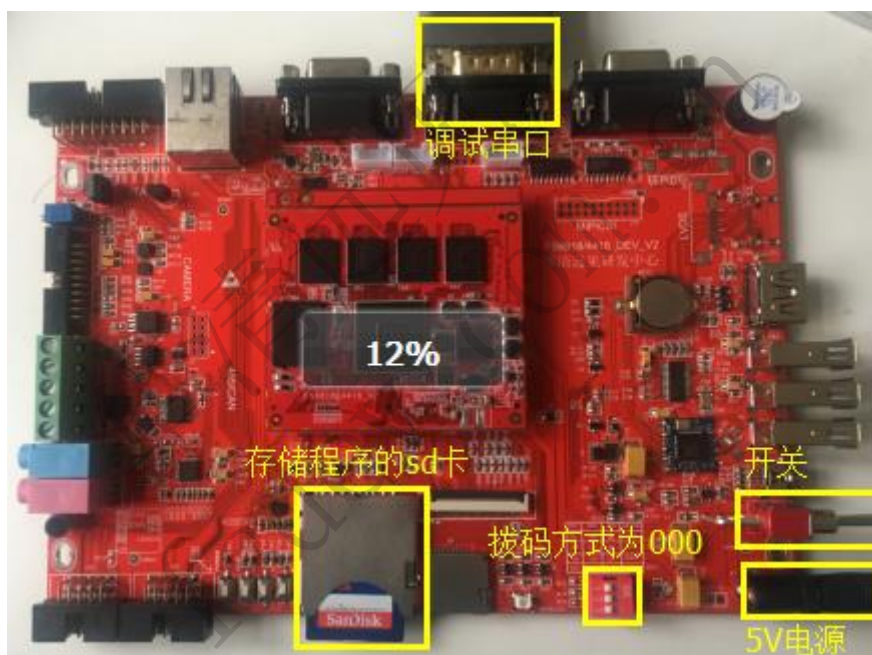
MCU_CAM1_PCLK 为高电平时三极管导通，此时 RGB_R 引脚为低电平右上角的 RGBLED1 中的 R 导通，此时红灯亮，相反则灭。由右侧的底板与核心板的连接图可以得知 RGB 的 R、G、B 引脚分别与 GPIOA28、GPIOE13、GPIOB12 相连，那么此时我们只需要控制相应的 IO 引脚输出高低电平即可控制灯的色彩。

底板引脚	核心板引脚	内核 IO 声明
MCU_CAM1_PCLK	GPIOA28	PAD_GPIO_A + 28
MCU_CAM1_HYNC	GPIOE13	PAD_GPIO_E + 13
MCU_ALE	GPIOB12	PAD_GPIO_B + 12

查找到相应的 GPIO 编号之后，需要找到相应的芯片手册查看该寄存器的地址及配置方式。

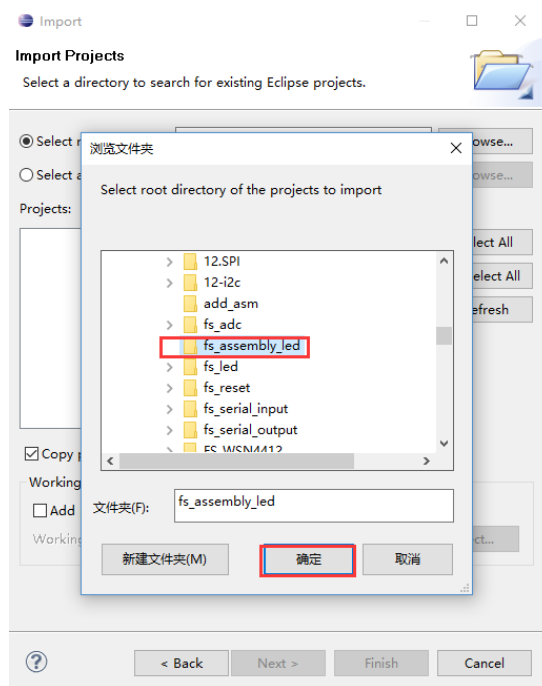
5.1.5 【实验步骤】

1. 硬件连接，需接好电源线，将启动方式调整为 SD 卡启动，接好调试串口并配置好串口调试工具。



2. 用户确认已经将 fs_assembly_led 工程导入到 eclipse 开发环境中。

工程路径【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:程序源码\ARM 裸机实验源码】，选择 eclipse 中 File 选项下的 import 导入工程，选择对应的工程。（详细步骤请查看本文“导入已有工程章节”）



3. 编译工程，编译后生成 fs_assembly_ledpak.bin。

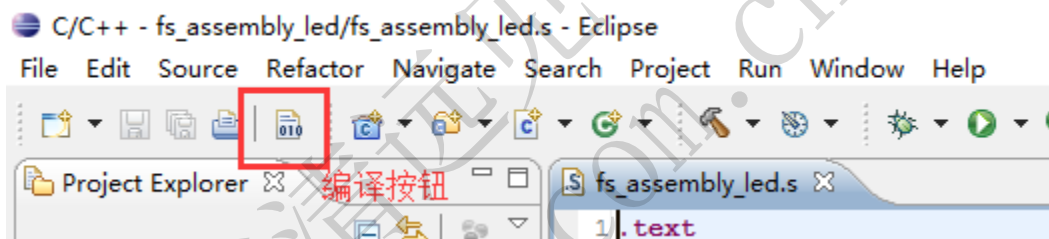


图 工程编译按钮

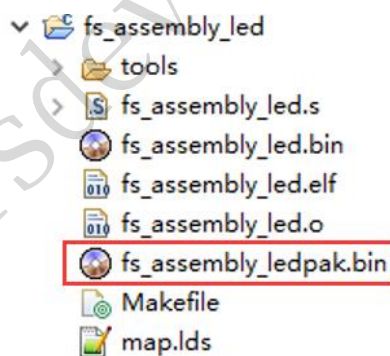


图 编译后的工程

4. 通过读卡器将 SD 卡连接至电脑。
5. 打开【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:工具软件\ARM 裸机开发工具\华清远见 SD 卡烧写软件\华清远见 SD 卡烧写软件.exe】软件，请注意必须使用管理员权限打开,如用户为 windows xp 系统，请打开【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:工具软件\ARM 裸机开发工具\华清远见 SD 卡烧写软件\windows_xp\ DiskImager.exe】软件。



使用制作启动盘下 ‘...’ 选项，选择用户对应工程编译生成的***pak.bin** 文件，我的工作目录为【E:\eclipse_projects\】



盘符会自动弹出电脑连接的可移动设备，也可以自己选择到烧写的 SD 卡所在盘符，盘符选择完成后在制作启动盘选项中选择生成的***pak.bin** 文件，点击烧写即可。

6. 拔出 SD 卡，插入到开发板的 SD 卡槽内，同时将拨码开关拨到【000】的位置，然后启动开发板，即可看到开发板的 RGB 灯红灯在闪烁。

5.2 蜂鸣器实验

5.2.1 【实验目的】

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 如何使用 pwm 的方式控制蜂鸣器

5.2.2 【实验环境】

- (1) FS6818 开发平台
- (2) Eclipse 集成开发环境
- (3) PC 机 windows 操作系统

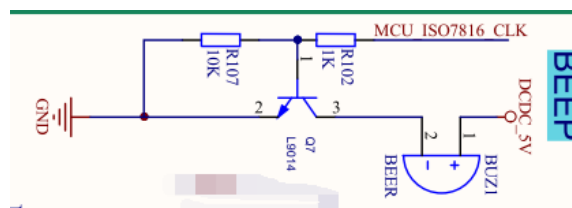


5.2.3 【实验原理】

一般的蜂鸣器分为有源和无源，这里的“源”不是指电源，而是指震荡源。有源蜂鸣器内部自带震荡源，理论上通电就可以发声，所以可以通过 PWM 波和简单的 IO 引脚的直流信号两种方式控制；无源蜂鸣器内部不带震荡源，普通的 IO 引脚的直流信号无法令其发声，必须使用 PWM 波形才能令其发声。

FS6818 平台板载了一个有源蜂鸣器，有源蜂鸣器有内部驱动电路，但也可以通过 PWM 波驱动蜂鸣器工作。在实例中，驱动方式为通过 PWM 控制。通过控制 pwm 输出不同的波形来调节蜂鸣器的响度。

硬件连接如下：



蜂鸣器硬件连接图

底板引脚	核心板引脚
MCU_ISO7816_CLK	PWM2

编程人员通过配置 pwm2 输出方波即可。

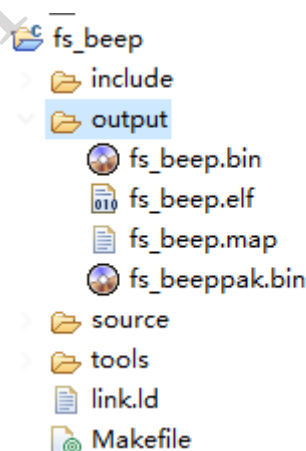
5.2.4 【实验步骤】

1. 硬件连接，需接好电源线，将启动方式调整为 SD 卡启动，接好调试串口并配置好串口调试工具。
2. 用户确认已经将 fs_beep 工程导入到 eclipse 开发环境中。

工程路径【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:程序源码\ARM 裸机实验源码】（详细步骤请查看本文“导入已有工程章节”）

3. 编译工程，编译后生成 fs_beep.pak.bin。

工程目录结构如图：



其中 source 下对应的是源码，source 下的 main.c 为主程序，output 目录下存放编译生成的 fs_beep.pak.bin 文件，tools 下存放的是制作*pak.bin 文件的工具。

4. 按照汇编点灯实验的方式制作 SD 卡，制作完成后插入实验平台，并重新上电，可以听到蜂鸣器鸣响。



5.3 RGB 彩灯实验

5.3.1 【实验目的】

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 如何使用 GPIO 控制 LED 灯

5.3.2 【实验环境】

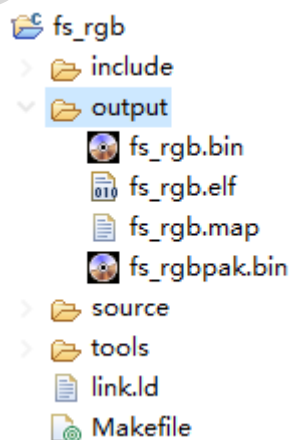
- (1) FS6818 开发平台
- (2) Eclipse 集成开发环境
- (3) PC 机 windows 操作系统

5.3.3 【实验原理】

实验原理与汇编点灯的原理一致，只是使用方法不同。

5.3.4 【实验步骤】

1. 硬件连接，需接好电源线，将启动方式调整为 SD 卡启动，接好调试串口并配置好串口调试工具。
2. 用户确认已经将 fs_rgb 工程导入到 eclipse 开发环境中。
工程路径【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:程序源码\ARM 裸机实验源码】（详细步骤请查看本文“导入已有工程章节”）
3. 编译工程，编译后生成 fs_rgbpak.bin。
工程目录结构如图：



其中 source 下对应的是源码，source 下的 main.c 为主程序，output 目录下存放编译生成的 fs_rgbpak.bin 文件，tools 下存放的是制作*pak.bin 文件的工具。

4. 按照汇编点灯实验的方法制作 SD 卡，制作完成后插入实验平台，并重新上电，可以看到 RGB 彩



灯不停变换红蓝绿灯。

5.4 串口输出实验

5.4.1 【实验目的】

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 了解串口的的基本配置方法

5.4.2 【实验环境】

- (1) FS6818 开发平台
- (2) Eclipse 集成开发环境
- (3) PC 机 windows 操作系统

5.4.3 【实验原理】

串口是开发板和开发人员之间进行交互必不可少的，所以用户需要了解如何初始化串口，如何配置串口能够让串口打印出用户需要的数据，该实验通过配置串口，可以使串口正常输出。

串口是计算机上一种非常通用设备通信的协议（不要与通用串行总线 Universal Serial Bus 或者 USB 混淆）。大多数计算机包含两个基于 RS232 的串口。串口同时也是仪器仪表设备通用的通信协议；很多 GPIB(通用接口总线)兼容的设备也带有 RS-232 口。同时，串口通信协议也可以用于获取远程采集设备的数据。

串口通信的概念非常简单，串口按位（bit）发送和接收字节。尽管比按字节（byte）的并行通信慢，但是串口可以在使用一根线发送数据的同时用另一根线接收数据。它很简单并且能够实现远距离通信。典型地，串口用于 ASCII 码字符的传输。通信使用 3 根线完成：（1）地线，（2）发送，（3）接收。由于串口通信是异步的，端口能够在在一根线上发送数据同时在另一根线上接收数据。其他线用于握手，但是不是必须的。串口通信最重要的参数是波特率、数据位、停止位和奇偶校验。

对于两个进行通行的端口，这些参数必须匹配：

波特率： 这是一个衡量通信速度的参数。它表示每秒钟传送的 bit 的个数。例如 300 波特表示每秒钟发送 300 个 bit。当我们提到时钟周期时，我们就是指波特率例如如果协议需要 4800 波特率，那么时钟是 4800Hz。这意味着串口通信在数据线上的采样率为 4800Hz。通常电话线的波特率为 14400，28800 和 36600。波特率可以远远大于这些值，但是波特率和距离成反比。高波特率常常用于放置的很近的仪器间的通信，典型的例子就是 GPIB 设备的通信。

数据位： 这是衡量通信中实际数据位的参数。当计算机发送一个信息包，实际的数据不会是 8 位的，标准的值是 5、7 和 8 位。如何设置取决于你想传送的信息。比如，标准的 ASCII 码是 0~127（7 位）。扩展的 ASCII 码是 0~255（8 位）。如果数据使用简单的文本（标准 ASCII 码），那么每个



数据包使用 7 位数据。每个包是指一个字节，包括开始/停止位，数据位和奇偶校验位。由于实际数据位取决于通信协议的选取，术语“包”指任何通信的情况。

停止位： 用于表示单个包的最后一位。典型的值为 1，1.5 和 2 位。由于数据是在传输线上定时的，并且每一个设备有其自己的时钟，很可能在通信中两台设备间出现了小小的不同步。因此停止位不仅仅是表示传输的结束，并且提供计算机校正时钟同步的机会。适用于停止位的位数越多，不同时钟同步的容忍程度越大，但是数据传输率同时也越慢。

奇偶校验位：在串口通信中一种简单的检错方式。有四种检错方式：偶、奇、高和低。当然没有校验位也是可以的。对于偶和奇校验的情况，串口会设置校验位（数据位后面的一位），用一个值确保传输的数据有偶个或者奇个逻辑高位。例如，如果数据是 011，那么对于偶校验，校验位为 0，保证逻辑高的位数是偶数个。如果是奇校验，校验位为 1，这样就有 3 个逻辑高位。高位和低位不真正的检查数据，简单置位逻辑高或者逻辑低校验。这样使得接收设备能够知道一个位的状态，有机会判断是否有噪声干扰了通信或者是否传输和接收数据是否不同步。

硬件流控制： 硬件流控制常用的有 RTS/CTS 流控制和 DTR/R（数据终端就绪/数据设置就绪）流控制。硬件流控制必须将相应的电缆线连上，用 RTS/CTS（请求发送/清除发送）流控制时，应将通讯两端的 RTS、CTS 线对应相连，数据终端设备（如计算机）使用 RTS 来起始调制解调器或其它数据通讯设备的数据流，而数据通讯设备（如调制解调器）则用 CTS 来起动和暂停来自计算机的数据流。这种硬件握手方式的过程为：我们在编程时根据接收端缓冲区大小设置一个高位标志（可为缓冲区大小的 75%）和一个低位标志（可为缓冲区大小的 25%），当缓冲区内数据量达到高位时，我们在接收端将 CTS 线置低电平（送逻辑 0），当发送端的程序检测到 CTS 为低后，就停止发送数据，直到接收端缓冲区的数据量低于低位而将 CTS 置高电平。RTS 则用来标明接收设备有没有准备好接收数据。

常用的流控制还有 DTR/R（数据终端就绪/数据设置就绪）。我们在此不再详述。

- 101 -

36600。波特率可以远远大于这些值，但是波特率和距离成反比。高波特率常常用于放置的很近的仪器间

的通信，典型的例子就是 GPIB 设备的通信。

数据位： 这是衡量通信中实际数据位的参数。当计算机发送一个信息包，实际的数据不会是 8 位的，标准的值是 5、7 和 8 位。如何设置取决于你想传送的信息。比如，标准的 ASCII 码是 0~127（7

位）。扩展的 ASCII 码是 0~255（8 位）。如果数据使用简单的文本（标准 ASCII 码），那么每个数据包

使用 7 位数据。每个包是指一个字节，包括开始/停止位，数据位和奇偶校验位。由于实际数据位取决于

通信协议的选取，术语“包”指任何通信的情况。

停止位： 用于表示单个包的最后一位。典型的值为 1，1.5 和 2 位。由于数据是在传输线上定时的

的，并且每一个设备有其自己的时钟，很可能在通信中两台设备间出现了小小的不同步。因此停止



位不

仅仅是表示传输的结束，并且提供计算机校正时钟同步的机会。适用于停止位的位数越多，不同时钟同步

步的容忍程度越大，但是数据传输率同时也越慢。

奇偶校验位：在串口通信中一种简单的检错方式。有四种检错方式：偶、奇、高和低。当然没有校验位也是可以的。对于偶和奇校验的情况，串口会设置校验位（数据位后面的一位），用一个值确保传输

的数据有偶个或者奇个逻辑高位。例如，如果数据是 011，那么对于偶校验，校验位为 0，保证逻辑高的

位数是偶数个。如果是奇校验，校验位为 1，这样就有 3 个逻辑高位。高位和低位不真正的检查数据，

简单置位逻辑高或者逻辑低校验。这样使得接收设备能够知道一个位的状态，有机会判断是否有噪声干扰

扰了通信或者是否传输和接收数据是否不同步。

硬件流控制：硬件流控制常用的有 RTS/CTS 流控制和 DTR/R（数据终端就绪/数据设置就绪）流控制

制。硬件流控制必须将相应的电缆线连上，用 RTS/CTS（请求发送/清除发送）流控制时，应将通讯两端

的 RTS、CTS 线对应相连，数据终端设备（如计算机）使用 RTS 来起始调制解调器或其它数据通讯设备

的数据流，而数据通讯设备（如调制解调器）则用 CTS 来起动和暂停来自计算机的数据流。这种硬件握

手方式的过程为：我们在编程时根据接收端缓冲区大小设置一个高位标志（可为缓冲区大小的 75%）和

一个低位标志（可为缓冲区大小的 25%），当缓冲区内数据量达到高位时，我们在接收端将 CTS 线置低

电平（送逻辑 0），当发送端的程序检测到 CTS 为低后，就停止发送数据，直到接收端缓冲区的数据量低

于低位而将 CTS 置高电平。RTS 则用来标明接收设备有没有准备好接收数据。

常用的流控制还有 DTR/R（数据终端就绪/数据设置就绪）。我们在此不再详述。

S5P6818 串口控制器概述：

1. 简述

通用异步收发（UART）可支持 5 个独立的异步串行输入/输出口，每个口皆可支持中断模式及 DMA 模式，UART 可产生一个中断或者发出一个 DMA 请求，来传送 CPU 与 UART 之间的数据，UART 的比特率最大可达到 4Mbps。每一个 UART 通道包含两 FIFOs 用于数据的收发，其中通道 0 的 FIFO 大小为 256 字节，通道 1、4 的 FIFO 大小为 64 字节，通道 2、3 的 FIFO 大小为 16 字



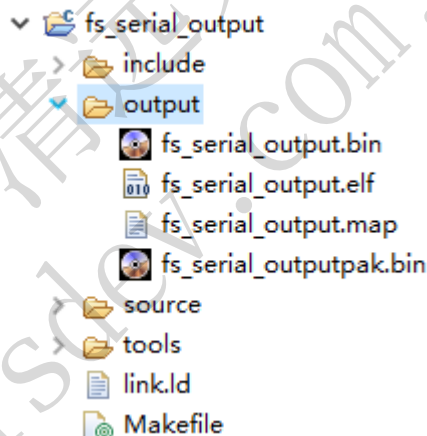
节。

2. 特点

- (1) 5 组收发通道，同时支持中断模式及 DMA 操作。
- (2) 通道 0、1、2 及带红外通道 3。
- (3) 通道 0 带 256byte 的 FIFO，通道 1、4 带 64 byte 的 FIFO，通道 2、3 带 16byteFIFO。通道 0、1、2 支持自动流控功能
- (4) 支持握手模式的发送/接收。

5.4.4 【实验步骤】

1. 硬件连接，需接好电源线，将启动方式调整为 SD 卡启动，接好调试串口并配置好串口调试工具（这里的工具使用 putty 或者 UartAssist 均可）。
2. 用户确认已经将 fs_serial_output 工程导入到 eclipse 开发环境中。
工程路径【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:程序源码\ARM 裸机实验源码】（详细步骤请查看本文“导入已有工程章节”）
3. 编译工程，编译后生成 fs_serial_outputpak.bin。
工程目录结构如图：



其中 source 下对应的是源码，source 下的 main.c 为主程序，output 目录下存放编译生成的 fs_serial_outputpak.bin 文件，tools 下存放的是制作*pak.bin 文件的工具。

4. 按照汇编点灯实验的方法制作 SD 卡，制作完成后插入实验平台，并重新上电.在串口调试工具中可以看到如下图的打印信息。



```

Loading from sdmmc...
Image Loading Done!
Launch to 0x0000000043C00000
s5p6818 serial0 stdio test ...
i_num = 0, f_num = 3.141594
i_num = 1, f_num = 3.141595
i_num = 2, f_num = 3.141596
i_num = 3, f_num = 3.141597
i_num = 4, f_num = 3.141598
i_num = 5, f_num = 3.141599
i_num = 6, f_num = 3.141600
i_num = 7, f_num = 3.141601
i_num = 8, f_num = 3.141602
    
```

5.5 串口输入实验

5.5.1 【实验目的】

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 了解串口的基本配置方法

5.5.2 【实验环境】

- (1) FS6818 开发平台
- (2) Eclipse 集成开发环境
- (3) PC 机 windows 操作系统

5.5.3 【实验原理】

实验原理与串口输出实验一致。

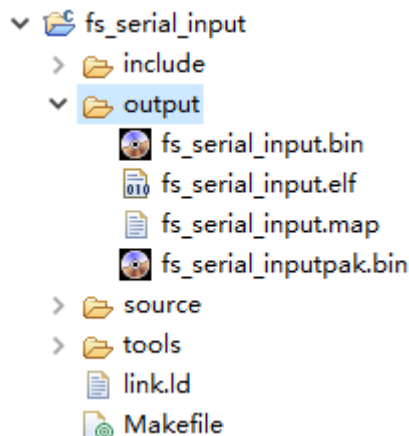
5.5.4 【实验步骤】

1. 硬件连接，需接好电源线，将启动方式调整为 SD 卡启动，接好调试串口并配置好串口调试工具（这里的工具使用 putty 或者 UartAssist 均可）。
2. 用户确认已经将 fs_serial_input 工程导入到 eclipse 开发环境中。

工程路径【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:程序源码\ARM 裸机实验源码】（详细步骤请查看本文“导入已有工程章节”）



3. 编译工程，编译后生成 fs_serial_inputpak.bin。
工程目录结构如图：



其中 source 下对应的是源码，source 下的 main.c 为主程序，output 目录下存放编译生成的 fs_serial_inputpak.bin 文件，tools 下存放的是制作*.pak.bin 文件的工具。

4. 按照汇编点灯实验的方法制作 SD 卡，制作完成后插入实验平台，并重新上电，在串口工具观察到等待用户输入，即可在串口调试助手输入单个字符，之后串口会回显出用户输入的字符。

```
s5p6818 serial0 echo test, please type some character...
serial0 recv: [1]
serial0 recv: [a]
serial0 recv: [d]
serial0 recv: [s]
serial0 recv: [g]
serial0 recv: [g]
```

5.6 复位实验

5.6.1 【实验环境】

- (1) FS6818 开发平台
- (2) Eclipse 集成开发环境
- (3) PC 机 windows 操作系统

5.6.2 【实验目的】

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 了解如何通过软复位的方式控制开发板的启动

5.6.3 【实验原理】

如何通过程序操控 PWRCONT 和 PWRMODE 寄存器控制开发板的复位功能。



4.9.1.29 PWRCONT

- Base Address: 0xC001_0000
- Address = Base Address + 0224h, Reset Value = 0x0000_FF00

Name	Bit	Type	Description	Reset Value
RSVD	[31:16]	R	Reserved	16'h0
USE_WFI	[15:12]	RW	Use STANDBYWFI[n] signal as indicating signal to go into STOP mode.	4'hF
USE_WFE	[11:8]	RW	Use STANDBYWFE[n] signal as indicating signal to go into STOP mode.	4'hF
RSVD	[7:5]	R	Reserved	3'b000
XTAL_PWRDN	[4]	RW	X-tal power down mode selection This controls the power down of X-tal-PAD in stop-mode. 0 = XTAL is powered down in STOP mode 1 = XTAL is not powered down in STOP mode	1'b0
SWRSTENB	[3]	RW	Software Reset Enable. 0 = Disable 1 = Enable	1'b0
RSVD	[2]	RW	Reserved (This bit always should be "0")	1'b0
RTCWKENB	[1]	RW	RTC Wake-up enable 0 = Disable 1 = Enable	1'b0
RSVD	[0]	RW	Reserved (This bit always should be "0")	1'b0

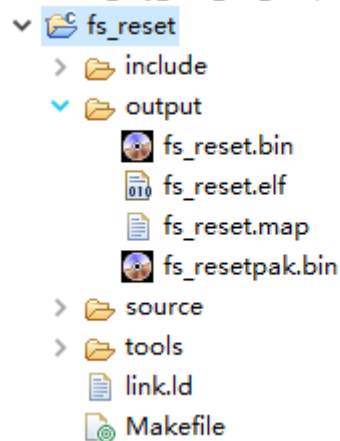
4.9.1.30 PWRMODE

- Base Address: 0xC001_0000
- Address = Base Address + 0228h, Reset Value = 0x0000_0000

Name	Bit	Type	Description	Reset Value
RSVD	[31:16]	RW	Reserved	16'h0
CHGPLL	[15]	RW	Change PLL Value with new value defined in PLL Setting Register (PLL0set, PLL1set) in clock Controller. Read 0 = Stable 1 = PLL is Unstable Write 0 = None 1 = PLL Value Change	1'b0
RSVD	[14:13]	-	Reserved	2'b00
SWRST	[12]	W	This bit is cleared after Software Reset 0 = Do Not Reset 1 = Go to Reset	1'b0

5.6.4 【实验步骤】

1. 硬件连接，需接好电源线，将启动方式调整为 SD 卡启动，接好调试串口并配置好串口调试工具（这里的工具使用 putty 或者 UartAssist 均可）。
2. 用户确认已经将 fs_reset 工程导入到 eclipse 开发环境中。
工程路径【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:程序源码\ARM 裸机实验源码】（详细步骤请查看本文“导入已有工程章节”）
3. 编译工程，编译后生成 fs_resetpak.bin。
工程目录结构如图：



其中 source 下对应的是源码，source 下的 main.c 为主程序，output 目录下存放编译生成的 fs_resetpak.bin 文件，tools 下存放的是制作*.pak.bin 文件的工具。

4. 按照汇编点灯实验的方法制作 SD 卡，制作完成后插入实验平台，并重新上电，可以在串口工具中看到重置倒计时，当倒计时为 0 后系统重启。

5.6.5 【实验目的】

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 了解 ARM 异常及中断处理

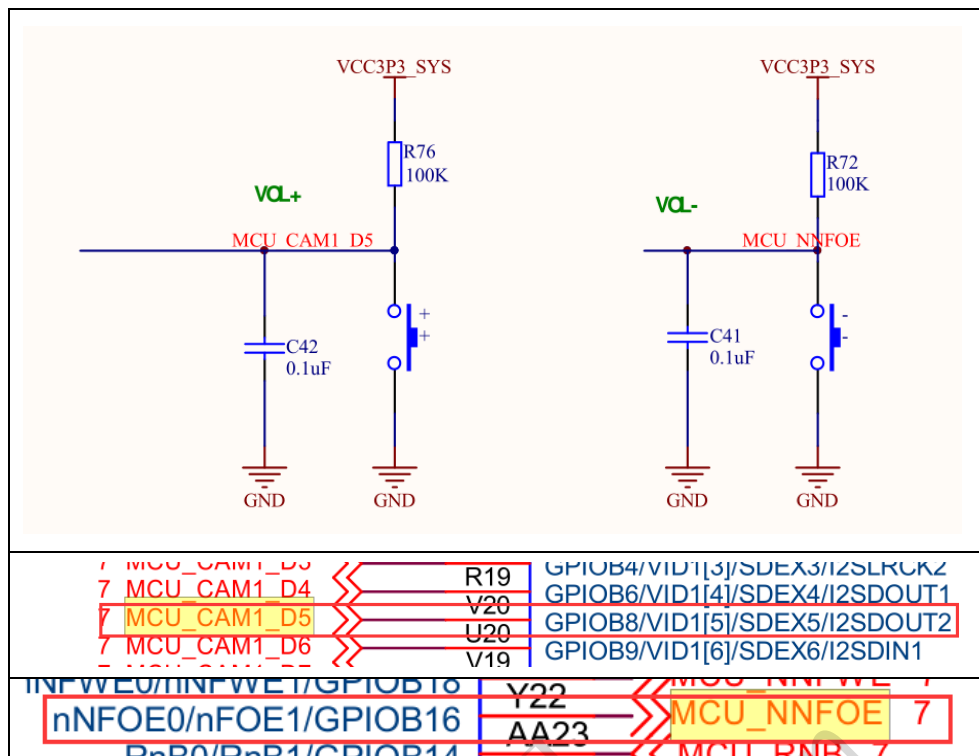
5.7 按键中断实验

5.7.1 【实验环境】

- (1) FS6818 开发平台
- (2) Eclipse 集成开发环境
- (3) PC 机 windows 操作系统

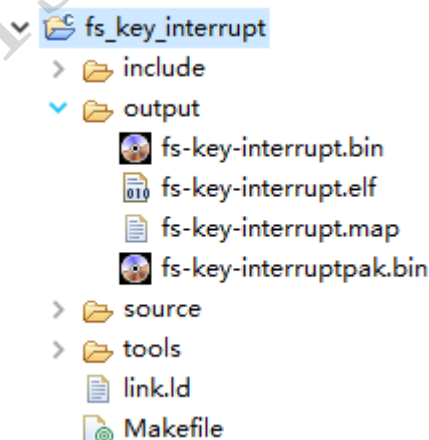
5.7.2 【实验原理】

本实验中通过按键中断实现 rgb 灯的点亮，首先需要将按键 GPIO 引脚配置为中断模式，并设置中断触发方式，分别使能中断控制器及中断源。如下图所示为实验两按键原理图，分别为 GPIOB_8 和 GPIOB_16。



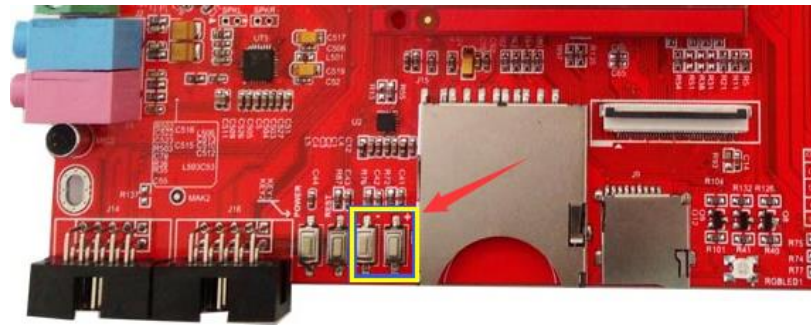
5.7.3 【实验步骤】

1. 硬件连接，需接好电源线，将启动方式调整为 SD 卡启动，接好调试串口并配置好串口调试工具（这里的工具使用 putty 或者 UartAssist 均可）。
2. 用户确认已经将 fs_key_interrupt 工程导入到 eclipse 开发环境中。
工程路径【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:程序源码\ARM 裸机实验源码】（详细步骤请查看本文“导入已有工程章节”）
3. 编译工程，编译后生成 fs_key_interrupt.bin，工程目录结构如图：



其中 source 下对应的是源码，source 下的 main.c 为主程序，output 目录下存放编译生成的 fs_key_interrupt.bin 文件，tools 下存放的是制作*pak.bin 文件的工具。

4. 按照汇编点灯实验的方法制作 SD 卡，制作完成后插入实验平台，并重新上电，按下如下两按键可观察到 RGB 灯被点亮。



5.8 按键轮询实验

5.8.1 【实验目的】

- (1) 熟悉 Eclipse 工具的使用
- (2) 能够导入已有工程
- (3) 能够掌握裸机实验的基本操作步骤
- (4) 通过本实验了解如何使用 C 语言编写裸机程序
- (5) 了解如何通过轮询的方式得到按键状态

5.8.2 【实验环境】

- (1) FS6818 开发平台
- (2) Eclipse 集成开发环境
- (3) PC 机 windows 操作系统

5.8.3 【实验原理】

外部设备与中央处理器交互一般有两种手段：轮询和中断。

(1) 轮询 (Polling)

很多 I/O 设备都有一个状态寄存器，用于描述设备当前的工作状态，每当设备状态发生改变时，设备将修改相应状态寄存器位。通过不断查询设备的状态寄存器，CPU 就可以了解设备的状态，从而进行必要的 I/O 操作。为了节约 CPU 资源，查询工作往往不是连续的，而是定时进行。

轮询方式具有简单、易实现、易控制等优势，在很多小型系统中有大量应用。对那些实时敏感性不高、具有大量 CPU 资源的系统来说，轮询方式有很广泛的应用。最典型的用途就是在那些任务比较单一的单片机上，嵌入式系统中也有应用。

轮询的一种典型的实现可能是这样的：`while(TRUE){/.../ select(,timeout); /.../};`当然这里的 `select()` 也可以使用 `poll()` 替换。

轮询方式主要存在以下不足：

<1>增加系统开销。无论是任务轮询还是定时器轮询都需要消耗对应的系统资源。

<2>无法及时感知设备状态变化。在轮询间隔内的设备状态变化只有在下次轮询时才能被发现，这将无法满足对实时性敏感的应用场合。



<3>浪费 CPU 资源。无论设备是否发生状态改变，轮询总在运行。在实际情况中，大多数设备的状态改变通常不会那么频繁，轮询空转将白白浪费 CPU 时间片。

(2) 中断 (Interrupt)

中断，顾名思义，就是打断正在进行中的工作。中断不需要处理器轮询设备的状态，设备在自己发生状态改变时将主动发送一个信号给处理器 (PIC)，后者在接收到这一通知信号时，会挂起当前正在执行的任务转而去处理响应外设的中断请求。中断通知机制通过硬件信号异步唤起处理器的注意，解决了外部设备与处理器之间速度不匹配导致的资源浪费问题。

原理图通按键中断实验相同，此处不再赘述，编程人员通过配置两按键为 GPIO 输入模式即可。

。

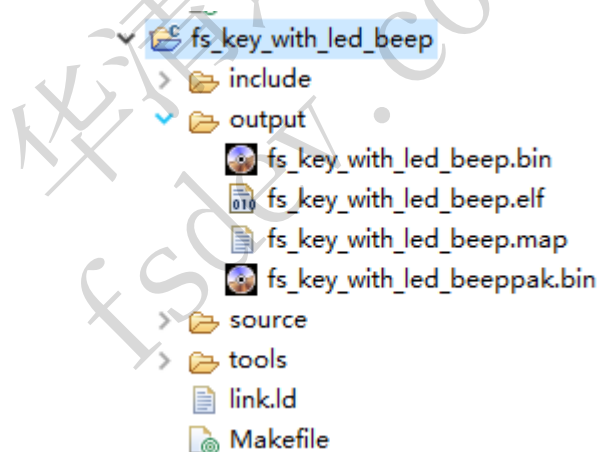
5.8.4 【实验步骤】

1. 硬件连接，需接好电源线，将启动方式调整为 SD 卡启动，接好调试串口并配置好串口调试工具（这里的工具使用 putty 或者 UartAssist 均可）。

2. 用户确认已经将 fs_key_with_led_beep 工程导入到 eclipse 开发环境中。

工程路径【华清远见-嵌入式 ARM 实验箱资料-I-FS6818:程序源码\ARM 裸机实验源码】（详细步骤请查看本文“导入已有工程章节”）

3. 编译工程，编译后生成 fs_key_with_led_beep.pak.bin。
工程目录结构如图：



其中 source 下对应的是源码，source 下的 main.c 为主程序，output 目录下存放编译生成的 fs_key_with_led_beep.pak.bin 文件，tools 下存放的是制作*.pak.bin 文件的工具。

4. 按照汇编点灯实验的方法制作 SD 卡，制作完成后插入实验平台，并重新上电，按下如下两按键可观察到 RGB 灯被点亮并且蜂鸣器鸣响。





0x2A: CTRL_REG1 System Control 1 Register

0x2A CTRL_REG1 Register (Read/Write)

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
ASLP_RATE1	ASLP_RATE0	DR2	DR1	DR0	LNOISE	F_READ	ACTIVE

Table 60. CTRL_REG1 Description

ASLP_RATE[1:0]	Configures the Auto-WAKE sample frequency when the device is in SLEEP Mode. Default value: 00. See Table 61 for more information.
DR[2:0]	Data rate selection. Default value: 000. See Table 62 for more information.
LNOISE	Reduced noise reduced Maximum range mode. Default value: 0. (0: Normal mode; 1: Reduced Noise mode)
F_READ	Fast Read mode: Data format limited to single Byte Default value: 0. (0: Normal mode 1: Fast Read Mode)
ACTIVE	Full Scale selection. Default value: 00. (0: STANDBY mode; 1: ACTIVE mode)

0x2B: CTRL_REG2 System Control 2 Register

0x2B CTRL_REG2 Register (Read/Write)

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
ST	RST	0	SMODS1	SMODS0	SLPE	MODS1	MODS0

Table 64. CTRL_REG2 Description

ST	Self-Test Enable. Default value: 0. 0: Self-Test disabled; 1: Self-Test enabled
RST	Software Reset. Default value: 0. 0: Device reset disabled; 1: Device reset enabled.
SMODS[1:0]	SLEEP mode power scheme selection. Default value: 00. See Table 65 and Table 66
SLPE	Auto-SLEEP enable. Default value: 0. 0: Auto-SLEEP is not enabled; 1: Auto-SLEEP is enabled.
MODS[1:0]	ACTIVE mode power scheme selection. Default value: 00. See Table 65 and Table 66

5.9.3 【实验内容】

编写程序，实现对重力感应芯片 mma8451 的坐标读操作

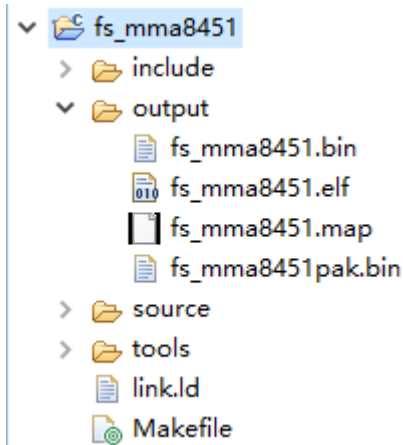
- 配置 GPIO 为 I2C 模式
- 写 I2C 设备地址
- 写 I2C 数据地址
- 读/写 I2C 芯片 mma8451 的寄存器数据

5.9.4 【实验步骤】

- 硬件连接，需接好电源线，将启动方式调整为 SD 卡启动，接好调试串口并配置好串口调试工具（这里的工具使用 putty 或者 UartAssist 均可）。
- 用户确认已经将 fs_mma8451 工程导入到 eclipse 开发环境中。
工程路径【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:程序源码\ARM 裸机实验源码】（详细步骤请查看本文“导入已有工程章节”）
- 编译工程，编译后生成 fs_mma8451pak.bin。



工程目录结构如图：



其中 source 下对应的是源码，source 下的 main.c 为主程序，output 目录下存放编译生成的 fs_mma8451pak.bin 文件，tools 下存放的是制作*.pak.bin 文件的工具。

按照汇编点灯实验的方法制作 SD 卡，制作完成后插入实验平台，并重新上电即可看到 mma8451 上报的 x,y,z 轴的数值。

5.9.5 【实验现象】

实验现象如图所示：通过串口终端打印出陀螺仪数值。转动电路板，观察变化规律。

```
COM3 - PuTTY
gray_x = 256
gray_y = -6144
gray_z = 0
*****
gray_x = 256
gray_y = 0
gray_z = -16
*****
gray_x = -768
gray_y = -255
gray_z = 16384
*****
gray_x = -4096
gray_y = -63
gray_z = 0
*****
gray_x = 12288
gray_y = 0
gray_z = 16128
*****
gray_x = -768
gray_y = 0
gray_z = 255
```

5.10 A/D 实验

5.10.1 【实验目的】

- 掌握模数转换 ADC 的原理；



- 掌握 S5P6818 处理器的 A/D 转换功能；

5.10.2 【实验原理】

如图所示，ADC 电路连接如图所示，利用一个电位计输出电压到 S5P6818 的 ADC0 引脚。输入的电压范围是 0~1.8V。旋转电位器 XR 使 ADC0 和 GND 两端的电压发生变化，即 ADC0 引脚采集变化的模拟电压，即 ADC 控制器的数据寄存器可输出对应二进制数值。

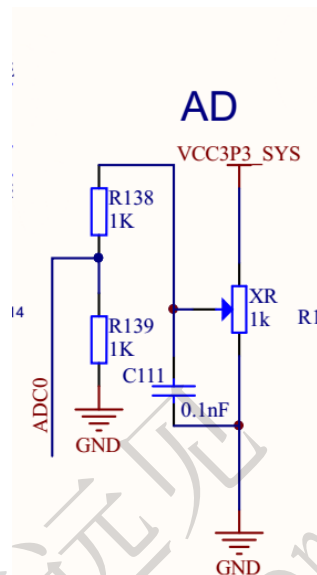


图 分压电路

5.10.3 【实验内容】

- (1) 寄存器设置：
 - a) 上电使能 ADC 控制器
 - b) 选择转换通道。
 - c) 使能 ADC 读操作和设置分频因子
 - d) 设置工作模式和转换分辨率

编写软件程序，实现电压值的获取、显示。程序主要是对 S5P6818 中的 A/D 模块进行操作，所以软件程序也主要是对 A/D 模块中的寄存器进行操作，其中包括对 ADC 控制寄存器（ADCCON）、ADC 数据寄存器（ADCDAT）的读/写操作。同时为了观察转换结果，可以通过串口在超级终端里面观察。

5.10.4 【实验步骤】

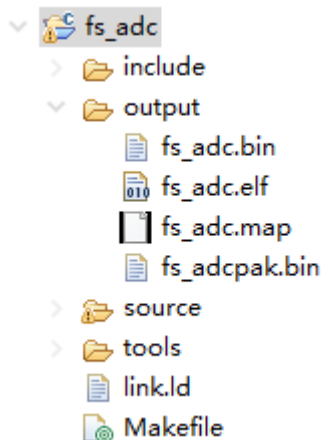
1. 硬件连接，需接好电源线，将启动方式调整为 SD 卡启动，接好调试串口并配置好串口调试工具（这里的工具使用 putty 或者 UartAssist 均可）。
2. 用户确认已经将 fs_adcpak.bin 工程导入到 eclipse 开发环境中。

工程路径【华清远见-嵌入式 ARM 实验箱资料- I -FS6818:程序源码\ARM 裸机实验源码】（详细步



骤请查看本文“导入已有工程章节”)

3. 编译工程，编译后生成 fs_adcpak.bin。
工程目录结构如图：



其中 source 下对应的是源码，source 下的 main.c 为主程序，output 目录下存放编译生成的 fs_adcpak.bin 文件，tools 下存放的是制作*.pak.bin 文件的工具。

按照汇编点灯实验的方法制作 SD 卡，制作完成后插入实验平台，并重新上电即可看到 ADC 采集到的 ADC 值。

5.10.5 【实验现象】

通过串口中断观察打印信息，旋转开发板上的电位器，观察电压的变化。

```

data = 1748
data = 1748
data = 1743
data = 1745
data = 1743
data = 1738
data = 1745
data = 1740
data = 1744
data = 1747
data = 1748
data = 1746
data = 1749
data = 1744
data = 1745
data = 1747
    
```

图 ADC 转换的数据